Assumptions that were never used: the catalogue

Companion to the note. Generated, not written.

September 4, 2026

What this is

Every one of the 645 survivors of a mechanical search across 19 areas of Mathlib: theorems whose stated algebraic setting is stronger than their own proof requires. Each entry gives the declaration, the module it lives in, the assumption as stated, and the weakest class at which the original proof still compiles.

The signature shown is what Lean prints for the theorem as the library has it, not our reconstruction of it.

Two instruments. All areas but CategoryTheory were swept with a hand-written table of 48 weakenings. That table names no categorical class, so CategoryTheory was swept with one derived from the library's own `class … extends` declarations. The two are not interchangeable and the areas are not comparable to each other. **Two kinds of entry.** An entry marked ‡ is one whose weakened proof is *not* the original proof: re-elaborated without the assumption, a tactic searched again and found another route, so the assumption was *replaceable* rather than unused. An unmarked entry is the same proof term modulo the substituted class — the assumption was never used at all. Of the entries that could be classified, 87.5% are unused and 12.5% replaceable. An entry marked ? could not be classified because the original declaration is `private` and cannot be named from outside its own file; those entries verify like any other and are findings, not failures.

Nothing here is filtered for interest. The search establishes that a proof survives a weaker hypothesis; it cannot establish that the weaker statement is worth having. Some entries are auxiliary lemmas whose conclusion barely involves the weakened structure, and they are listed alongside the rest because dropping them would claim a judgement the machine did not make. *Depth is not worth.*

Contents

Algebra

201 survivors in 29 families; 18 give up subtraction or division (†).

stated over	holds over	n	magnitude
`CommMonoidWithZero`	`CommMonoid`	2	4
`Semiring`	`AddCommMonoid`	2	4
`IsDomain`	`NoZeroDivisors`	6	3
`Field`	`CommRing`	5	3
`Monoid`	`MulOneClass`	20	2
`LinearOrder`	`PartialOrder`	12	2
`AddCommGroup` †	`AddCommMonoid`	10	2
`CommSemigroup`	`CommMagma`	6	2
`Ring` †	`Semiring`	6	2
`AddCommSemigroup`	`AddCommMagma`	2	2
`AddGroup` †	`AddMonoid`	2	2
`AddGroup`	`SubNegMonoid`	2	2
`Group`	`DivInvMonoid`	2	2
`CommMonoid`	`CommSemigroup`	1	2
`PartialOrder`	`Preorder`	57	1
`Semiring`	`NonUnitalSemiring`	13	1
`Field`	`DivisionRing`	12	1
`CommSemiring`	`Semiring`	9	1
`CommMonoid`	`Monoid`	6	1
`AddCommMonoid`	`AddMonoid`	5	1
`AddZeroClass`	`AddZero`	5	1
`CommRing`	`Ring`	5	1
`MulOneClass`	`MulOne`	3	1
`AddCommGroup`	`AddGroup`	1	1
`NonUnitalSemiring`	`NonUnitalNonAssocSemiring`	1	1
`StarRing`	`StarMul`	3	-1
`CancelMonoid`	`LeftCancelMonoid`	1	-1
`IsDomain`	`Nontrivial`	1	-1
`StarAddMonoid`	`InvolutiveStar`	1	-1

CommMonoidWithZero → CommMonoid (2, magnitude 4)

Algebra · GroupWithZero · Associated

`Associates.irreducible_mk`

`∀ {M : Type*} [CommMonoidWithZero M] {a : M}, Irreducible (Associates.mk a) ↔ Irreducible a`

Algebra · Squarefree · Basic

`Squarefree.isRadical`

`∀ {R : Type*} [CommMonoidWithZero R] [DecompositionMonoid R] {x : R}, Squarefree x → IsRadical x`

Semiring → AddCommMonoid (2, magnitude 4)

Algebra · Order · Ring · IsNonarchimedean

`IsNonarchimedean.multiset_image_add`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {F : Type*} {α : Type u_3} {β : Type u_4} [AddCommMonoid α] [FunLike F α R] [ZeroHomClass F α R] [NonnegHomClass F α R] [Nonempty β] {f : F}, IsNonarchimedean ⇑f → ∀ (g : β → α) (s : Multiset β), ∃ b, (s ≠ 0 → b ∈ s) ∧ f (Multiset.map g s).sum ≤ f (g b)`

`IsNonarchimedean.nsmul_le`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {F : Type*} {α : Type u_3} [AddMonoid α] [FunLike F α R] [ZeroHomClass F α R] [NonnegHomClass F α R] {f : F}, IsNonarchimedean ⇑f → ∀ {n : ℕ} {a : α}, f (n • a) ≤ f a`

IsDomain → NoZeroDivisors (6, magnitude 3)

Algebra · Module · Card

`Cardinal.mk_le_of_module`

`∀ (R : Type u_1) (E : Type u_2) [AddCommGroup E] [Ring R] [IsDomain R] [Module R E] [Nontrivial E] [Module.IsTorsionFree R E], Cardinal.lift.{u_2, u_1} (Cardinal.mk R) ≤ Cardinal.lift.{u_1, u_2} (Cardinal.mk E)`

Algebra · Module · Torsion · Basic

`infinite_range_add_smul_iff`

`∀ {R M : Type*} [Ring R] [IsDomain R] [Infinite R] [AddCommGroup M] [Module R M] [Module.IsTorsionFree R M] (x y : M), (Set.range fun r => x + r • y).Infinite ↔ y ≠ 0`

Algebra · Order · Module · Defs

`SMulPosMono.toSMulPosStrictMono`

`∀ {α : Type u_1} {β : Type u_2} [Ring α] [AddCommGroup β] [Module α β] [PartialOrder α] [PartialOrder β] [IsDomain α] [Module.IsTorsionFree α β] [SMulPosMono α β], SMulPosStrictMono α β`

`SMulPosReflectLT.toSMulPosReflectLE`

`∀ {α : Type u_1} {β : Type u_2} [Ring α] [AddCommGroup β] [Module α β] [PartialOrder α] [PartialOrder β] [IsDomain α] [Module.IsTorsionFree α β] [SMulPosReflectLT α β], SMulPosReflectLE α β`

Algebra · Polynomial · Roots

`Polynomial.Monic.isUnit_leadingCoeff_of_dvd`

`∀ {R : Type*} [CommRing R] [IsDomain R] {a p : Polynomial R}, p.Monic → a □ p → IsUnit a.leadingCoeff`

Algebra · Polynomial · Sequence

`Polynomial.Sequence.linearIndependent` ‡

`∀ {R : Type*} [Ring R] (S : Polynomial.Sequence R) [IsDomain R], LinearIndependent R ⇑S`

Field → CommRing (5, magnitude 3)

Algebra · Algebra · Spectrum · Quasispectrum

`QuasispectrumRestricts.of_subset_range_algebraMap`

`∀ {R S A : Type*} [Semifield R] [Field S] [NonUnitalRing A] [Module R A] [Module S A] [Algebra R S] {a : A} {f : S → R}, Function.LeftInverse f ⇑(algebraMap R S) → quasispectrum S a ⊆ Set.range ⇑(algebraMap R S) → QuasispectrumRestricts a f`

Algebra · CharP · Algebra

`IsFractionRing.charP_of_isFractionRing`

`∀ (R : Type u_1) {K : Type*} [CommRing R] [Field K] [Algebra R K] [IsFractionRing R K] (p : ℕ) [CharP R p], CharP K p`

Algebra · Order · CauSeq · Basic

`CauSeq.rat_inf_continuous_lemma`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {ε a₁ a₂ b₁ b₂ : α}, |a₁ - b₁| < ε → |a₂ - b₂| < ε → |min a₁ a₂ - min b₁ b₂| < ε`

`CauSeq.rat_sup_continuous_lemma`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {ε a₁ a₂ b₁ b₂ : α}, |a₁ - b₁| < ε → |a₂ - b₂| < ε → |max a₁ a₂ - max b₁ b₂| < ε`

Algebra · Order · Floor · Semifield

`Mathlib.Meta.NormNum.IsRat.natFloor` ‡

`∀ {R : Type*} [Field R] [LinearOrder R] [IsStrictOrderedRing R] [FloorSemiring R] (r : R) (n d : ℕ), Mathlib.Meta.NormNum.IsRat r (Int.negOfNat n) d → Mathlib.Meta.NormNum.IsNat ⌊r⌋₊ 0`

Monoid → MulOneClass (20, magnitude 2)

Algebra · BigOperators · Group · List · Basic

`List.length_pos_of_prod_ne_one` ‡

`∀ {M : Type*} [Monoid M] (L : List M), L.prod ≠ 1 → 0 < L.length`

Algebra · BigOperators · Group · List · Defs

`List.prod_hom_rel`

`∀ {ι : Type u_1} {M N : Type*} [Monoid M] [Monoid N] (l : List ι) {r : M → N → Prop} {f : ι → M} {g : ι → N}, r 1 1 → (∀ □ i : ι□ □ a : M□ □ b : N□ , r a b → r (f i * a) (g i * b)) → r (List.map f l).prod (List.map g l).prod`

Algebra · Group · Hom · CompTypeclasses

`MonoidHom.CompTriple.comp` ‡

`∀ {M N P : Type*} [Monoid M] [Monoid N] [Monoid P] {φ : M →* N} {ψ : N →* P}, φ.CompTriple ψ (ψ.comp φ)`

Algebra · Group · Submonoid · Pointwise

`Submonoid.closure_mul_le` ‡

`∀ {M : Type*} [Monoid M] (S T : Set M), Submonoid.closure (S * T) ≤ Submonoid.closure S □ Submonoid.closure T`

`Submonoid.mul_subset`

`∀ {M : Type*} [Monoid M] {s t : Set M} {S : Submonoid M}, s ⊆ ↑S → t ⊆ ↑S → s * t ⊆ ↑S`

Algebra · Order · BigOperators · Group · List

`List.Forall₂.prod_le_prod'`

`∀ {M : Type*} [Monoid M] [Preorder M] [MulRightMono M] [MulLeftMono M] {l₁ l₂ : List M}, List.Forall₂ (fun x1 x2 => x1 ≤ x2) l₁ l₂ → l₁.prod ≤ l₂.prod`

`List.Sublist.prod_le_prod'`

`∀ {M : Type*} [Monoid M] [Preorder M] [MulRightMono M] [MulLeftMono M] {l₁ l₂ : List M}, l₁.Sublist l₂ → (∀ a ∈ l₂, 1 ≤ a) → l₁.prod`

`≤ l₂.prod`

`List.le_prod_nonempty_of_submultiplicative_on_pred`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [CommMonoid β] [Preorder β] [IsOrderedMonoid β] (f : α → β) (p : α → Prop), (∀ (a b : α), p a → p b → f (a * b) ≤ f a * f b) → (∀ (a b : α), p a → p b → p (a * b)) → ∀ (l : List α), l ≠ [] → (∀ a ∈ l, p a) → f l.prod ≤ (List.map f l).prod`

`List.le_prod_of_mem`

`∀ {M : Type*} [Monoid M] [Preorder M] [CanonicallyOrderedMul M] {xs : List M} {x : M}, x ∈ xs → x ≤ xs.prod`

`List.le_prod_of_submultiplicative_on_pred`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [CommMonoid β] [Preorder β] [IsOrderedMonoid β] (f : α → β) (p : α → Prop), f 1 ≤ 1 → p 1 → (∀ (a b : α), p a → p b → f (a * b) ≤ f a * f b) → (∀ (a b : α), p a → p b → p (a * b)) → ∀ (l : List α), (∀ a ∈ l, p a) → f l.prod ≤ (List.map f l).prod`

`List.one_le_prod_of_one_le`

`∀ {M : Type*} [Monoid M] [Preorder M] [MulLeftMono M] {l : List M}, (∀ x ∈ l, 1 ≤ x) → 1 ≤ l.prod`

Algebra · Order · Group · Indicator

`Set.mulIndicator_apply_le`

`∀ {α : Type u_1} {M : Type*} [Monoid M] [PartialOrder M] [CanonicallyOrderedMul M] {a : α} {s : Set α} {f g : α → M}, (a ∈ s → f a ≤ g a) → s.mulIndicator f a ≤ g a`

`Set.mulIndicator_le`

`∀ {α : Type u_1} {M : Type*} [Monoid M] [PartialOrder M] [CanonicallyOrderedMul M] {s : Set α} {f g : α → M}, (∀ a ∈ s, f a ≤ g a) → s.mulIndicator f ≤ g`

`Set.mulIndicator_le_self`

`∀ {α : Type u_1} {M : Type*} [Monoid M] [PartialOrder M] [CanonicallyOrderedMul M] (s : Set α) (f : α → M), s.mulIndicator f ≤ f`

Algebra · Order · Monoid · Lex

`MonoidHom.fst_mono`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [Preorder α] [Monoid β] [Preorder β], Monotone ⇑(MonoidHom.fst α β)`

`MonoidHom.inl_mono`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [Preorder α] [Monoid β] [Preorder β], Monotone ⇑(MonoidHom.inl α β)`

`MonoidHom.inl_strictMono`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [Preorder α] [Monoid β] [Preorder β], StrictMono ⇑(MonoidHom.inl α β)`

`MonoidHom.inr_mono`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [Preorder α] [Monoid β] [Preorder β], Monotone ⇑(MonoidHom.inr α β)`

`MonoidHom.inr_strictMono`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [Preorder α] [Monoid β] [Preorder β], StrictMono ⇑(MonoidHom.inr α β)`

`MonoidHom.snd_mono`

`∀ {α : Type u_1} {β : Type u_2} [Monoid α] [Preorder α] [Monoid β] [Preorder β], Monotone ⇑(MonoidHom.snd α β)`

LinearOrder → PartialOrder (12, magnitude 2)

Algebra · Order · Archimedean · Basic

`exists_rat_lt` ‡

`∀ {K : Type*} [Field K] [LinearOrder K] [IsStrictOrderedRing K] [Archimedean K] (x : K), ∃ q, ↑q < x`

Algebra · Order · Archimedean · Defs

`exists_pow_lt` ‡

`∀ {R : Type*} [CommGroup R] [LinearOrder R] [IsOrderedMonoid R] [MulArchimedean R] {a : R}, a < 1 → ∀ (b : R), ∃ n, a ^ n < b`

Algebra · Order · Floor · Semiring

`Nat.ceil_mono`

`∀ {R : Type*} [Semiring R] [LinearOrder R] [FloorSemiring R], Monotone Nat.ceil`

`Nat.floor_le`

`∀ {R : Type*} [Semiring R] [LinearOrder R] [FloorSemiring R] {a : R}, 0 ≤ a → ↑⌊a⌋₊ ≤ a`

`Nat.le_ceil`

`∀ {R : Type*} [Semiring R] [LinearOrder R] [FloorSemiring R] (a : R), a ≤ ↑⌈a⌉₊`

Algebra · Order · Module · Defs

`neg_of_smul_neg_left'`

`∀ {α : Type u_1} {β : Type u_2} {a : α} {b : β} [Zero α] [Zero β] [SMulWithZero α β] [LinearOrder α] [LinearOrder β] [SMulPosMono α β], a • b < 0 → 0 ≤ a → b < 0`

`neg_of_smul_neg_right'`

`∀ {α : Type u_1} {β : Type u_2} {a : α} {b : β} [Zero α] [Zero β] [SMulWithZero α β] [LinearOrder α] [LinearOrder β] [PosSMulMono α β], a • b < 0 → 0 ≤ b → a < 0`

Algebra · Order · Monoid · Unbundled · Pow

`pow_le_pow_iff_right'`

```
∀ {M : Type*} [Monoid M] [LinearOrder M] [MulLeftStrictMono M] {a : M} {m n : ℕ}, 1 < a → (a ^ m ≤ a ^ n ↔ m ≤ n)
```

`pow_lt_pow_iff_right'`

```
∀ {M : Type*} [Monoid M] [LinearOrder M] [MulLeftStrictMono M] {a : M} {m n : ℕ}, 1 < a → (a ^ m < a ^ n ↔ m < n)
```

Algebra · Order · Ring · Cast

`Int.cast_one_le_of_pos`

```
∀ {R : Type*} [Ring R] [LinearOrder R] [IsStrictOrderedRing R] {a : ℤ}, 0 < a → 1 ≤ ↑a
```

Algebra · Order · Ring · Unbundled · Basic

`neg_one_lt_zero`

```
∀ {R : Type*} [Ring R] [LinearOrder R] [ZeroLEOneClass R] [NeZero 1] [AddLeftStrictMono R], -1 < 0
```

`sub_one_lt` ‡

```
∀ {R : Type*} [Ring R] [LinearOrder R] [ZeroLEOneClass R] [NeZero 1] [AddLeftStrictMono R] (a : R), a - 1 < a
```

AddCommGroup → AddCommMonoid (10, magnitude 2) †

Algebra · Algebra · Tower

`Algebra.lsmul_injective`

```
∀ (R : Type u_1) (A : Type u_2) (B : Type u_3) (M : Type u_4) [CommSemiring R] [Semiring A] [IsDomain A] [Semiring B] [Algebra R A] [Algebra R B] [AddCommGroup M] [Module R M] [Module A M] [Module B M] [IsScalarTower R A M] [IsScalarTower R B M] [SMulCommClass A B M] [Module.IsTorsionFree A M] {x : A}, x ≠ 0 → Function.Injective ⇑((Algebra.lsmul R B M) x)
```

Algebra · DirectSum · LinearMap

`LinearMap.mapsTo_biSup_of_mapsTo` ‡

```
∀ {R M : Type*} [CommRing R] [AddCommGroup M] [Module R M] {ι : Type u_3} {N : ι → Submodule R M} (s : Set ι) {f : Module.End R M}, (∀ (i : ι), Set.MapsTo ⇑f ↑(N i) ↑(N i)) → Set.MapsTo ⇑f ↑(⨆ i ∈ s, N i) ↑(⨆ i ∈ s, N i)
```

Algebra · Module · Lattice

`Submodule.IsLattice.of_le_of_isLattice_of_fg`

```
∀ {R : Type*} [CommRing R] (A : Type u_2) [CommRing A] [Algebra R A] {V : Type*} [AddCommGroup V] [Module R V] [Module A V] [IsScalarTower R A V] {M N : Submodule R V}, M ≤ N → ∀ [Submodule.IsLattice A M], N.FG → Submodule.IsLattice A N
```

Algebra · Module · LinearMap · End

`Module.End.commute_id_right`

```
∀ {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] (f : Module.End R M), Commute f LinearMap.id
```

Algebra · Order · Module · Defs

`PosSMulMono.toPosSMulStrictMono`

```
∀ {α : Type u_1} {β : Type u_2} [Semiring α] [AddCommGroup β] [Module α β] [IsDomain α] [Module.IsTorsionFree α β] [Preorder α] [PartialOrder β] [PosSMulMono α β], PosSMulStrictMono α β
```

Algebra · Order · WithTop · Untop0

`WithTop.le_of_untop₀_le_untop₀`

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a b : WithTop α}, a ≠ ⊤ → a.untop₀ ≤ b.untop₀ → a ≤ b
```

`WithTop.untop₀_le_untop₀`

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a b : WithTop α}, b ≠ ⊤ → a ≤ b → a.untop₀ ≤ b.untop₀
```

`WithTop.untop₀_le_untop₀_iff`

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a b : WithTop α}, a ≠ ⊤ → b ≠ ⊤ → (a.untop₀ ≤ b.untop₀ ↔ a ≤ b)
```

`WithTop.untop₀_nonneg`

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a : WithTop α}, 0 ≤ a.untop₀ ↔ 0 ≤ a
```

Algebra · Polynomial · Module · FiniteDimensional

`Module.AEval.isTorsion_of_finiteDimensional` ‡

```
∀ (K : Type u_1) (M : Type u_2) {A : Type*} (a : A) [Field K] [Ring A] [Algebra K A] [AddCommGroup M] [Module A M] [Module K M] [IsScalarTower K A M] [FiniteDimensional K A], Module.IsTorsion (Polynomial K) (Module.AEval K M a)
```

CommSemigroup → CommMagma (6, magnitude 2)

Algebra · Group · Pointwise · Finset · Basic

`Finset.inter_mul_union_subset`

```
∀ {α : Type u_1} [DecidableEq α] [CommSemigroup α] {s t : Finset α}, s ∩ t * (s ∪ t) ⊆ s * t
```

`Finset.union_mul_inter_subset`

```
∀ {α : Type u_1} [DecidableEq α] [CommSemigroup α] {s t : Finset α}, (s ∪ t) * (s ∩ t) ⊆ s * t
```

Algebra · Group · Pointwise · Set · Basic

`Set.inter_mul_union_subset`

```
∀ {α : Type u_1} [CommSemigroup α] {s t : Set α}, s ∩ t * (s ∪ t) ⊆ s * t
```

`Set.union_mul_inter_subset`

```
∀ {α : Type u_1} [CommSemigroup α] {s t : Set α}, (s ∪ t) * (s ∩ t) ⊆ s * t
```

Algebra · Regular · SMul

`IsSMulRegular.mul_iff`

```
∀ {R M : Type*} {a b : R} [CommSemigroup R] [SMul R M] [IsScalarTower R R M], IsSMulRegular M (a * b) ↔ IsSMulRegular M a ∧ IsSMulRegular M b
```

Algebra · Star · SelfAdjoint

`IsSelfAdjoint.mul`

```
∀ {R : Type*} [CommSemigroup R] [StarMul R] {x y : R}, IsSelfAdjoint x → IsSelfAdjoint y → IsSelfAdjoint (x * y)
```

Ring → Semiring (6, magnitude 2) †

Algebra · Algebra · StrictPositivity

`IsStrictlyPositive.smul`

```
∀ {A : Type*} {□ : Type u_2} [Ring A] [PartialOrder A] [Semifield □] [PartialOrder □] [Algebra □ A] [PosSMulMono □ A] {c : □}, 0 < c → ∀ {a : A}, IsStrictlyPositive a → IsStrictlyPositive (c • a)
```

Algebra · CharP · Invertible

`not_ringChar_dvd_of_invertible`

```
∀ {R : Type*} [Ring R] {t : ℕ} [Invertible ↑t] [Nontrivial R], ¬ringChar R □ t
```

Algebra · Module · LinearMap · End

`Module.End.commute_id_right`

```
∀ {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] (f : Module.End R M), Commute f LinearMap.id
```

Algebra · Module · Submodule · Ker

`LinearMap.injective_codRestrict_iff`

```
∀ {R R₂ M M₂ : Type*} [Ring R] [Ring R₂] [AddCommGroup M] [AddCommGroup M₂] [Module R M] [Module R₂ M₂] {τ₁₂ : R →+* R₂} {q : Submodule R₂ M₂} {f : M →□ □ [τ₁₂] M₂} (hf : ∀ (x : M), f x ∈ q), Function.Injective ⇑(LinearMap.codRestrict q f hf) ↔ Function.Injective ⇑f
```

Algebra · Order · AbsoluteValue · Basic

`IsAbsoluteValue.abv_sub_le`

```
∀ {S : Type*} [Ring S] [PartialOrder S] {R : Type*} [Ring R] (abv : R → S) [IsAbsoluteValue abv] (a b c : R), abv (a - c) ≤ abv (a - b) + abv (b - c)
```

Algebra · Order · Module · Defs

`smul_nonneg_of_nonpos_of_nonpos`

```
∀ {α : Type u_1} {β : Type u_2} {a : α} {b : β} [Ring α] [PartialOrder α] [AddCommGroup β] [PartialOrder β] [IsOrderedAddMonoid β] [Module α β] [SMulPosMono α β], a ≤ 0 → b ≤ 0 → 0 ≤ a • b
```

AddCommSemigroup → AddCommMagma (2, magnitude 2)

Algebra · Order · Sub · Defs

`lt_tsub_iff_right`

```
∀ {α : Type u_1} {a b c : α} [LinearOrder α] [AddCommSemigroup α] [Sub α] [OrderedSub α], a < b - c ↔ a + c < b
```

`tsub_le_iff_left`

```
∀ {α : Type u_1} [Preorder α] [AddCommSemigroup α] [Sub α] [OrderedSub α] {a b c : α}, a - b ≤ c ↔ a ≤ b + c
```

AddGroup → AddMonoid (2, magnitude 2) †

Algebra · Order · GroupWithZero · Canonical

`WithZero.le_log_iff_exp_le` ‡

```
∀ {G : Type*} [Preorder G] {a : G} [AddGroup G] {x : WithZero (Multiplicative G)}, x ≠ 0 → (a ≤ x.log ↔ WithZero.exp a ≤ x)
```

`WithZero.lt_log_iff_exp_lt` ‡

```
∀ {G : Type*} [Preorder G] {a : G} [AddGroup G] {x : WithZero (Multiplicative G)}, x ≠ 0 → (a < x.log ↔ WithZero.exp a < x)
```

AddGroup → SubNegMonoid (2, magnitude 2)

Algebra · Order · GroupWithZero · Canonical

`WithZero.log_le_iff_le_exp`

```
∀ {G : Type*} [Preorder G] {a : G} [AddGroup G] {x : WithZero (Multiplicative G)}, x ≠ 0 → (x.log ≤ a ↔ x ≤ WithZero.exp a)
```

`WithZero.log_lt_iff_lt_exp`

```
∀ {G : Type*} [Preorder G] {a : G} [AddGroup G] {x : WithZero (Multiplicative G)}, x ≠ 0 → (x.log < a ↔ x < WithZero.exp a)
```

Group → DivInvMonoid (2, magnitude 2)

Algebra · Order · Group · PosPart

`oneLePart_lt`

```
∀ {α : Type u_1} [LinearOrder α] [Group α] {a b : α}, a⁺ᵐ < b ↔ a < b ∧ 1 < b
```

`one_lt_oneLePart_iff`

```
∀ {α : Type u_1} [LinearOrder α] [Group α] {a : α}, 1 < a⁺ᵐ ↔ 1 < a
```

CommMonoid → CommSemigroup (1, magnitude 2)

Algebra · Group · Action · Defs

`IsScalarTower.of_commMonoid`

```
∀ (R₁ : Type u_1) (R : Type u_2) [Monoid R₁] [CommMonoid R] [MulAction R₁ R] [SMulCommClass R₁ R R], IsScalarTower R₁ R R
```

PartialOrder → Preorder (57, magnitude 1)

Algebra · AddConstMap · Basic

`AddConstMapClass.monotone_iff_Icc`

```
∀ {F G H : Type*} [FunLike F G H] {a : G} {b : H} [AddCommGroup G] [LinearOrder G] [IsOrderedAddMonoid G] [Archimedean G] [AddCommGroup H] [PartialOrder H] [IsOrderedAddMonoid H] [AddConstMapClass F G H a b] {f : F}, 0 < a → ∀ (l : G), Monotone ⇑f ↔ MonotoneOn (⇑f) (Set.Icc l (l + a))
```

Algebra · Algebra · StrictPositivity

`IsStrictlyPositive.of_subsingleton`

```
∀ {A : Type*} [PartialOrder A] [Monoid A] [Zero A] [Subsingleton A] {a : A}, IsStrictlyPositive a
```

`IsStrictlyPositive.smul`

```
∀ {A : Type*} {𝕜 : Type u_2} [Ring A] [PartialOrder A] [Semifield 𝕜] [PartialOrder 𝕜] [Algebra 𝕜 A] [PosSMulMono 𝕜 A] {c : 𝕜}, 0 < c → ∀ {a : A}, IsStrictlyPositive a → IsStrictlyPositive (c • a)
```

`IsStrictlyPositive.smul`

```
∀ {A : Type*} {𝕜 : Type u_2} [Ring A] [PartialOrder A] [Semifield 𝕜] [PartialOrder 𝕜] [Algebra 𝕜 A] [PosSMulMono 𝕜 A] {c : 𝕜}, 0 < c → ∀ {a : A}, IsStrictlyPositive a → IsStrictlyPositive (c • a)
```

Algebra · BigOperators · Finprod

`finprod_le_finprod`

```
∀ {α : Type u_1} {M : Type*} [CommMonoidWithZero M] [PartialOrder M] [ZeroLEOneClass M] [PosMulMono M] {f g : α → M}, Function.HasFiniteMulSupport f → (∀ (a : α), 0 ≤ f a) → Function.HasFiniteMulSupport g → f ≤ g → ∏ᶠ (a : α), f a ≤ ∏ᶠ (a : α), g a
```

`finprod_le_finprod'`

```
∀ {α : Type u_1} {M : Type*} [CommMonoid M] {f g : α → M} [PartialOrder M] [MulLeftMono M], Function.HasFiniteMulSupport f → Function.HasFiniteMulSupport g → f ≤ g → ∏ᶠ (a : α), f a ≤ ∏ᶠ (a : α), g a
```

Algebra · Group · Submonoid · Pointwise

`Set.IsPWO.submonoid_closure`

```
∀ {α : Type u_1} [CommMonoid α] [PartialOrder α] [IsOrderedCancelMonoid α] {s : Set α}, (∀ x ∈ s, 1 ≤ x) → s.IsPWO → (↑(Submonoid.closure s)).IsPWO
```

Algebra · Order · Algebra

`IsOrderedModule.of_algebraMap_mono`

```
∀ {α : Type u_1} {β : Type u_2} [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [PosMulMono β] [MulPosMono β], Monotone ⇑(algebraMap α β) → IsOrderedModule α β
```

`IsOrderedModule.of_algebraMap_mono`

```
∀ {α : Type u_1} {β : Type u_2} [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [PosMulMono β] [MulPosMono β], Monotone ⇑(algebraMap α β) → IsOrderedModule α β
```

`algebraMap_le_algebraMap`

```
∀ {α : Type u_1} {β : Type u_2} [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [ZeroLEOneClass β] [Nontrivial β] [SMulPosMono α β] [SMulPosReflectLE α β] {a₁ a₂ : α}, (algebraMap α β) a₁ ≤ (algebraMap α β) a₂ ↔ a₁ ≤ a₂
```

`algebraMap_lt_algebraMap`

```
∀ {α : Type u_1} {β : Type u_2} [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [ZeroLEOneClass β] [Nontrivial β] [SMulPosStrictMono α β] [SMulPosReflectLT α β] {a₁ a₂ : α}, (algebraMap α β) a₁ < (algebraMap α β) a₂ ↔ a₁ < a₂
```

`algebraMap_mono`

```
∀ {α : Type u_1} (β : Type u_2) [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [ZeroLEOneClass β] [SMulPosMono α β], Monotone ⇑(algebraMap α β)
```

`algebraMap_mono`

```
∀ {α : Type u_1} (β : Type u_2) [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [ZeroLEOneClass β] [SMulPosMono α β], Monotone ⇑(algebraMap α β)
```

`algebraMap_strictMono`

```
∀ {α : Type u_1} (β : Type u_2) [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [ZeroLEOneClass β]
```

[Nontrivial β] [SMulPosStrictMono α β], StrictMono ⇑(algebraMap α β)

isOrderedModule_iff_algebraMap_mono

∀ {α : Type u_1} {β : Type u_2} [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [ZeroLEOneClass β] [PosMulMono β] [MulPosMono β], IsOrderedModule α β ↔ Monotone ⇑(algebraMap α β)

isOrderedModule_iff_algebraMap_mono

∀ {α : Type u_1} {β : Type u_2} [CommSemiring α] [PartialOrder α] [Semiring β] [PartialOrder β] [Algebra α β] [ZeroLEOneClass β] [PosMulMono β] [MulPosMono β], IsOrderedModule α β ↔ Monotone ⇑(algebraMap α β)

Algebra · Order · BigOperators · Group · Finset

Finset.prod_lt_prod_of_subset_erase_union_singleton ‡

∀ {ι : Type u_1} {M : Type*} [DecidableEq ι] [CommMonoid M] [PartialOrder M] [CanonicallyOrderedMul M] [MulLeftStrictMono M] {S S' : Finset ι} {f : ι → M} {d d' : ι}, d ∈ S → S' ≤ S.erase d ∪ {d'} → f d' < f d → ∏ x ∈ S', f x < ∏ x ∈ S, f x

Algebra · Order · BigOperators · GroupWithZero · List

List.one_le_prod

∀ {R : Type*} [CommMonoidWithZero R] [PartialOrder R] [ZeroLEOneClass R] [PosMulMono R] {s : List R}, (∀ a ∈ s, 1 ≤ a) → 1 ≤ s.prod

List.prod_nonneg

∀ {R : Type*} [CommMonoidWithZero R] [PartialOrder R] [ZeroLEOneClass R] [PosMulMono R] {s : List R}, (∀ a ∈ s, 0 ≤ a) → 0 ≤ s.prod

Algebra · Order · BigOperators · Ring · Multiset

Multiset.le_prod_of_submultiplicative_on_pred_of_nonneg

∀ {α : Type u_1} {β : Type u_2} [CommMonoid α] [CommMonoidWithZero β] [PartialOrder β] [PosMulMono β] (f : α → β) (p : α → Prop), (∀ (a : α), 0 ≤ f a) → f 1 ≤ 1 → (∀ (a b : α), p a → p b → f (a * b) ≤ f a * f b) → (∀ (a b : α), p a → p b → p (a * b)) → ∀ (s : Multiset α), (∀ a ∈ s, p a) → f s.prod ≤ (Multiset.map f s).prod

Algebra · Order · Field · Basic

Mathlib.Meta.Positivity.div_ne_zero_of_ne_zero_of_pos

∀ {α : Type u_1} [GroupWithZero α] [PartialOrder α] {a b : α}, a ≠ 0 → 0 < b → a / b ≠ 0

Mathlib.Meta.Positivity.div_ne_zero_of_pos_of_ne_zero

∀ {α : Type u_1} [GroupWithZero α] [PartialOrder α] {a b : α}, 0 < a → b ≠ 0 → a / b ≠ 0

Algebra · Order · Group · Basic

zpow_left_mono

∀ (α : Type u_1) [CommGroup α] [PartialOrder α] [IsOrderedMonoid α] {n : ℤ}, 0 ≤ n → Monotone fun x => x ^ n

zpow_right_mono

∀ {α : Type u_1} [CommGroup α] [PartialOrder α] [IsOrderedMonoid α] {a : α}, 1 ≤ a → Monotone fun n => a ^ n

Algebra · Order · Group · Defs

inv_le_inv'

∀ {α : Type u_1} [CommGroup α] [PartialOrder α] [IsOrderedMonoid α] {a b : α}, a ≤ b → b^{-1} ≤ a^{-1}

inv_le_one_of_one_le

∀ {α : Type u_1} [CommGroup α] [PartialOrder α] [IsOrderedMonoid α] {a : α}, 1 ≤ a → a^{-1} ≤ 1

one_le_inv_of_le_one

∀ {α : Type u_1} [CommGroup α] [PartialOrder α] [IsOrderedMonoid α] {a : α}, a ≤ 1 → 1 ≤ a^{-1}

Algebra · Order · Group · Indicator

Set.mulIndicator_apply_le

∀ {α : Type u_1} {M : Type*} [Monoid M] [PartialOrder M] [CanonicallyOrderedMul M] {a : α} {s : Set α} {f g : α → M}, (a ∈ s → f a ≤ g a) → s.mulIndicator f a ≤ g a

Set.mulIndicator_le

∀ {α : Type u_1} {M : Type*} [Monoid M] [PartialOrder M] [CanonicallyOrderedMul M] {s : Set α} {f g : α → M}, (∀ a ∈ s, f a ≤ g a) → s.mulIndicator f ≤ g

Set.mulIndicator_le_self

∀ {α : Type u_1} {M : Type*} [Monoid M] [PartialOrder M] [CanonicallyOrderedMul M] (s : Set α) (f : α → M), s.mulIndicator f ≤ f

Algebra · Order · GroupWithZero · Basic

Monotone.mul_strictMono

∀ {α : Type u_1} {M_0 : Type*} [MonoidWithZero M_0] [PartialOrder M_0] [Preorder α] {f g : α → M_0} [PosMulStrictMono M_0] [MulPosMono M_0], Monotone f → StrictMono g → (∀ (x : α), 0 < f x) → (∀ (x : α), 0 ≤ g x) → StrictMono (f * g)

StrictMono.mul_monotone

∀ {α : Type u_1} {M_0 : Type*} [MonoidWithZero M_0] [PartialOrder M_0] [Preorder α] {f g : α → M_0} [PosMulMono M_0] [MulPosStrictMono M_0], StrictMono f → Monotone g → (∀ (x : α), 0 ≤ f x) → (∀ (x : α), 0 < g x) → StrictMono (f * g)

sq_pos_of_pos

∀ {M_0 : Type*} [MonoidWithZero M_0] [PartialOrder M_0] {a : M_0} [PosMulStrictMono M_0], 0 < a → 0 < a ^ 2

strictMono_mul_left_of_pos

```
∀ {M₀ : Type*} [MonoidWithZero M₀] [PartialOrder M₀] {a : M₀} [PosMulStrictMono M₀], 0 < a → StrictMono fun x => a * x
```

Algebra · Order · GroupWithZero · Canonical

WithZero.isOrderedAddMonoid

```
∀ {α : Type u_1} [AddCommMonoid α] [PartialOrder α] [IsOrderedAddMonoid α], (∀ (a : α), 0 ≤ a) → IsOrderedAddMonoid (WithZero α)
```

Algebra · Order · Interval · Basic

NonemptyInterval.one_mem_one

```
∀ {α : Type u_1} [PartialOrder α] [One α], 1 ∈ 1
```

Algebra · Order · Interval · Set · Group

Set.add_mem_Icc_iff_left

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] [IsOrderedAddMonoid α] {a b c d : α}, a + b ∈ Set.Icc c d ↔ a ∈ Set.Icc (c - b) (d - b)
```

Set.add_mem_Icc_iff_right

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] [IsOrderedAddMonoid α] {a b c d : α}, a + b ∈ Set.Icc c d ↔ b ∈ Set.Icc (c - a) (d - a)
```

Set.inv_mem_Icc_iff

```
∀ {α : Type u_1} [CommGroup α] [PartialOrder α] [IsOrderedMonoid α] {a c d : α}, a⁻¹ ∈ Set.Icc c d ↔ a ∈ Set.Icc d⁻¹ c⁻¹
```

Set.sub_mem_Icc_iff_left

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] [IsOrderedAddMonoid α] {a b c d : α}, a - b ∈ Set.Icc c d ↔ a ∈ Set.Icc (c + b) (d + b)
```

Set.sub_mem_Icc_iff_right

```
∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] [IsOrderedAddMonoid α] {a b c d : α}, a - b ∈ Set.Icc c d ↔ b ∈ Set.Icc (a - d) (a - c)
```

Algebra · Order · Module · Defs

PosSMulMono.of_smul_nonneg

```
∀ {α : Type u_1} {β : Type u_2} [Semiring α] [AddCommGroup β] [Module α β] [PartialOrder α] [PartialOrder β] [IsOrderedAddMonoid β], (∀ (a : α), 0 ≤ a → ∀ (b : β), 0 ≤ b → 0 ≤ a • b) → PosSMulMono α β
```

PosSMulMono.of_smul_nonneg

```
∀ {α : Type u_1} {β : Type u_2} [Semiring α] [AddCommGroup β] [Module α β] [PartialOrder α] [PartialOrder β] [IsOrderedAddMonoid β], (∀ (a : α), 0 ≤ a → ∀ (b : β), 0 ≤ b → 0 ≤ a • b) → PosSMulMono α β
```

SMulPosMono.toSMulPosStrictMono

```
∀ {α : Type u_1} {β : Type u_2} [Ring α] [AddCommGroup β] [Module α β] [PartialOrder α] [PartialOrder β] [IsDomain α] [Module.IsTorsionFree α β] [SMulPosMono α β], SMulPosStrictMono α β
```

SMulPosReflectLT.toSMulPosReflectLE

```
∀ {α : Type u_1} {β : Type u_2} [Ring α] [AddCommGroup β] [Module α β] [PartialOrder α] [PartialOrder β] [IsDomain α] [Module.IsTorsionFree α β] [SMulPosReflectLT α β], SMulPosReflectLE α β
```

le_of_smul_le_smul_of_neg

```
∀ {α : Type u_1} {β : Type u_2} {a : α} {b₁ b₂ : β} [Ring α] [PartialOrder α] [IsOrderedRing α] [AddCommGroup β] [PartialOrder β] [IsOrderedAddMonoid β] [Module α β] [PosSMulReflectLE α β], a • b₁ ≤ a • b₂ → a < 0 → b₂ ≤ b₁
```

smul_le_smul_iff_of_neg_left

```
∀ {α : Type u_1} {β : Type u_2} {a : α} {b₁ b₂ : β} [Ring α] [PartialOrder α] [IsOrderedRing α] [AddCommGroup β] [PartialOrder β] [IsOrderedAddMonoid β] [Module α β] [PosSMulMono α β] [PosSMulReflectLE α β], a < 0 → (a • b₁ ≤ a • b₂ ↔ b₂ ≤ b₁)
```

smul_le_smul_of_nonpos_left

```
∀ {α : Type u_1} {β : Type u_2} {a : α} {b₁ b₂ : β} [Ring α] [PartialOrder α] [IsOrderedRing α] [AddCommGroup β] [PartialOrder β] [IsOrderedAddMonoid β] [Module α β] [PosSMulMono α β], b₁ ≤ b₂ → a ≤ 0 → a • b₂ ≤ a • b₁
```

Algebra · Order · Ring · Canonical

CanonicallyOrderedAdd.toIsOrderedMonoid

```
∀ {R : Type*} [CommSemiring R] [PartialOrder R] [CanonicallyOrderedAdd R], IsOrderedMonoid R
```

Algebra · Order · Ring · Unbundled · Basic

mul_lt_mul_of_neg_left

```
∀ {R : Type*} [Semiring R] [PartialOrder R] {a b c : R} [ExistsAddOfLE R] [PosMulStrictMono R] [AddRightStrictMono R] [AddRightReflectLT R], b < a → c < 0 → c * a < c * b
```

mul_lt_mul_of_neg_right

```
∀ {R : Type*} [Semiring R] [PartialOrder R] {a b c : R} [ExistsAddOfLE R] [MulPosStrictMono R] [AddRightStrictMono R] [AddRightReflectLT R], b < a → c < 0 → a * c < b * c
```

Algebra · Order · SuccPred

Order.not_isSuccLimit_add_one

```
∀ {α : Type u_1} [PartialOrder α] (a : α) [Add α] [One α] [SuccAddOrder α] [NoMaxOrder α], ¬Order.IsSuccLimit (a + 1)
```

Order.not_isSuccPrelimit_add_one

`∀ {α : Type u_1} [PartialOrder α] (a : α) [Add α] [One α] [SuccAddOrder α] [NoMaxOrder α], ¬Order.IsSuccPrelimit (a + 1)`

Algebra · Order · WithTop · Untop0

`WithTop.le_of_untop₀_le_untop₀`

`∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a b : WithTop α}, a ≠ □ → a.untop₀ ≤ b.untop₀ → a ≤ b`

`WithTop.untop₀_le_untop₀`

`∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a b : WithTop α}, b ≠ □ → a ≤ b → a.untop₀ ≤ b.untop₀`

`WithTop.untop₀_le_untop₀_iff`

`∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a b : WithTop α}, a ≠ □ → b ≠ □ → (a.untop₀ ≤ b.untop₀ ↔ a ≤ b)`

`WithTop.untop₀_nonneg`

`∀ {α : Type u_1} [AddCommGroup α] [PartialOrder α] {a : WithTop α}, 0 ≤ a.untop₀ ↔ 0 ≤ a`

Semiring → NonUnitalSemiring (13, magnitude 1)

Algebra · Order · Ring · Unbundled · Basic

`mul_le_mul_of_nonpos_left`

`∀ {R : Type*} [Semiring R] [Preorder R] {a b c : R} [ExistsAddOfLE R] [PosMulMono R] [AddRightMono R] [AddRightReflectLE R], b ≤ a → c ≤ 0 → c * a ≤ c * b`

`mul_le_mul_of_nonpos_right`

`∀ {R : Type*} [Semiring R] [Preorder R] {a b c : R} [ExistsAddOfLE R] [MulPosMono R] [AddRightMono R] [AddRightReflectLE R], b ≤ a → c ≤ 0 → a * c ≤ b * c`

`mul_lt_mul_of_neg_left`

`∀ {R : Type*} [Semiring R] [PartialOrder R] {a b c : R} [ExistsAddOfLE R] [PosMulStrictMono R] [AddRightStrictMono R] [AddRightReflectLT R], b < a → c < 0 → c * a < c * b`

`mul_lt_mul_of_neg_right`

`∀ {R : Type*} [Semiring R] [PartialOrder R] {a b c : R} [ExistsAddOfLE R] [MulPosStrictMono R] [AddRightStrictMono R] [AddRightReflectLT R], b < a → c < 0 → a * c < b * c`

`mul_nonneg_iff_of_pos_left`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {b c : R} [PosMulStrictMono R], 0 < c → (0 ≤ c * b ↔ 0 ≤ b)`

`mul_nonneg_iff_of_pos_right`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {b c : R} [MulPosStrictMono R], 0 < c → (0 ≤ b * c ↔ 0 ≤ b)`

`nonneg_and_nonneg_or_nonpos_and_nonpos_of_mul_nonneg`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {a b : R} [MulPosStrictMono R] [PosMulStrictMono R], 0 ≤ a * b → 0 ≤ a ∧ 0 ≤ b ∨ a ≤ 0 ∧ b ≤ 0`

`nonneg_of_mul_nonneg_left`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {a b : R} [MulPosStrictMono R], 0 ≤ a * b → 0 < b → 0 ≤ a`

`nonneg_of_mul_nonneg_right`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {a b : R} [PosMulStrictMono R], 0 ≤ a * b → 0 < a → 0 ≤ b`

`nonpos_of_mul_nonneg_left`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {a b : R} [PosMulStrictMono R], 0 ≤ a * b → b < 0 → a ≤ 0`

`nonpos_of_mul_nonneg_right`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {a b : R} [MulPosStrictMono R], 0 ≤ a * b → a < 0 → b ≤ 0`

`nonpos_of_mul_nonpos_left`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {a b : R} [PosMulStrictMono R], a * b ≤ 0 → 0 < b → a ≤ 0`

`nonpos_of_mul_nonpos_right`

`∀ {R : Type*} [Semiring R] [LinearOrder R] {a b : R} [PosMulStrictMono R], a * b ≤ 0 → 0 < a → b ≤ 0`

Field → DivisionRing (12, magnitude 1)

Algebra · Order · Archimedean · Basic

`archimedean_iff_nat_lt`

`∀ {K : Type*} [Field K] [LinearOrder K] [IsStrictOrderedRing K], Archimedean K ↔ ∀ (x : K), ∃ n, x < ↑n`

`exists_rat_lt` ‡

`∀ {K : Type*} [Field K] [LinearOrder K] [IsStrictOrderedRing K] [Archimedean K] (x : K), ∃ q, ↑q < x`

Algebra · Order · Field · Basic

`div_neg_iff`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {a b : α}, a / b < 0 ↔ 0 < a ∧ b < 0 ∨ a < 0 ∧ 0 < b`

`div_nonneg_iff`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {a b : α}, 0 ≤ a / b ↔ 0 ≤ a ∧ 0 ≤ b ∨ a ≤ 0 ∧ b ≤ 0` `div_nonpos_iff`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {a b : α}, a / b ≤ 0 ↔ 0 ≤ a ∧ b ≤ 0 ∨ a ≤ 0 ∧ 0 ≤ b`

`div_pos_iff`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {a b : α}, 0 < a / b ↔ 0 < a ∧ 0 < b ∨ a < 0 ∧ b < 0`

Algebra · Order · Field · Power

`Even.zpow_nonneg`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {n : ℤ}, Even n → ∀ (a : α), 0 ≤ a ^ n`

`Even.zpow_pos_iff`

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {a : α} {n : ℤ}, Even n → n ≠ 0 → (0 < a ^ n ↔ a ≠ 0)`

`Nat.cast_le_pow_sub_div_sub` ‡

`∀ {α : Type u_1} [Field α] [LinearOrder α] [IsStrictOrderedRing α] {a : α}, 1 < a → ∀ (n : ℕ), ↑n ≤ (a ^ n - 1) / (a - 1)`

Algebra · Order · Floor · Ring

`Int.sub_floor_div_mul_lt` ‡

`∀ {k : Type*} [Field k] [LinearOrder k] [IsOrderedRing k] [FloorRing k] {b : k} (a : k), 0 < b → a - ↑⌊a / b⌋ * b < b`

`Int.sub_floor_div_mul_nonneg` ‡

`∀ {k : Type*} [Field k] [LinearOrder k] [IsOrderedRing k] [FloorRing k] {b : k} (a : k), 0 < b → 0 ≤ a - ↑⌊a / b⌋ * b`

Algebra · Polynomial · FieldDivision

`Polynomial.not_irreducible_C`

`∀ {R : Type*} [Field R] (x : R), ¬Irreducible (Polynomial.C x)`

CommSemiring → Semiring (9, magnitude 1)

Algebra · Algebra · Bilinear

`LinearMap.commute_mulLeft_right`

`∀ {R A : Type*} [CommSemiring R] [NonUnitalSemiring A] [Module R A] [SMulCommClass R A A] [IsScalarTower R A A] (a b : A), Commute (LinearMap.mulLeft R a) (LinearMap.mulRight R b)`

Algebra · Algebra · NonUnitalSubalgebra

`NonUnitalSubalgebra._root_.Set.smul_mem_center`

`∀ {R A : Type*} [CommSemiring R] [NonUnitalNonAssocSemiring A] [Module R A] [IsScalarTower R A A] [SMulCommClass R A A] (r : R) {a : A}, a ∈ Set.center A → r • a ∈ Set.center A`

Algebra · CharP · Algebra

`Algebra.charP_iff`

`∀ (K : Type u_1) (L : Type u_2) [Field K] [CommSemiring L] [Nontrivial L] [Algebra K L] (p : ℕ), CharP K p ↔ CharP L p`

Algebra · Module · Torsion · Basic

`Submodule._root_.Module.isTorsionBySet_annihilator_top`

`∀ (R : Type u_1) (M : Type u_2) [CommSemiring R] [AddCommMonoid M] [Module R M], Module.IsTorsionBySet R M ↑⊤ .annihilator`

Algebra · Order · Ring · Canonical

`CanonicallyOrderedAdd.mul_pos`

`∀ {R : Type*} [CommSemiring R] [PartialOrder R] [CanonicallyOrderedAdd R] [NoZeroDivisors R] {a b : R}, 0 < a * b ↔ 0 < a ∧ 0 < b`

`CanonicallyOrderedAdd.pow_pos` ‡

`∀ {R : Type*} [CommSemiring R] [PartialOrder R] [CanonicallyOrderedAdd R] [IsReduced R] {a : R}, 0 < a → ∀ (n : ℕ), 0 < a ^ n`

`CanonicallyOrderedAdd.toIsOrderedRing` ‡

`∀ {R : Type*} [CommSemiring R] [PartialOrder R] [CanonicallyOrderedAdd R], IsOrderedRing R`

Algebra · Polynomial · Eval · Coeff

`Polynomial.IsRoot.map`

`∀ {R S : Type*} [CommSemiring R] [CommSemiring S] {f : R →+* S} {x : R} {p : Polynomial R}, p.IsRoot x → (Polynomial.map f p).IsRoot (f x)`

Algebra · Ring · SumsOfSquares

`IsSumSq.sum_sq`

`∀ {R : Type*} [CommSemiring R] {ι : Type u_2} (I : Finset ι) (a : ι → R), IsSumSq (∑ i ∈ I, a i ^ 2)`

CommMonoid → Monoid (6, magnitude 1)

Algebra · BigOperators · Finprod

`Multiset.mulSupport_fun_pow_count_subset`

`∀ {α : Type u_1} {M : Type*} [DecidableEq α] [CommMonoid M] (s : Multiset α) (f : α → M), (Function.mulSupport fun a => f a ^ Multiset.count a s) ⊆ ↑s.toFinset`

Algebra · Divisibility · Units

`IsRelPrime.isUnit_of_dvd`

`∀ {α : Type u_1} [CommMonoid α] {x y : α}, IsRelPrime x y → x □ y → IsUnit x`

`IsRelPrime.of_mul_left_left`

`∀ {α : Type u_1} [CommMonoid α] {x y z : α}, IsRelPrime (x * y) z → IsRelPrime x z`

`IsRelPrime.symm`

`∀ {α : Type u_1} [CommMonoid α] {x y : α}, IsRelPrime x y → IsRelPrime y x`

Algebra · Group · Pointwise · Set · BigOperators

`Set.list_prod_mem_list_prod` ‡

`∀ {ι : Type u_1} {α : Type u_2} [CommMonoid α] (t : List ι) (f : ι → Set α) (g : ι → α), (∀ i ∈ t, g i ∈ f i) → (List.map g t).prod ∈ (List.map f t).prod`

Algebra · Squarefree · Basic

`squarefree_iff_emultiplicity_le_one`

`∀ {R : Type*} [CommMonoid R] (r : R), Squarefree r ↔ ∀ (x : R), emultiplicity x r ≤ 1 ∨ IsUnit x`

AddCommMonoid → AddMonoid (5, magnitude 1)

Algebra · BigOperators · Finsupp · Basic

`Finsupp.comp_liftAddHom`

`∀ {α : Type u_1} {M N P : Type*} [AddZeroClass M] [AddCommMonoid N] [AddCommMonoid P] (g : N →+ P) (f : α → M →+ N), g.comp (Finsupp.liftAddHom f) = Finsupp.liftAddHom fun a => g.comp (f a)`

`Finsupp.liftAddHom_apply`

`∀ {α : Type u_1} {M N : Type*} [AddZeroClass M] [AddCommMonoid N] (F : α → M →+ N) (f : α →₀ M), (Finsupp.liftAddHom F) f = f.sum fun x => ⇑(F x)`

`Finsupp.liftAddHom_apply_single`

`∀ {α : Type u_1} {M N : Type*} [AddZeroClass M] [AddCommMonoid N] (f : α → M →+ N) (a : α) (b : M), ((Finsupp.liftAddHom f) fun₀ | a => b) = (f a) b`

`Finsupp.liftAddHom_comp_single`

`∀ {α : Type u_1} {M N : Type*} [AddZeroClass M] [AddCommMonoid N] (f : α → M →+ N) (a : α), (Finsupp.liftAddHom f).comp (Finsupp.singleAddHom a) = f a`

Algebra · Order · Sub · Unbundled · Hom

`AddMonoidHom.le_map_tsub`

`∀ {α : Type u_1} {β : Type u_2} [Preorder α] [AddCommMonoid α] [Sub α] [OrderedSub α] [Preorder β] [AddZeroClass β] [Sub β] [OrderedSub β] (f : α →+ β), Monotone ⇑f → ∀ (a b : α), f a - f b ≤ f (a - b)`

AddZeroClass → AddZero (5, magnitude 1)

Algebra · MonoidAlgebra · Degree

`AddMonoidAlgebra.le_infDegree_mul`

`∀ {R A T : Type*} [Semiring R] [SemilatticeInf T] [OrderTop T] [AddZeroClass A] [Add T] [AddLeftMono T] [AddRightMono T] (D : A →□ + T) (f g : AddMonoidAlgebra R A), AddMonoidAlgebra.infDegree (⇑D) f + AddMonoidAlgebra.infDegree (⇑D) g ≤ AddMonoidAlgebra.infDegree (⇑D) (f * g)`

`AddMonoidAlgebra.monic_one`

`∀ {R A B : Type*} [Semiring R] [LinearOrder B] [OrderBot B] {D : A → B} [AddZeroClass A], Function.Injective D → AddMonoidAlgebra.Monic D 1`

`AddMonoidAlgebra.supDegree_mem_support`

`∀ {R A B : Type*} [Semiring R] [LinearOrder B] [OrderBot B] {p : AddMonoidAlgebra R A} {D : A → B} [AddZeroClass A], Function.Injective D → p ≠ 0 → Function.invFun D (AddMonoidAlgebra.supDegree D p) ∈ p.coeff.support`

Algebra · Order · BigOperators · Group · List

`List.sum_le_foldr_max`

`∀ {M N : Type*} [AddZeroClass M] [Zero N] [LinearOrder N] (f : M → N), f 0 ≤ 0 → (∀ (x y : M), f (x + y) ≤ max (f x) (f y)) → ∀ (l : List M), f l.sum ≤ List.foldr max 0 (List.map f l)`

Algebra · Order · Sub · Unbundled · Hom

`AddMonoidHom.le_map_tsub`

`∀ {α : Type u_1} {β : Type u_2} [Preorder α] [AddCommMonoid α] [Sub α] [OrderedSub α] [Preorder β] [AddZeroClass β] [Sub β] [OrderedSub β] (f : α →+ β), Monotone ⇑f → ∀ (a b : α), f a - f b ≤ f (a - b)`

CommRing → Ring (5, magnitude 1)

Algebra · Category · ModuleCat · ChangeOfRingsExact

`ModuleCat.restrictScalars_map_exact`

`∀ {R : Type*} [CommRing R] {R' : Type*} [CommRing R'] (f : R →+* R') (S : CategoryTheory.ShortComplex (ModuleCat R')), S.Exact → (S.map (ModuleCat.restrictScalars f)).Exact`

Algebra · DirectSum · LinearMap

`LinearMap.mapsTo_biSup_of_mapsTo` ‡

```
∀ {R M : Type*} [CommRing R] [AddCommGroup M] [Module R M] {ι : Type u_3} {N : ι → Submodule R M} (s : Set ι) {f : Module.End R M}, (∀ (i : ι), Set.MapsTo ⇑f ↑(N i) ↑(N i)) → Set.MapsTo ⇑f ↑(⨆ i ∈ s, N i) ↑(⨆ i ∈ s, N i)
```

Algebra · Polynomial · AlgebraMap

`Polynomial.dvd_term_of_dvd_eval_of_dvd_terms`

```
∀ {S : Type*} [CommRing S] {z p : S} {f : Polynomial S} (i : ℕ), p ∣ Polynomial.eval z f → (∀ (j : ℕ), j ≠ i → p ∣ f.coeff j * z ^ j) → p ∣ f.coeff i * z ^ i
```

Algebra · Polynomial · Div

`Polynomial.le_rootMultiplicity_iff` ‡

```
∀ {R : Type*} [CommRing R] {p : Polynomial R}, p ≠ 0 → ∀ {a : R} {n : ℕ}, n ≤ Polynomial.rootMultiplicity a p ↔ (Polynomial.X - Polynomial.C a) ^ n ∣ p
```

Algebra · Polynomial · Roots

`Polynomial.le_rootMultiplicity_map`

```
∀ {A B : Type*} [CommRing A] [CommRing B] {p : Polynomial A} {f : A →+* B}, Polynomial.map f p ≠ 0 → ∀ (a : A), Polynomial.rootMultiplicity a p ≤ Polynomial.rootMultiplicity (f a) (Polynomial.map f p)
```

MulOneClass → MulOne (3, magnitude 1)

Algebra · Group · Equiv · Defs

`MulEquiv.comp_left_injective`

```
∀ {M N P : Type*} [MulOneClass M] [MulOneClass N] [MulOneClass P] (e : M ≃* N), Function.Injective fun f => f.comp ↑e
```

`MulEquiv.comp_right_injective`

```
∀ {M N P : Type*} [MulOneClass M] [MulOneClass N] [MulOneClass P] (e : M ≃* N), Function.Injective fun f => (↑e).comp f
```

Algebra · Group · Even

`IsSquare.map` ‡

```
∀ {F : Type*} {α : Type u_2} {β : Type u_3} [MulOneClass α] [MulOneClass β] [FunLike F α β] [MonoidHomClass F α β] {a : α} (f : F), IsSquare a → IsSquare (f a)
```

AddCommGroup → AddGroup (1, magnitude 1)

Algebra · Order · Star · Basic

`IsSelfAdjoint.map'`

```
∀ {F E R : Type*} [AddCommGroup E] [PartialOrder E] [StarAddMonoid E] [NonUnitalRing R] [PartialOrder R] [StarRing R] [StarOrderedRing R] [SelfAdjointDecompose E] [FunLike F E R] [OrderHomClass F E R] [AddMonoidHomClass F E R] {a : E}, IsSelfAdjoint a → ∀ (f : F), IsSelfAdjoint (f a)
```

NonUnitalSemiring → NonUnitalNonAssocSemiring (1, magnitude 1)

Algebra · Algebra · Unitization

`Unitization.isStarProjection_inr_iff`

```
∀ {R A : Type*} [Semiring R] [StarRing R] [NonUnitalSemiring A] [StarRing A] [Module R A] {p : A}, IsStarProjection ↑p ↔ IsStarProjection p
```

StarRing → StarMul (3, unranked)

Algebra · Algebra · Unitization

`Unitization.isStarProjection_inr_iff`

```
∀ {R A : Type*} [Semiring R] [StarRing R] [NonUnitalSemiring A] [StarRing A] [Module R A] {p : A}, IsStarProjection ↑p ↔ IsStarProjection p
```

Algebra · Star · SelfAdjoint

`IsSelfAdjoint.div`

```
∀ {R : Type*} [Semifield R] [StarRing R] {x y : R}, IsSelfAdjoint x → IsSelfAdjoint y → IsSelfAdjoint (x / y)
```

Algebra · Star · StarProjection

`IsStarProjection.mul`

```
∀ {R : Type*} {p q : R} [NonUnitalSemiring R] [StarRing R], IsStarProjection p → IsStarProjection q → Commute p q → IsStarProjection (p * q)
```

CancelMonoid → LeftCancelMonoid (1, unranked)

Algebra · Ring · NonZeroDivisors

`IsMulTorsionFree.pow_right_injective`

```
∀ {M : Type*} [CancelMonoid M] [IsMulTorsionFree M] {x : M}, x ≠ 1 → Function.Injective fun n => x ^ n
```

IsDomain → Nontrivial (1, unranked)

Algebra · Order · CauSeq · Basic

`CauSeq.one_not_equiv_zero`

```
∀ {α : Type u_1} {β : Type u_2} [Field α] [LinearOrder α] [IsStrictOrderedRing α] [Ring β] [IsDomain β] (abv : β → α) [IsAbsoluteValue abv], ¬CauSeq.const abv 1 ≈ CauSeq.const abv 0
```

StarAddMonoid → InvolutiveStar (1, unranked)

Algebra · Order · Star · Basic

`IsSelfAdjoint.map'`

```
∀ {F E R : Type*} [AddCommGroup E] [PartialOrder E] [StarAddMonoid E] [NonUnitalRing R] [PartialOrder R] [StarRing R] [StarOrderedRing R] [SelfAdjointDecompose E] [FunLike F E R] [OrderHomClass F E R] [AddMonoidHomClass F E R] {a : E}, IsSelfAdjoint a → ∀ (f : F), IsSelfAdjoint (f a)
```

AlgebraicGeometry

1 survivor in 1 family; 1 gives up subtraction or division (†).

stated over	holds over	n	magnitude
`CommRing` †	`CommSemiring`	1	2

CommRing → CommSemiring (1, magnitude 2) †

AlgebraicGeometry · EllipticCurve · Affine · AddSubMap

`WeierstrassCurve.CXY` ?

signature not resolved

Analysis

145 survivors in 18 families; 30 give up subtraction or division (†).

stated over	holds over	n	magnitude
`CommRing`	`CommMonoid`	2	4
`NormedField`	`NormedRing`	6	3
`IsDomain`	`NoZeroDivisors`	2	3
`AddCommGroup` †	`AddCommMonoid`	25	2
`Ring` †	`Semiring`	5	2
`LinearOrder`	`PartialOrder`	4	2
`NormedRing`	`NonUnitalNormedRing`	4	2
`NormedAddCommGroup`	`SeminormedAddCommGroup`	66	1
`NontriviallyNormedField`	`NormedField`	12	1
`Field`	`DivisionRing`	6	1
`CommRing`	`Ring`	3	1
`PartialOrder`	`Preorder`	3	1
`MetricSpace`	`PseudoMetricSpace`	2	1
`AddCommGroup`	`AddGroup`	1	1
`CommGroup`	`Group`	1	1
`IsDomain`	`Nontrivial`	1	-1
`SeminormedCommGroup`	`CommGroup`	1	-1
`StarRing`	`StarMul`	1	-1

CommRing → CommMonoid (2, magnitude 4)

Analysis · Normed · Unbundled · SeminormFromBounded

`map_one_ne_zero`

```
∀ {R : Type*} [CommRing R] {f : R → ℝ} {c : ℝ}, f ≠ 0 → 0 ≤ f → (∀ (x y : R), f (x * y) ≤ c * f x * f y) → f 1 ≠ 0
```

`seminormFromBounded_aux`

```
∀ {R : Type*} [CommRing R] {f : R → ℝ} {c : ℝ}, 0 ≤ f → (∀ (x y : R), f (x * y) ≤ c * f x * f y) → ∀ (x : R), 0 ≤ c * f x
```

NormedField → NormedRing (6, magnitude 3)

Analysis · Asymptotics · SpecificAsymptotics

`Asymptotics.IsBigO.trans_tendsto_norm_atTop`

```
∀ {□ : Type u_1} [NormedField □] {α : Type u_2} {u v : α → □} {l : Filter α}, u =O[l] v → Filter.Tendsto (fun x => □ u x□) l Filter.atTop → Filter.Tendsto (fun x => □ v x□) l Filter.atTop
```

Analysis · Complex · UpperHalfPlane · FunctionsBoundedAtInfty

`UpperHalfPlane.zero_form_isBoundedAtImInfty`

```
∀ {α : Type u_1} [NormedField α], UpperHalfPlane.IsBoundedAtImInfty 0
```

Analysis · LocallyConvex · Basic

`Balanced.closure`

```
∀ {□ : Type u_1} {E : Type*} [NormedField □] [AddCommGroup E] [Module □ E] {A : Set E} [TopologicalSpace E] [ContinuousSMul □ E], Balanced □ A → Balanced □ (closure A)
```

Analysis · LocallyConvex · WithSeminorms

`WithSeminorms.continuous_of_continuous_comp`

```
∀ {□ : Type u_1} {□₂ : Type u_2} {E F : Type*} {ι' : Type u_5} [AddCommGroup E] [NormedField □] [Module □ E] [AddCommGroup F] [NormedField □₂] [Module □₂ F] {τ₁₂ : □ →+* □₂} [RingHomIsometric τ₁₂] {q : SeminormFamily □₂ F ι'} [TopologicalSpace E] [IsTopologicalAddGroup E] [TopologicalSpace F], WithSeminorms q → ∀ (f : E →□ □ [τ₁₂] F), (∀ (i : ι'), Continuous ⇑((q i).comp f)) → Continuous ⇑f
```

Analysis · Normed · Field · WithAbs

`AbsoluteValue.Completion.locallyCompactSpace` ‡

```
∀ {K : Type*} [Field K] {v : AbsoluteValue K ℝ} {L : Type*} [NormedField L] [CompleteSpace L] {f : WithAbs v →+* L} [LocallyCompactSpace L], Isometry ⇑f → LocallyCompactSpace v.Completion
```

Analysis · Seminorm

`Seminorm.smul_closedBall_subset`

```
∀ {□ : Type u_1} {E : Type*} [NormedDivisionRing □] [AddCommGroup E] [Module □ E] {p : Seminorm □ E} {k : □} {r : ℝ}, k • p.closedBall 0 r ⊆ p.closedBall 0 (□ k□ * r)
```

IsDomain → NoZeroDivisors (2, magnitude 3)

Analysis · Convex · Between

sbtw_iff_mem_image_Ioo_and_ne

```
∀ {R V P : Type*} [Ring R] [PartialOrder R] [AddCommGroup V] [Module R V] [AddTorsor V P] [IsOrderedRing R] [IsDomain R] [Module.IsTorsionFree R V] {x y z : P}, Sbtw R x y z ↔ y ∈ ⇑(AffineMap.lineMap x z) '' Set.Ioo 0 1 ∧ x ≠ z
```

Analysis · SpecificLimits · Normed

tendsto_zero_of_isBoundedUnder_smul_of_tendsto_cobounded

```
∀ {α : Type u_1} {R K : Type*} [NormedRing K] [IsDomain K] [NormedAddCommGroup R] [Module K R] [Module.IsTorsionFree K R] [NormSMulClass K R] {f : α → K} {g : α → R} {l : Filter α}, (Filter.IsBoundedUnder (fun x1 x2 => x1 ≤ x2) l fun x => □ f x • g x□ ) → Filter.Tendsto f l (Bornology.cobounded K) → Filter.Tendsto g l (nhds 0)
```

AddCommGroup → AddCommMonoid (25, magnitude 2) †

Analysis · Convex · Basic

Convex.add_smul_mem

```
∀ {□ : Type u_1} {E : Type*} [Ring □ ] [PartialOrder □ ] [AddCommGroup E] [Module □ E] {s : Set E} [AddRightMono □ ], Convex □ s → ∀ {x y : E}, x ∈ s → x + y ∈ s → ∀ {t : □ }, t ∈ Set.Icc 0 1 → x + t • y ∈ s
```

convex_openSegment

```
∀ {□ : Type u_1} {E : Type*} [CommSemiring □ ] [PartialOrder □ ] [IsStrictOrderedRing □ ] [AddCommGroup E] [Module □ E] (a b : E), Convex □ (openSegment □ a b)
```

Analysis · Convex · Cone · TensorProduct

PointedCone.basis_coord_mem_dual

```
∀ {R M : Type*} [CommRing R] [PartialOrder R] [IsOrderedRing R] [AddCommGroup M] [Module R M] {ι : Type u_3} (b : Module.Basis ι R M) (C : PointedCone R M), ↑C ⊆ ↑(PointedCone.hull R (Set.range ⇑b)) → ∀ (i : ι), b.coord i ∈ PointedCone.dual (Module.Dual.eval R M) ↑C
```

Analysis · Convex · EGauge

le_egauge_smul_left

```
∀ {□ : Type u_1} [NormedDivisionRing □ ] {E : Type*} [AddCommGroup E] [Module □ E] (c : □ ) (s : Set E) (x : E), egauge □ s x / □ c□ ₑ ≤ egauge □ (c • s) x
```

le_egauge_smul_right

```
∀ {□ : Type u_1} [NormedDivisionRing □ ] {E : Type*} [AddCommGroup E] [Module □ E] (c : □ ) (s : Set E) (x : E), □ c□ ₑ * egauge □ s x ≤ egauge □ s (c • x)
```

Analysis · Convex · Gauge

Absorbent.gauge_set_nonempty ‡

```
∀ {E : Type*} [AddCommGroup E] [Module ℝ E] {s : Set E} {x : E}, Absorbent ℝ s → {r | 0 < r ∧ x ∈ r • s}.Nonempty
```

bddBelow_gauge_set ?

signature not resolved

Analysis · Convex · Star

starConvex_iff_div

```
∀ {□ : Type u_1} {E : Type*} [Field □ ] [LinearOrder □ ] [IsStrictOrderedRing □ ] [AddCommGroup E] [Module □ E] {x : E} {s : Set E}, StarConvex □ x s ↔ ∀ □ y : E□ , y ∈ s → ∀ □ a b : □ □ , 0 ≤ a → 0 ≤ b → 0 < a + b → (a / (a + b)) • x + (b / (a + b)) • y ∈ s
```

Analysis · Convex · Strict

strictConvex_iff_div

```
∀ {□ : Type u_1} {E : Type*} [Field □ ] [LinearOrder □ ] [IsStrictOrderedRing □ ] [TopologicalSpace E] [AddCommGroup E] [Module □ E] {s : Set E}, StrictConvex □ s ↔ s.Pairwise fun x y => ∀ □ a b : □ □ , 0 < a → 0 < b → (a / (a + b)) • x + (b / (a + b)) • y ∈ interior s
```

Analysis · LocallyConvex · AbsConvex

balancedHull_convexHull_subset_absConvexHull

```
∀ (□ : Type u_1) {E : Type*} [NontriviallyNormedField □ ] [PartialOrder □ ] [AddCommGroup E] [Module □ E] {s : Set E}, balancedHull □ ((convexHull □ ) s) ⊆ (absConvexHull □ ) s
```

Analysis · LocallyConvex · BalancedCoreHull

IsOpen.balancedHull

```
∀ {□ : Type u_1} {E : Type*} [NormedDivisionRing □ ] [AddCommGroup E] [Module □ E] [TopologicalSpace E] [ContinuousConstSMul □ E] {s : Set E}, IsOpen s → 0 ∈ s → IsOpen (balancedHull □ s)
```

balancedCoreAux_balanced

```
∀ {□ : Type u_1} {E : Type*} [NormedDivisionRing □ ] [AddCommGroup E] [Module □ E] {s : Set E}, 0 ∈ balancedCoreAux □ s → Balanced □ (balancedCoreAux □ s)
```

balancedCoreAux_maximal

```
∀ {□ : Type u_1} {E : Type*} [NormedDivisionRing □ ] [AddCommGroup E] [Module □ E] {s t : Set E}, t ⊆ s → Balanced □ t → t ⊆ balancedCoreAux □ s
```

balancedCoreAux_subset

```
∀ {□ : Type u_1} {E : Type*} [NormedDivisionRing □ ] [AddCommGroup E] [Module □ E] (s : Set E), balancedCoreAux □ s ⊆ s
```

`balancedHull.balanced`

```
∀ {𝕜 : Type u_1} {E : Type*} [SeminormedRing 𝕜] [AddCommGroup E] [Module 𝕜 E] (s : Set E), Balanced 𝕜 (balancedHull 𝕜 s)
```

`subset_balancedHull`

```
∀ (𝕜 : Type u_1) {E : Type*} [SeminormedRing 𝕜] [AddCommGroup E] [Module 𝕜 E] [NormOneClass 𝕜] {s : Set E}, s ⊆ balancedHull 𝕜 s
```

Analysis · LocallyConvex · Basic

`Balanced.add`

```
∀ {𝕜 : Type u_1} {E : Type*} [SeminormedRing 𝕜] [AddCommGroup E] [Module 𝕜 E] {s t : Set E}, Balanced 𝕜 s → Balanced 𝕜 t → Balanced 𝕜 (s + t)
```

`Balanced.closure`

```
∀ {𝕜 : Type u_1} {E : Type*} [NormedField 𝕜] [AddCommGroup E] [Module 𝕜 E] {A : Set E} [TopologicalSpace E] [ContinuousSMul 𝕜 E], Balanced 𝕜 A → Balanced 𝕜 (closure A)
```

`Balanced.convexHull`

```
∀ {𝕜 : Type u_1} {E : Type*} [NontriviallyNormedField 𝕜] [AddCommGroup E] [Module 𝕜 E] {s : Set E} [PartialOrder 𝕜], Balanced 𝕜 s → Balanced 𝕜 ((convexHull 𝕜) s)
```

`Balanced.starConvex`

```
∀ {E : Type*} [AddCommGroup E] [Module ℝ E] {s : Set E}, Balanced ℝ s → StarConvex ℝ 0 s
```

`Balanced.zero_insert_interior`

```
∀ {𝕜 : Type u_1} {E : Type*} [NormedField 𝕜] [AddCommGroup E] [Module 𝕜 E] {A : Set E} [TopologicalSpace E] [ContinuousSMul 𝕜 E], Balanced 𝕜 A → Balanced 𝕜 (insert 0 (interior A))
```

`absorbent_nhds_zero`

```
∀ {𝕜 : Type u_1} {E : Type*} [NormedField 𝕜] [AddCommGroup E] [Module 𝕜 E] {A : Set E} [TopologicalSpace E] [ContinuousSMul 𝕜 E], A ∈ nhds 0 → Absorbent 𝕜 A
```

`balanced_zero`

```
∀ {𝕜 : Type u_1} {E : Type*} [SeminormedRing 𝕜] [AddCommGroup E] [Module 𝕜 E], Balanced 𝕜 0
```

Analysis · LocallyConvex · Bounded

`Bornology.IsVonNBounded.closure`

```
∀ {𝕜 : Type u_1} {E : Type*} [NormedField 𝕜] [AddCommGroup E] [Module 𝕜 E] [TopologicalSpace E] [T1Space E] [RegularSpace E] [ContinuousConstSMul 𝕜 E] {a : Set E}, Bornology.IsVonNBounded 𝕜 a → Bornology.IsVonNBounded 𝕜 (closure a)
```

`Bornology.IsVonNBounded.of_topologicalSpace_le`

```
∀ {𝕜 : Type u_1} {E : Type*} [SeminormedRing 𝕜] [AddCommGroup E] [Module 𝕜 E] {t t' : TopologicalSpace E}, t ≤ t' → ∀ {s : Set E}, Bornology.IsVonNBounded 𝕜 s → Bornology.IsVonNBounded 𝕜 s
```

Ring → Semiring (5, magnitude 2) †

Analysis · Convex · Basic

`Convex.neg`

```
∀ {𝕜 : Type u_1} {E : Type*} [Ring 𝕜] [PartialOrder 𝕜] [AddCommGroup E] [Module 𝕜 E] {s : Set E}, Convex 𝕜 s → Convex 𝕜 (-s)
```

`convex_vadd`

```
∀ {𝕜 : Type u_1} {E : Type*} [Ring 𝕜] [PartialOrder 𝕜] [AddCommGroup E] [Module 𝕜 E] {s : Set E} (a : E), Convex 𝕜 (a +ᵥ s) ↔ Convex 𝕜 s
```

Analysis · Convex · Star

`StarConvex.neg`

```
∀ {𝕜 : Type u_1} {E : Type*} [Ring 𝕜] [PartialOrder 𝕜] [AddCommGroup E] [Module 𝕜 E] {x : E} {s : Set E}, StarConvex 𝕜 x s → StarConvex 𝕜 (-x) (-s)
```

Analysis · Convex · Strict

`StrictConvex.neg`

```
∀ {𝕜 : Type u_1} {E : Type*} [Ring 𝕜] [PartialOrder 𝕜] [TopologicalSpace E] [AddCommGroup E] [Module 𝕜 E] {s : Set E} [IsTopologicalAddGroup E], StrictConvex 𝕜 s → StrictConvex 𝕜 (-s)
```

Analysis · Normed · Operator · ContinuousLinearMap

`ContinuousLinearMap.isUniformEmbedding_of_bound`

```
∀ {𝕜 : Type u_1} {𝕜₂ : Type u_2} {E F : Type*} [Ring 𝕜] [Ring 𝕜₂] [NormedAddCommGroup E] [NormedAddCommGroup F] [Module 𝕜 E] [Module 𝕜₂ F] {σ : 𝕜 →+* 𝕜₂} (f : E →SL[σ] F) {K : NNReal}, (∀ (x : E), ‖x‖ ≤ ↑K * ‖f x‖) → IsUniformEmbedding ⇑f
```

LinearOrder → PartialOrder (4, magnitude 2)

Analysis · Asymptotics · SpecificAsymptotics

`pow_div_pow_eventuallyEq_atBot` ‡

```
∀ {𝕜 : Type u_1} [Field 𝕜] [LinearOrder 𝕜] [IsStrictOrderedRing 𝕜] {p q : ℕ}, (fun x => x ^ p / x ^ q) =ᶠ[Filter.atBot] fun x => x ^ (↑p - ↑q)
```

`pow_div_pow_eventuallyEq_atTop` ‡

```
∀ {𝕜 : Type u_1} [Field 𝕜] [LinearOrder 𝕜] [IsStrictOrderedRing 𝕜] {p q : ℕ}, (fun x => x ^ p / x ^ q) =ᶠ[Filter.atTop] fun x => x ^ (↑p - ↑q)
```

Analysis · Convex · Basic

`Convex_subadditive_le`

```
∀ {𝕜 : Type u_1} {E : Type*} [Semiring 𝕜] [AddCommMonoid E] [LinearOrder 𝕜] [IsOrderedRing 𝕜] [SMul 𝕜 E] {f : E → 𝕜}, (∀ (x y : E), f (x + y) ≤ f x + f y) → (∀ ⦃c : 𝕜⦄ (x : E), 0 ≤ c → f (c • x) ≤ c * f x) → ∀ (B : 𝕜), Convex 𝕜 {x | f x ≤ B}
```

Analysis · SpecificLimits · ArithmeticGeometric

`div_lt_arithGeom` ‡

```
∀ {R : Type*} {a b u₀ : R} [Field R] [LinearOrder R] [IsStrictOrderedRing R], 0 < a → a ≠ 1 → b / (1 - a) < u₀ → ∀ (n : ℕ), b / (1 - a) < arithGeom a b u₀ n
```

NormedRing → NonUnitalNormedRing (4, magnitude 2)

Analysis · Asymptotics · Defs

`Asymptotics.isBigOWith_self_const_mul`

```
∀ {α : Type u_1} {S : Type*} [NormedRing S] [NormMulClass S] {c : S}, c ≠ 0 → ∀ (f : α → S) (l : Filter α), Asymptotics.IsBigOWith ‖c‖⁻¹ l f fun x => c * f x
```

Analysis · Asymptotics · Lemmas

`Asymptotics.continuousAt_iff_isLittleO`

```
∀ {F : Type*} [Norm F] {α : Type u_2} {E : Type*} [NormedRing E] [One F] [NormOneClass F] [TopologicalSpace α] {f : α → E} {x : α}, ContinuousAt f x ↔ (fun x_1 => f x_1 - f x) =o[nhds x] fun x => 1
```

Analysis · Normed · Ring · InfiniteSum

`Summable.mul_norm`

```
∀ {R : Type*} {ι : Type u_2} {ι' : Type u_3} [NormedRing R] {f : ι → R} {g : ι' → R}, (Summable fun x => ‖f x‖) → (Summable fun x => ‖g x‖) → Summable fun x => ‖f x.1 * g x.2‖
```

`summable_norm_sum_mul_antidiagonal_of_summable_norm` ‡

```
∀ {R : Type*} [NormedRing R] {f g : ℕ → R}, (Summable fun x => ‖f x‖) → (Summable fun x => ‖g x‖) → Summable fun n => ‖∑ kl ∈ Finset.HasAntidiagonal.antidiagonal n, f kl.1 * g kl.2‖
```

NormedAddCommGroup → SeminormedAddCommGroup (66, magnitude 1)

Analysis · Asymptotics · AsymptoticEquivalent

`Asymptotics.IsEquivalent.add_isLittleO`

```
∀ {α : Type u_1} {β : Type u_2} [NormedAddCommGroup β] {u v w : α → β} {l : Filter α}, Asymptotics.IsEquivalent l u v → w =o[l] v → Asymptotics.IsEquivalent l (u + w) v
```

`Asymptotics.IsEquivalent.comp_tendsto`

```
∀ {α : Type u_1} {β : Type u_2} [NormedAddCommGroup β] {l : Filter α} {α₂ : Type u_3} {f g : α₂ → β} {l' : Filter α₂}, Asymptotics.IsEquivalent l' f g → ∀ {k : α → α₂}, Filter.Tendsto k l l' → Asymptotics.IsEquivalent l (f ∘ k) (g ∘ k)
```

`Asymptotics.IsEquivalent.congr_left`

```
∀ {α : Type u_1} {β : Type u_2} [NormedAddCommGroup β] {u v w : α → β} {l : Filter α}, Asymptotics.IsEquivalent l u v → u =ᶠ[l] w → Asymptotics.IsEquivalent l w v
```

`Asymptotics.IsEquivalent.mono`

```
∀ {α : Type u_1} {β : Type u_2} [NormedAddCommGroup β] {l : Filter α} {f g : α → β} {l' : Filter α}, Asymptotics.IsEquivalent l' f g → l ≤ l' → Asymptotics.IsEquivalent l f g
```

`Asymptotics.IsEquivalent.refl`

```
∀ {α : Type u_1} {β : Type u_2} [NormedAddCommGroup β] {u : α → β} {l : Filter α}, Asymptotics.IsEquivalent l u u
```

`Asymptotics.isEquivalent_map`

```
∀ {α : Type u_1} {β : Type u_2} [NormedAddCommGroup β] {l : Filter α} {α₂ : Type u_3} {f g : α₂ → β} {k : α → α₂}, Asymptotics.IsEquivalent (Filter.map k l) f g ↔ Asymptotics.IsEquivalent l (f ∘ k) (g ∘ k)
```

Analysis · Calculus · FDeriv · Add

`HasFDerivAtFilter.add`

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f g : E → F} {f' g' : E →L[𝕜] F} {L : Filter (E × E)}, HasFDerivAtFilter f f' L → HasFDerivAtFilter g g' L → HasFDerivAtFilter (f + g) (f' + g') L
```

`HasFDerivAtFilter.fun_sum`

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {L : Filter (E × E)} {ι : Type u_4} {u : Finset ι} {A : ι → E → F} {A' : ι → E →L[𝕜] F}, (∀ i ∈ u, HasFDerivAtFilter (A i) (A' i) L) → HasFDerivAtFilter (fun y => ∑ i ∈ u, A i y) (∑ i ∈ u, A' i) L
```

`HasStrictFDerivAt.fun_sum`

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup
```

```
F] [NormedSpace 𝕜 F] {x : E} {ι : Type u_4} {u : Finset ι} {A : ι → E → F} {A' : ι → E →L[𝕜] F}, (∀ i ∈ u, HasStrictFDerivAt (A i) (A' i) x) → HasStrictFDerivAt (fun y => ∑ i ∈ u, A i y) (∑ i ∈ u, A' i) x
```

hasFDerivWithinAt_comp_add_left

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E} {s : Set E} (a : E), HasFDerivWithinAt (fun x => f (a + x)) f' s x ↔ HasFDerivWithinAt f f' (a +ᵥ s) (a + x)
```

hasStrictFDerivAt_add_const_iff

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E} (c : F), HasStrictFDerivAt (fun x => f x + c) f' x ↔ HasStrictFDerivAt f f' x
```

Analysis · Calculus · FDeriv · Basic

HasFDerivAt.isBigO_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x₀ : E}, HasFDerivAt f f' x₀ → (fun x => f x - f x₀) =O[nhds x₀] fun x => x - x₀
```

HasFDerivAt.isTheta_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E}, HasFDerivAt f f' x → Topology.IsInducing ⇑f' → (fun x_1 => f x_1 - f x) =Θ[nhds x] fun x_1 => x_1 - x
```

HasFDerivAtFilter.isBigO_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {L : Filter (E × E)}, HasFDerivAtFilter f f' L → (fun p => f p.1 - f p.2) =O[L] fun p => p.1 - p.2
```

HasFDerivAtFilter.isEquivalent_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [AddCommGroup E] [Module 𝕜 E] [TopologicalSpace E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {L : Filter (E × E)}, HasFDerivAtFilter f f' L → Topology.IsInducing ⇑f' → Asymptotics.IsEquivalent L (fun p => f p.1 - f p.2) fun p => f' (p.1 - p.2)
```

HasFDerivAtFilter.isTheta_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {L : Filter (E × E)}, HasFDerivAtFilter f f' L → Topology.IsInducing ⇑f' → (fun p => f p.1 - f p.2) =Θ[L] fun p => p.1 - p.2
```

HasFDerivWithinAt.isBigO_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x₀ : E} {s : Set E}, HasFDerivWithinAt f f' s x₀ → (fun x => f x - f x₀) =O[nhdsWithin x₀ s] fun x => x - x₀
```

HasFDerivWithinAt.isTheta_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E} {s : Set E}, HasFDerivWithinAt f f' s x → Topology.IsInducing ⇑f' → (fun x_1 => f x_1 - f x) =Θ[nhdsWithin x s] fun x_1 => x_1 - x
```

HasStrictFDerivAt.exists_lipschitzOnWith_of_nnnorm_lt

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E}, HasStrictFDerivAt f f' x → ∀ (K : NNReal), ‖f'‖₊ < K → ∃ s ∈ nhds x, LipschitzOnWith K f s
```

HasStrictFDerivAt.isBigO_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E}, HasStrictFDerivAt f f' x → (fun p => f p.1 - f p.2) =O[nhds (x, x)] fun p => p.1 - p.2
```

HasStrictFDerivAt.isTheta_sub

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E}, HasStrictFDerivAt f f' x → Topology.IsInducing ⇑f' → (fun p => f p.1 - f p.2) =Θ[nhds (x, x)] fun p => p.1 - p.2
```

hasFDerivAt_iff_isLittleO_nhds_zero

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {f' : E →L[𝕜] F} {x : E}, HasFDerivAt f f' x ↔ (fun h => f (x + h) - f x - f' h) =o[nhds 0] fun h => h
```

Analysis · Calculus · FDeriv · Comp

HasFDerivAtFilter.comp

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {G : Type*} [NormedAddCommGroup G] [NormedSpace 𝕜 G] {f : E → F} {f' : E →L[𝕜] F} {L : Filter (E × E)} {g : F → G} {g' : F →L[𝕜] G} {L' : Filter (F × F)}, HasFDerivAtFilter g g' L' → HasFDerivAtFilter f f' L → Filter.Tendsto (Prod.map f f) L L' → HasFDerivAtFilter (g ∘ f) (g' ∘SL f') L
```

HasFDerivAtFilter.comp

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {G : Type*} [NormedAddCommGroup G] [NormedSpace 𝕜 G] {f : E → F} {f' : E →L[𝕜] F} {L : Filter (E × E)} {g : F → G} {g' : F →L[𝕜] G} {L' : Filter (F × F)}, HasFDerivAtFilter g g' L' → HasFDerivAtFilter f f' L → Filter.Tendsto (Prod.map f f) L L' → HasFDerivAtFilter (g ∘ f) (g' ∘SL f') L
```

Analysis · Calculus · FDeriv · CompCLM

HasFDerivAt.continuousAlternatingMap_apply_const

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {G : Type*} [NormedAddCommGroup G] [NormedSpace 𝕜 G] {x : E} {ι : Type u_5} {c : E → F [𝕜 ^ι]→L[𝕜] G} {c' : E → L[𝕜] F [𝕜 ^ι]→L[𝕜] G} [Fintype ι], HasFDerivAt c c' x → ∀ (u : ι → F), HasFDerivAt (fun y => (c y) u) (c'.flipAlternating u) x
```

HasFDerivWithinAt.continuousAlternatingMap_apply_const

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {G : Type*} [NormedAddCommGroup G] [NormedSpace 𝕜 G] {x : E} {s : Set E} {ι : Type u_5} {c : E → F [𝕜 ^ι]→L[𝕜] G} {c' : E →L[𝕜] F [𝕜 ^ι]→L[𝕜] G} [Fintype ι], HasFDerivWithinAt c c' s x → ∀ (u : ι → F), HasFDerivWithinAt (fun x => (c x) u) (c'.flipAlternating u) s x
```

HasStrictFDerivAt.continuousAlternatingMap_apply_const

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {G : Type*} [NormedAddCommGroup G] [NormedSpace 𝕜 G] {x : E} {ι : Type u_5} {c : E → F [𝕜 ^ι]→L[𝕜] G} {c' : E → L[𝕜] F [𝕜 ^ι]→L[𝕜] G} [Fintype ι], HasStrictFDerivAt c c' x → ∀ (u : ι → F), HasStrictFDerivAt (fun x => (c x) u) (c'.flipAlternating u) x
```

Analysis · Calculus · FDeriv · Equiv

ContinuousLinearEquiv.hasFDerivAt

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {x : E} (iso : E ≃L[𝕜] F), HasFDerivAt (⇑iso) (↑iso) x
```

ContinuousLinearEquiv.hasFDerivWithinAt

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {x : E} {s : Set E} (iso : E ≃L[𝕜] F), HasFDerivWithinAt (⇑iso) (↑iso) s x
```

ContinuousLinearEquiv.hasStrictFDerivAt

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {x : E} (iso : E ≃L[𝕜] F), HasStrictFDerivAt (⇑iso) (↑iso) x
```

has_fderiv_at_filter_real_equiv

```
∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ F] {f : E → F} {f' : E → L[ℝ] F} {x : E} {L : Filter E}, Filter.Tendsto (fun x' => ‖x' - x‖⁻¹ * ‖f x' - f x - f' (x' - x)‖) L (nhds 0) ↔ Filter.Tendsto (fun x' => ‖x' - x‖⁻¹ • (f x' - f x - f' (x' - x))) L (nhds 0)
```

Analysis · Calculus · FDeriv · Linear

IsBoundedLinearMap.hasFDerivAtFilter

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {f : E → F} {L : Filter (E × E)} (h : IsBoundedLinearMap 𝕜 f), HasFDerivAtFilter f (IsBoundedLinearMap.toContinuousLinearMap f h) L
```

Analysis · Calculus · FDeriv · Prod

HasFDerivAtFilter.prodMk

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {G : Type*} [NormedAddCommGroup G] [NormedSpace 𝕜 G] {f₁ : E → F} {f₁' : E →L[𝕜] F} {L : Filter (E × E)} {f₂ : E → G} {f₂' : E →L[𝕜] G}, HasFDerivAtFilter f₁ f₁' L → HasFDerivAtFilter f₂ f₂' L → HasFDerivAtFilter (fun x => (f₁ x, f₂ x)) (f₁'.prod f₂') L
```

HasFDerivAtFilter.prodMk


```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {G : Type*} [NormedAddCommGroup G] [NormedSpace 𝕜 G] {f₁ : E → F} {f₁' : E →L[𝕜] F} {L : Filter (E × E)} {f₂ : E → G} {f₂' : E →L[𝕜] G}, HasFDerivAtFilter f₁ f₁' L → HasFDerivAtFilter f₂ f₂' L → HasFDerivAtFilter (fun x => (f₁ x, f₂ x)) (f₁'.prod f₂') L
```


hasFDerivAtFilter_fst

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {L : Filter ((E × F) × E × F)}, HasFDerivAtFilter Prod.fst (ContinuousLinearMap.fst 𝕜 E F) L
```

hasFDerivAtFilter_snd

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜 F] {L : Filter ((E × F) × E × F)}, HasFDerivAtFilter Prod.snd (ContinuousLinearMap.snd 𝕜 E F) L
```

Analysis · Calculus · FDeriv · RestrictScalars

HasFDerivAtFilter.restrictScalars

```
∀ (𝕜 : Type u_1) [NontriviallyNormedField 𝕜] {𝕜' : Type u_2} [NontriviallyNormedField 𝕜'] [NormedAlgebra 𝕜 𝕜'] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] [NormedSpace 𝕜' E] [IsScalarTower 𝕜 𝕜' E] {F : Type*} [NormedAddCommGroup F] [NormedSpace 𝕜
```

F] [NormedSpace 𝕜' F] [IsScalarTower 𝕜 𝕜' F] {f : E → F} {f' : E →L[𝕜'] F} {L : Filter (E × E)}, HasFDerivAtFilter f f' L → HasFDerivAtFilter f (ContinuousLinearMap.restrictScalars 𝕜 f') L

Analysis · Complex · Hadamard

Complex.HadamardThreeLines.scale_bddAbove

∀ {E : Type*} [NormedAddCommGroup E] {f : ℂ → E} {l u : ℝ}, l < u → BddAbove (norm ∘ f '' Complex.HadamardThreeLines.verticalClosedStrip l u) → BddAbove (norm ∘ Complex.HadamardThreeLines.scale f l u '' Complex.HadamardThreeLines.verticalClosedStrip 0 1)

Complex.HadamardThreeLines.scale_bound_left

∀ {E : Type*} [NormedAddCommGroup E] {f : ℂ → E} {l u a : ℝ}, (∀ z ∈ Complex.re ⁻¹' {l}, ‖f z‖ ≤ a) → ∀ z ∈ Complex.re ⁻¹' {0}, ‖Complex.HadamardThreeLines.scale f l u z‖ ≤ a

Complex.HadamardThreeLines.scale_bound_right

∀ {E : Type*} [NormedAddCommGroup E] {f : ℂ → E} {l u b : ℝ}, (∀ z ∈ Complex.re ⁻¹' {u}, ‖f z‖ ≤ b) → ∀ z ∈ Complex.re ⁻¹' {1}, ‖Complex.HadamardThreeLines.scale f l u z‖ ≤ b

Analysis · Convex · AmpleSet

ampleSet_univ ‡

∀ {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ F], AmpleSet Set.univ

Analysis · Convex · Gauge

le_gauge_of_subset_closedBall

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {s : Set E} {r : ℝ} {x : E}, Absorbent ℝ s → 0 ≤ r → s ⊆ Metric.closedBall 0 r → ‖x‖ / r ≤ gauge s x

Analysis · Fourier · FourierTransform

Real.fourierIntegral_convergent_iff'

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℂ E] {V W : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V] [NormedAddCommGroup W] [NormedSpace ℝ W] [MeasurableSpace V] [BorelSpace V] {μ : MeasureTheory.Measure V} {f : V → E} (L : V →L[ℝ] W →L[ℝ] ℝ) (w : W), MeasureTheory.Integrable (fun v => Real.fourierChar (-(L v) w) • f v) μ ↔ MeasureTheory.Integrable f μ

Analysis · InnerProductSpace · GramMatrix

Matrix.posSemidef_opNorm_smul_gram_sub_gram

∀ {E n : Type*} {𝕜 : Type u_3} [RCLike 𝕜] [NormedAddCommGroup E] [InnerProductSpace 𝕜 E] {F : Type*} [NormedAddCommGroup F] [InnerProductSpace 𝕜 F] (v : n → E) (f : E →L[𝕜] F), (‖f‖ ^ 2 • Matrix.gram 𝕜 v - Matrix.gram 𝕜 (⇑f ∘ v)).PosSemidef

Analysis · Normed · Group · CocompactMap

CocompactMapClass.norm_le

∀ {E F : Type*} {𝓕 : Type u_3} [NormedAddCommGroup E] [NormedAddCommGroup F] {f : 𝓕} [ProperSpace F] [FunLike 𝓕 E F] [CocompactMapClass 𝓕 E F] (ε : ℝ), ∃ r, ∀ (x : E), r < ‖x‖ → ε < ‖f x‖

Filter.tendsto_cocompact_cocompact_of_norm

∀ {E F : Type*} [NormedAddCommGroup E] [NormedAddCommGroup F] [ProperSpace E] {f : E → F}, (∀ (ε : ℝ), ∃ r, ∀ (x : E), r < ‖x‖ → ε < ‖f x‖) → Filter.Tendsto f (Filter.cocompact E) (Filter.cocompact F)

Analysis · Normed · Group · Completeness

NormedAddCommGroup.completeSpace_of_summable_imp_tendsto

∀ {E : Type*} [NormedAddCommGroup E], (∀ (u : ℕ → E), (Summable fun x => ‖u x‖) → ∃ a, Filter.Tendsto (fun n => ∑ i ∈ Finset.range n, u i) Filter.atTop (nhds a)) → CompleteSpace E

NormedAddCommGroup.summable_imp_tendsto_of_complete

∀ {E : Type*} [NormedAddCommGroup E] [CompleteSpace E] (u : ℕ → E), (Summable fun x => ‖u x‖) → ∃ a, Filter.Tendsto (fun n => ∑ i ∈ Finset.range n, u i) Filter.atTop (nhds a)

Analysis · Normed · Module · Connected

Metric.contractibleSpace_ball

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ℝ}, 0 < r → ContractibleSpace ↑(Metric.ball x r)

Metric.contractibleSpace_closedBall

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ℝ}, 0 ≤ r → ContractibleSpace ↑(Metric.closedBall x r)

Metric.contractibleSpace_eball

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ENNReal}, 0 < r → ContractibleSpace ↑(Metric.eball x r)

Metric.isPathConnected_ball

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ℝ}, 0 < r → IsPathConnected (Metric.ball x r)

Metric.isPathConnected_closedBall

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ℝ}, 0 ≤ r → IsPathConnected (Metric.closedBall x r)

Metric.isPathConnected_eball

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ENNReal}, 0 < r → IsPathConnected (Metric.eball x r)

Metric.isPreconnected_ball

∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ℝ}, IsPreconnected (Metric.ball x r)

Metric.isPreconnected_closedBall

```
∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ℝ}, IsPreconnected (Metric.closedBall x r)
```

`Metric.isPreconnected_closedEBall`

```
∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ENNReal}, IsPreconnected (Metric.closedEBall x r)
```

`Metric.isPreconnected_eball`

```
∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {x : E} {r : ENNReal}, IsPreconnected (Metric.eball x r)
```

Analysis · Normed · Module · FiniteDimension

`summable_of_isBigO'`

```
∀ {ι : Type u_1} {E F : Type*} [NormedAddCommGroup E] [CompleteSpace E] [NormedAddCommGroup F] [NormedSpace ℝ F] [FiniteDimensional ℝ F] {f : ι → E} {g : ι → F}, Summable g → f =O[Filter.cofinite] g → Summable f
```

`summable_of_isBigO_nat'`

```
∀ {E F : Type*} [NormedAddCommGroup E] [CompleteSpace E] [NormedAddCommGroup F] [NormedSpace ℝ F] [FiniteDimensional ℝ F] {f : ℕ → E} {g : ℕ → F}, Summable g → f =O[Filter.atTop] g → Summable f
```

Analysis · Normed · Module · RieszLemma

`riesz_lemma` ‡

```
∀ {𝕜 : Type u_1} [NormedField 𝕜] {E : Type*} [NormedAddCommGroup E] [NormedSpace 𝕜 E] {F : Subspace 𝕜 E}, IsClosed ↑F → (∃ x, x ∉ F) → ∀ {r : ℝ}, r < 1 → ∃ x₀ ∉ F, ∀ y ∈ F, r * ‖x₀‖ ≤ ‖x₀ - y‖
```

Analysis · SpecialFunctions · JapaneseBracket

`le_rpow_one_add_norm_iff_norm_le`

```
∀ {E : Type*} [NormedAddCommGroup E] {r t : ℝ}, 0 < r → 0 < t → ∀ (x : E), t ≤ (1 + ‖x‖) ^ (-r) ↔ ‖x‖ ≤ t ^ (-r⁻¹) - 1
```

`one_add_norm_le_sqrt_two_mul_sqrt` ‡

```
∀ {E : Type*} [NormedAddCommGroup E] (x : E), 1 + ‖x‖ ≤ √2 * √(1 + ‖x‖ ^ 2)
```

`sqrt_one_add_norm_sq_le`

```
∀ {E : Type*} [NormedAddCommGroup E] (x : E), √(1 + ‖x‖ ^ 2) ≤ 1 + ‖x‖
```

Analysis · SpecialFunctions · Sigmoid

`Continuous.sigmoid` ‡

```
∀ {E : Type*} [NormedAddCommGroup E] {f : E → ℝ}, Continuous f → Continuous (Real.sigmoid ∘ f)
```

Analysis · SpecificLimits · Normed

`Monotone.cauchySeq_series_mul_of_tendsto_zero_of_bounded`

```
∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {b : ℝ} {f : ℕ → ℝ} {z : ℕ → E}, Monotone f → Filter.Tendsto f Filter.atTop (nhds 0) → (∀ (n : ℕ), ‖∑ i ∈ Finset.range n, z i‖ ≤ b) → CauchySeq fun n => ∑ i ∈ Finset.range n, f i • z i
```

NontriviallyNormedField → NormedField (12, magnitude 1)

Analysis · Complex · CoveringMap

`isQuotientCoveringMap_npow` ‡

```
∀ {𝕜 : Type u_1} [NontriviallyNormedField 𝕜] [ProperSpace 𝕜] (n : ℕ), ↑n ≠ 0 → (Function.Surjective fun x => x ^ n) → IsQuotientCoveringMap (fun x => x ^ n) ↥(powMonoidHom n).ker
```

Analysis · LocallyConvex · AbsConvex

`absConvexHull_add_subset`

```
∀ (𝕜 : Type u_1) {E : Type*} [NontriviallyNormedField 𝕜] [PartialOrder 𝕜] [AddCommGroup E] [Module 𝕜 E] {s t : Set E}, (absConvexHull 𝕜) (s + t) ⊆ (absConvexHull 𝕜) s + (absConvexHull 𝕜) t
```

`balancedHull_convexHull_subset_absConvexHull`

```
∀ (𝕜 : Type u_1) {E : Type*} [NontriviallyNormedField 𝕜] [PartialOrder 𝕜] [AddCommGroup E] [Module 𝕜 E] {s : Set E}, balancedHull 𝕜 ((convexHull 𝕜) s) ⊆ (absConvexHull 𝕜) s
```

Analysis · LocallyConvex · Basic

`Balanced.convexHull`

```
∀ {𝕜 : Type u_1} {E : Type*} [NontriviallyNormedField 𝕜] [AddCommGroup E] [Module 𝕜 E] {s : Set E} [PartialOrder 𝕜], Balanced 𝕜 s → Balanced 𝕜 ((convexHull 𝕜) s)
```

Analysis · LocallyConvex · Polar

`StrongDual.polar_nonempty`

```
∀ (𝕜 : Type u_1) [NontriviallyNormedField 𝕜] {E : Type*} [AddCommMonoid E] [TopologicalSpace E] [Module 𝕜 E] (s : Set E), (StrongDual.polar 𝕜 s).Nonempty
```

`StrongDual.zero_mem_polar`

```
∀ (𝕜 : Type u_1) [NontriviallyNormedField 𝕜] {E : Type*} [AddCommMonoid E] [TopologicalSpace E] [Module 𝕜 E] (s : Set E), 0 ∈ StrongDual.polar 𝕜 s
```

Analysis · LocallyConvex · WithSeminorms

`WithSeminorms.firstCountableTopology`

```
∀ {𝕜 : Type u_1} {E : Type*} {ι : Type u_3} [NontriviallyNormedField 𝕜] [AddCommGroup E] [Module 𝕜 E] [Countable ι] {p : SeminormFamily 𝕜 E ι} [TopologicalSpace E], WithSeminorms p → FirstCountableTopology E
```

Analysis · Normed · Algebra · GelfandFormula

spectrum.hasDerivAt_resolvent_const_right

```
∀ {□ : Type u_1} {A : Type*} [NontriviallyNormedField □ ] [NontriviallyNormedField A] [NormedAlgebra □ A] [CompleteSpace A] {a : A}
{k : □ }, k ∈ resolventSet □ a → HasDerivAt (fun x => resolvent x k) (resolvent a k ^ 2) a
```

Analysis · Normed · Algebra · Spectrum

spectrum.eventually_isUnit_resolvent

```
∀ {□ : Type u_1} {A : Type*} [NontriviallyNormedField □ ] [NormedRing A] [NormedAlgebra □ A] [CompleteSpace A] (a : A), ∀ᶠ (z : □ )
in Bornology.cobounded □ , IsUnit (resolvent a z)
```

spectrum.isUnit_one_sub_smul_of_lt_inv_radius

```
∀ {□ : Type u_1} {A : Type*} [NontriviallyNormedField □ ] [NormedRing A] [NormedAlgebra □ A] {a : A} {z : □ }, ↑□ z□ ₊ <
(spectralRadius □ a)⁻¹ → IsUnit (1 - z • a)
```

spectrum.resolvent_isBigO_inv ‡

```
∀ {□ : Type u_1} {A : Type*} [NontriviallyNormedField □ ] [NormedRing A] [NormedAlgebra □ A] [CompleteSpace A] (a : A), resolvent a
=O[Bornology.cobounded □ ] Inv.inv
```

Analysis · Normed · Module · WeakDual

WeakDual.exists_countable_separating

```
∀ (□ : Type u_1) (E : Type u_2) [NontriviallyNormedField □ ] [SeminormedAddCommGroup E] [NormedSpace □ E] [TopologicalSpace.
SeparableSpace E], ∃ gs, (∀ (n : ℕ), Continuous (gs n)) ∧ ∀ □ x y : WeakDual □ E□ , x ≠ y → ∃ n, gs n x ≠ gs n y
```

Field → DivisionRing (6, magnitude 1)

Analysis · AbsoluteValue · Equivalence

AbsoluteValue.IsEquiv.log_div_log_pos

```
∀ {F : Type*} [Field F] {v w : AbsoluteValue F ℝ}, v.IsEquiv w → ∀ {a : F}, a ≠ 0 → w a ≠ 1 → 0 < Real.log (w a) / Real.log (v a)
```

AbsoluteValue.isEquiv_iff_lt_one_iff

```
∀ {R S : Type*} [Field R] [Semifield S] [LinearOrder S] {v w : AbsoluteValue R S} [IsStrictOrderedRing S], v.IsEquiv w ↔ ∀ (x : R),
v x < 1 ↔ w x < 1
```

Analysis · Asymptotics · SpecificAsymptotics

pow_div_pow_eventuallyEq_atBot

```
∀ {□ : Type u_1} [Field □ ] [LinearOrder □ ] [IsStrictOrderedRing □ ] {p q : ℕ}, (fun x => x ^ p / x ^ q) =ᶠ[Filter.atBot] fun x =>
x ^ (↑p - ↑q)
```

pow_div_pow_eventuallyEq_atTop

```
∀ {□ : Type u_1} [Field □ ] [LinearOrder □ ] [IsStrictOrderedRing □ ] {p q : ℕ}, (fun x => x ^ p / x ^ q) =ᶠ[Filter.atTop] fun x =>
x ^ (↑p - ↑q)
```

Analysis · Convex · Basic

Convex.mem_smul_of_zero_mem

```
∀ {□ : Type u_1} {E : Type*} [Field □ ] [LinearOrder □ ] [IsStrictOrderedRing □ ] [AddCommGroup E] [Module □ E] {s : Set E}, Convex
□ s → ∀ {x : E}, 0 ∈ s → x ∈ s → ∀ {t : □ }, 1 ≤ t → x ∈ t • s
```

Analysis · Convex · Between

wbtw_iff_of_le ‡

```
∀ {R : Type*} [Field R] [LinearOrder R] [IsStrictOrderedRing R] {x y z : R}, x ≤ z → (Wbtw R x y z ↔ x ≤ y ∧ y ≤ z)
```

CommRing → Ring (3, magnitude 1)

Analysis · Normed · Unbundled · SmoothingSeminorm

smoothingSeminormSeq_exists_pnat ?

signature not resolved

smoothingSeminormSeq_tendsto_aux ?

signature not resolved

zero_mem_lowerBounds_smoothingSeminormSeq_range

```
∀ {R : Type*} [CommRing R] (μ : RingSeminorm R) (x : R), 0 ∈ lowerBounds (Set.range fun n => μ (x ^ ↑n) ^ (1 / ↑↑n))
```

PartialOrder → Preorder (3, magnitude 1)

Analysis · Convex · Basic

convex_Iic

```
∀ {□ : Type u_1} {β : Type u_2} [Semiring □ ] [PartialOrder □ ] [AddCommMonoid β] [PartialOrder β] [IsOrderedAddMonoid β] [Module □
β] [PosSMulMono □ β] (r : β), Convex □ (Set.Iic r)
```

Analysis · Convex · Star

Set.OrdConnected.starConvex

```
∀ {□ : Type u_1} {E : Type*} [Semiring □ ] [PartialOrder □ ] [AddCommMonoid E] [PartialOrder E] [IsOrderedAddMonoid E] [Module □ E]
```

`[PosSMulMono □ E] {x : E} {s : Set E}, s.OrdConnected → x ∈ s → (∀ y ∈ s, x ≤ y ∨ y ≤ x) → StarConvex □ x s`

Analysis · Normed · Order · Lattice

`isClosed_le_of_isClosed_nonneg`

`∀ {G : Type*} [AddCommGroup G] [PartialOrder G] [IsOrderedAddMonoid G] [TopologicalSpace G] [ContinuousSub G], IsClosed {x | 0 ≤ x} → IsClosed {p | p.1 ≤ p.2}`

MetricSpace → PseudoMetricSpace (2, magnitude 1)

Analysis · Normed · Affine · AddTorsorBases

`isOpenMap_barycentric_coord` ‡

`∀ {ι : Type u_1} {□ : Type u_2} {E P : Type*} [NontriviallyNormedField □ ] [CompleteSpace □ ] [NormedAddCommGroup E] [NormedSpace □ E] [MetricSpace P] [NormedAddTorsor E P] [Nontrivial ι] (b : AffineBasis ι □ P) (i : ι), IsOpenMap ⇑(b.coord i)`

Analysis · Normed · Affine · AsymptoticCone

`AffineSpace.asymptoticNhds_le_cobounded` ‡

`∀ {V P : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V] [MetricSpace P] [NormedAddTorsor V P] {v : V}, v ≠ 0 → AffineSpace.asymptoticNhds ℝ P v ≤ Bornology.cobounded P`

AddCommGroup → AddGroup (1, magnitude 1)

Analysis · Normed · Group · Seminorm

`NonarchAddGroupSeminorm.add_bddBelow_range_add`

`∀ {E : Type*} [AddCommGroup E] {p q : NonarchAddGroupSeminorm E} {x : E}, BddBelow (Set.range fun y => p y + q (x - y))`

CommGroup → Group (1, magnitude 1)

Analysis · Normed · Group · Seminorm

`GroupSeminorm.mul_bddBelow_range_add`

`∀ {E : Type*} [CommGroup E] {p q : GroupSeminorm E} {x : E}, BddBelow (Set.range fun y => p y + q (x / y))`

IsDomain → Nontrivial (1, unranked)

Analysis · SpecificLimits · Normed

`tendsto_smul_comp_nat_floor_of_tendsto_nsmul` ‡

`∀ {R K : Type*} [NormedRing K] [IsDomain K] [NormedAddCommGroup R] [Module K R] [Module.IsTorsionFree K R] [NormSMulClass K R] [NormSMulClass ℤ K] [LinearOrder K] [IsStrictOrderedRing K] [FloorSemiring K] [HasSolidNorm K] {g : ℕ → R} {t : R}, Filter.Tendsto (fun n => n • g n) Filter.atTop (nhds t) → Filter.Tendsto (fun x => x • g ⌊x⌋₊) Filter.atTop (nhds t)`

SeminormedCommGroup → CommGroup (1, unranked)

Analysis · Normed · Group · Continuity

`controlled_prod_of_mem_closure_range`

`∀ {E F : Type*} [SeminormedCommGroup E] [SeminormedCommGroup F] {j : E →* F} {b : F}, b ∈ closure ↑j.range → ∀ {f : ℕ → ℝ}, (∀ (n : ℕ), 0 < f n) → ∃ a, Filter.Tendsto (fun n => ∏ i ∈ Finset.range (n + 1), j (a i)) Filter.atTop (nhds b) ∧ □ (j (a 0))⁻¹ * b□ < f 0 ∧ ∀ (n : ℕ), 0 < n → □ j (a n)□ < f n`

StarRing → StarMul (1, unranked)

Analysis · CStarAlgebra · Unitization

`Unitization.norm_splitMul_snd_sq`

`∀ (□ : Type u_1) {E : Type*} [DenselyNormedField □] [NonUnitalNormedRing E] [StarRing E] [CStarRing E] [NormedSpace □ E] [IsScalarTower □ E E] [SMulCommClass □ E E] [StarRing □] [StarModule □ E] (x : Unitization □ E), □ ((Unitization.splitMul □ E) x).2□ ^ 2 ≤ □ ((Unitization.splitMul □ E) (star x * x)).2□`

CategoryTheory

1 survivor in 1 family; 0 give up subtraction or division (†).

stated over	holds over	n	magnitude
`IsFiltered`	`IsFilteredOrEmpty`	1	-1

IsFiltered → IsFilteredOrEmpty (1, unranked)

CategoryTheory · Filtered · Basic

`CategoryTheory.IsFiltered.isDirectedOrder`

`∀ (α : Type u_1) [Preorder α] [CategoryTheory.IsFiltered α], IsDirectedOrder α`

Combinatorics

12 survivors in 8 families; 2 give up subtraction or division (†).

stated over	holds over	n	magnitude
`Group` †	`Monoid`	2	2
`LinearOrder`	`PartialOrder`	2	2
`SemilatticeSup`	`PartialOrder`	2	1
`CommGroup`	`Group`	1	1
`CommMonoid`	`Monoid`	1	1
`CommRing`	`Ring`	1	1
`CommSemiring`	`Semiring`	1	1
`DistribLattice`	`Lattice`	2	-1

Group → Monoid (2, magnitude 2) †

Combinatorics · Additive · VerySmallDoubling

`Finset.nonempty_of_doubling` ?

signature not resolved

`Finset.weaken_doubling` ?

signature not resolved

LinearOrder → PartialOrder (2, magnitude 2)

Combinatorics · Additive · SubsetSum

`Finset.nonneg_of_mem_subsetSum`

`∀ {M : Type*} [DecidableEq M] [AddCommMonoid M] {A : Finset M} [LinearOrder M] [IsOrderedCancelAddMonoid M], (∀ x ∈ A, 0 ≤ x) → ∀ x ∈ A.subsetSum, 0 ≤ x`

Combinatorics · SimpleGraph · Extremal · Basic

`SimpleGraph.extremalNumber_le_iff_of_nonneg`

`∀ {V W : Type*} [Fintype V] {R : Type*} [Semiring R] [LinearOrder R] [FloorSemiring R] (H : SimpleGraph W) {m : R}, 0 ≤ m → (↑(SimpleGraph.extremalNumber (Fintype.card V) H) ≤ m ↔ ∀ ▯ G : SimpleGraph V▯ [DecidableRel G.Adj], H.Free G → ↑G.edgeFinset.card ≤ m)`

SemilatticeSup → PartialOrder (2, magnitude 1)

Combinatorics · SetFamily · AhlswedeZhang

`Finset.lower_aux` ?

signature not resolved

`Finset.sup_aux` ?

signature not resolved

CommGroup → Group (1, magnitude 1)

Combinatorics · Additive · PluenneckeRuzsa

`Finset.mul_aux` ?

signature not resolved

CommMonoid → Monoid (1, magnitude 1)

Combinatorics · Additive · AP · Three · Defs

`ThreeGPFree.image'`

`∀ {F : Type*} {α : Type u_2} {β : Type u_3} [CommMonoid α] [CommMonoid β] {s : Set α} [FunLike F α β] [MulHomClass F α β] (f : F), Set.InjOn (⇑f) (s * s) → ThreeGPFree s → ThreeGPFree (⇑f '' s)`

CommRing → Ring (1, magnitude 1)

Combinatorics · Enumerative · Pentagonal · PowerSeries

`Pentagonal.tendsto_order_neg_X_pow` ?

signature not resolved

CommSemiring → Semiring (1, magnitude 1)

Combinatorics · Enumerative · Partition · GenFun

`Nat.Partition.tendsto_order_genFun_term_atTop_nhds_top` ‡

`∀ {R : Type*} [CommSemiring R] (f : ℕ → ℕ → R) (i : ℕ), Filter.Tendsto (fun j => (f (i + 1) (j + 1) • PowerSeries.X ^ ((i + 1) * (j + 1))).order) Filter.atTop (nhds ▯ )`

DistribLattice → Lattice (2, unranked)

Combinatorics · SetFamily · AhlswedeZhang

`Finset.infs_aux` ?

signature not resolved

`Finset.sups_aux` ?

signature not resolved

Computability

2 survivors in 1 family; 0 give up subtraction or division (†).

stated over	holds over	n	magnitude
`Primcodable`	`Encodable`	2	-1

Primcodable → Encodable (2, unranked)

Computability · Primrec · Basic

`Primrec.of_equiv_iff`

```
∀ {α : Type u_1} {σ : Type u_2} [Primcodable α] [Primcodable σ] {β : Type u_3} (e : β ≃ α) {f : σ → β}, (Primrec fun a => e (f a)) ↔ Primrec f
```

`Primrec.of_equiv_symm_iff`

```
∀ {α : Type u_1} {σ : Type u_2} [Primcodable α] [Primcodable σ] {β : Type u_3} (e : β ≃ α) {f : σ → α}, (Primrec fun a => e.symm (f a)) ↔ Primrec f
```

Data

17 survivors in 8 families; 0 give up subtraction or division (†).

stated over	holds over	n	magnitude
`Monoid`	`Semigroup`	3	2
`Fintype`	`Finite`	1	2
`LinearOrder`	`PartialOrder`	1	2
`PartialOrder`	`Preorder`	4	1
`AddCommMonoid`	`AddMonoid`	2	1
`Field`	`DivisionRing`	2	1
`NonUnitalSemiring`	`NonUnitalNonAssocSemiring`	2	1
`SemilatticeInf`	`PartialOrder`	2	1

Monoid → Semigroup (3, magnitude 2)

Data · Finset · NoncommProd

`Multiset.map_noncommProd_aux`

```
∀ {F : Type*} {α : Type u_2} {β : Type u_3} [Monoid α] [Monoid β] [FunLike F α β] [MulHomClass F α β] (s : Multiset α), {x | x ∈ s}.Pairwise Commute → ∀ (f : F), {x | x ∈ Multiset.map (⇑f) s}.Pairwise Commute
```

`Finset.noncommProd_mul_distrib_aux` ‡

```
∀ {α : Type u_1} {β : Type u_2} [Monoid β] {s : Finset α} {f g : α → β}, (↑s).Pairwise (Function.onFun Commute f) → (↑s).Pairwise (Function.onFun Commute g) → ((↑s).Pairwise fun x y => Commute (g x) (f y)) → (↑s).Pairwise fun x y => Commute ((f * g) x) ((f * g) y)
```

Data · Nat · DvdSequence

`smul_dvd_smul`

```
∀ {α : Type u_1} {β : Type u_2} [Monoid α] [Monoid β] [SMul α β] [IsScalarTower α β β] [IsScalarTower α α β] [SMulCommClass α β β] {a b : α} {c d : β}, a ∣ b → c ∣ d → a • c ∣ b • d
```

Fintype → Finite (1, magnitude 2)

Data · Fin · Pigeonhole

`Fin.card_range_le` ‡

```
∀ {m : ℕ} {α : Type u_1} [Fintype α] [DecidableEq α] (f : Fin m → α), Fintype.card ↑(Set.range f) ≤ m
```

LinearOrder → PartialOrder (1, magnitude 2)

Data · Real · Embedding

`num_le_nat_mul_den`

```
∀ {M : Type*} [AddCommGroup M] [LinearOrder M] [IsOrderedAddMonoid M] [One M] [ZeroLEOneClass M] [NeZero 1] {num : ℤ} {den : ℕ} {x : M}, num • 1 ≤ den • x → ∀ {n : ℤ}, x ≤ n • 1 → num ≤ n * ↑den
```

PartialOrder → Preorder (4, magnitude 1)

Data · Finset · MulAntidiagonal

`Set.IsPWO.mul`

```
∀ {α : Type u_1} {s t : Set α} [CommMonoid α] [PartialOrder α] [IsOrderedCancelMonoid α], s.IsPWO → t.IsPWO → (s * t).IsPWO
```

Data · Finsupp · Order

`Finsupp.le_iff'`

```
∀ {ι : Type u_1} {α : Type u_2} [AddCommMonoid α] [PartialOrder α] [CanonicallyOrderedAdd α] (f g : ι →₀ α) {s : Finset ι}, f.support ⊆ s → (f ≤ g ↔ ∀ i ∈ s, f i ≤ g i)
```

Data · Nat · Cast · Order · Basic

`Nat.mono_cast`

```
∀ {α : Type u_1} [AddMonoidWithOne α] [PartialOrder α] [AddLeftMono α] [ZeroLEOneClass α], Monotone Nat.cast
```

Data · SetLike · Basic

`SetLike.lt_iff_le_and_exists`

```
∀ {A B : Type*} [SetLike A B] [PartialOrder A] [IsConcreteLE A B] {p q : A}, p < q ↔ p ≤ q ∧ ∃ x ∈ q, x ∉ p
```

AddCommMonoid → AddMonoid (2, magnitude 1)

Data · Finsupp · Order

`Finsupp.le_iff'`

```
∀ {ι : Type u_1} {α : Type u_2} [AddCommMonoid α] [PartialOrder α] [CanonicallyOrderedAdd α] (f g : ι →₀ α) {s : Finset ι}, f.support ⊆ s → (f ≤ g ↔ ∀ i ∈ s, f i ≤ g i)
```

`Finsupp.support_monotone`

```
∀ {ι : Type u_1} {α : Type u_2} [AddCommMonoid α] [PartialOrder α] [CanonicallyOrderedAdd α], Monotone Finsupp.support
```

Field → DivisionRing (2, magnitude 1)

Data · NNRat · Floor

`Mathlib.Meta.NormNum.IsRat.natCeil` ‡

```
∀ {R : Type*} [Field R] [LinearOrder R] [IsStrictOrderedRing R] [FloorSemiring R] (r : R) (n d : ℕ), Mathlib.Meta.NormNum.IsRat r (Int.negOfNat n) d → Mathlib.Meta.NormNum.IsNat ⌈r⌉₊ 0
```

Data · Rat · Cast · Order

`Rat.cast_pos_of_pos`

```
∀ {q : ℚ} {K : Type*} [Field K] [LinearOrder K] [IsStrictOrderedRing K], 0 < q → 0 < ↑q
```

NonUnitalSemiring → NonUnitalNonAssocSemiring (2, magnitude 1)

Data · Matrix · Mul

`Matrix.mul_left_injective_iff_vecMul_injective`

```
∀ {l m n : Type*} {α : Type u_1} [NonUnitalSemiring α] [Nonempty l] [Fintype m] {A : Matrix m n α}, (Function.Injective fun B => B * A) ↔ Function.Injective fun v => Matrix.vecMul v A
```

`Matrix.mul_right_injective_iff_mulVec_injective`

```
∀ {l m n : Type*} {α : Type u_1} [NonUnitalSemiring α] [Fintype m] [Nonempty n] {A : Matrix l m α}, (Function.Injective fun B => A * B) ↔ Function.Injective A.mulVec
```

SemilatticeInf → PartialOrder (2, magnitude 1)

Data · Finset · Pairwise

`Set.PairwiseDisjoint.attach`

```
∀ {α : Type u_1} {ι : Type u_2} [SemilatticeInf α] [OrderBot α] {s : Finset ι} {f : ι → α}, (↑s).PairwiseDisjoint f → (↑s.attach).PairwiseDisjoint (f ∘ Subtype.val)
```

`Set.PairwiseDisjoint.image_finset_of_le`

```
∀ {α : Type u_1} {ι : Type u_2} [SemilatticeInf α] [OrderBot α] [DecidableEq ι] {s : Finset ι} {f : ι → α}, (↑s).PairwiseDisjoint f → ∀ {g : ι → ι}, (∀ (a : ι), f (g a) ≤ f a) → (↑(Finset.image g s)).PairwiseDisjoint f
```

Dynamics

2 survivors in 2 families; 0 give up subtraction or division (†).

stated over	holds over	n	magnitude
`NormedAddCommGroup`	`SeminormedAddCommGroup`	1	1
`PartialOrder`	`Preorder`	1	1

NormedAddCommGroup → SeminormedAddCommGroup (1, magnitude 1)

Dynamics · BirkhoffSum · NormedSpace

`dist_birkhoffSum_birkhoffSum_le`

```
∀ {α : Type u_1} {E : Type*} [NormedAddCommGroup E] (f : α → α) (g : α → E) (n : ℕ) (x y : α), dist (birkhoffSum f g n x) (birkhoffSum f g n y) ≤ ∑ k ∈ Finset.range n, dist (g (f^[k] x)) (g (f^[k] y))
```

PartialOrder → Preorder (1, magnitude 1)

Dynamics · Flow

`IsForwardInvariant.isInvariant`

```
∀ {τ : Type u_1} {α : Type u_2} [AddMonoid τ] [PartialOrder τ] [CanonicallyOrderedAdd τ] {ϕ : τ → α → α} {s : Set α}, IsForwardInvariant ϕ s → IsInvariant ϕ s
```

FieldTheory

5 survivors in 4 families; 1 gives up subtraction or division (†).

stated over	holds over	n	magnitude
`Field`	`CommRing`	1	3
`AddCommGroup` †	`AddCommMonoid`	1	2
`Field`	`DivisionRing`	2	1
`LinearOrderedCommGroupWithZero`	`LinearOrderedCommMonoidWithZero`	1	-1

Field → CommRing (1, magnitude 3)

FieldTheory · IsSepClosed

`IsSepClosed.splits_codomain`

`∀ {k : Type*} [Field k] {K : Type*} [Field K] [IsSepClosed K] {f : k →+* K} (p : Polynomial k), p.Separable → (Polynomial.map f p).Splits`

AddCommGroup → AddCommMonoid (1, magnitude 2) †

FieldTheory · Finiteness

`IsNoetherian.finite_basis_index` ‡

`∀ {K V : Type*} [DivisionRing K] [AddCommGroup V] [Module K V] {ι : Type u_3} {s : Set ι} [IsNoetherian K V] (b : Module.Basis (↑s) K V), s.Finite`

Field → DivisionRing (2, magnitude 1)

FieldTheory · Finite · GaloisField

`GaloisField._root_.Polynomial.splits_X_pow_nat_card_sub_X` ‡

`∀ {K : Type*} [Field K], (Polynomial.X ^ Nat.card K - Polynomial.X).Splits`

FieldTheory · KummerPolynomial

`ne_zero_of_irreducible_X_pow_sub_C'`

`∀ {K : Type*} [Field K] {n : ℕ}, n ≠ 1 → ∀ {a : K}, Irreducible (Polynomial.X ^ n - Polynomial.C a) → a ≠ 0`

LinearOrderedCommGroupWithZero → LinearOrderedCommMonoidWithZero (1, unranked)

FieldTheory · RatFunc · AsPolynomial

`Polynomial.valuation_le_one_of_valuation_X_le_one` ‡

`∀ (K : Type u_1) [Field K] {Γ : Type u_2} [LinearOrderedCommGroupWithZero Γ] {v : Valuation (RatFunc K) Γ} [hv : Valuation.IsTrivialOn K v], v RatFunc.X ≤ 1 → ∀ (p : Polynomial K), v ↑p ≤ 1`

Geometry

11 survivors in 5 families; 0 give up subtraction or division (†).

stated over	holds over	n	magnitude
`DivisionRing`	`Ring`	2	3
`LinearOrder`	`PartialOrder`	3	2
`Field`	`DivisionRing`	2	1
`PartialOrder`	`Preorder`	2	1
`Semifield`	`CommSemiring`	2	-1

DivisionRing → Ring (2, magnitude 3)

Geometry · Convex · Cone · Face · Basic

`PointedCone.IsFaceOf.lineal_le`

```
∀ {R M : Type*} [DivisionRing R] [LinearOrder R] [IsOrderedRing R] [AddCommGroup M] [Module R M] {C F : PointedCone R M}, F.IsFaceOf C → ↑C.lineal ≤ F
```

`PointedCone.IsFaceOf.prod`

```
∀ {R M N : Type*} [DivisionRing R] [LinearOrder R] [IsOrderedRing R] [AddCommGroup M] [Module R M] [AddCommGroup N] [Module R N] {C₁ F₁ : PointedCone R M} {C₂ F₂ : PointedCone R N}, F₁.IsFaceOf C₁ → F₂.IsFaceOf C₂ → PointedCone.IsFaceOf (Submodule.prod F₁ F₂) (Submodule.prod C₁ C₂)
```

LinearOrder → PartialOrder (3, magnitude 2)

Geometry · Convex · Cone · Face · Basic

`PointedCone.IsFaceOf.prod`

```
∀ {R M N : Type*} [DivisionRing R] [LinearOrder R] [IsOrderedRing R] [AddCommGroup M] [Module R M] [AddCommGroup N] [Module R N] {C₁ F₁ : PointedCone R M} {C₂ F₂ : PointedCone R N}, F₁.IsFaceOf C₁ → F₂.IsFaceOf C₂ → PointedCone.IsFaceOf (Submodule.prod F₁ F₂) (Submodule.prod C₁ C₂)
```

Geometry · Convex · ConvexSpace · Defs

`Convexity.StdSimplex.restrict_ne_zero_aux` ?

signature not resolved

`Convexity.StdSimplex.restrict_nonneg_aux` ?

signature not resolved

Field → DivisionRing (2, magnitude 1)

Geometry · Convex · Cone · Basic

`ConvexCone.smul_mem_iff`

```
∀ {□ : Type u_1} {M : Type*} [Field □] [LinearOrder □] [IsStrictOrderedRing □] [AddCommMonoid M] [MulAction □ M] (C : ConvexCone □ M) {c : □}, 0 < c → ∀ {x : M}, c • x ∈ C ↔ x ∈ C
```

Geometry · Convex · Cone · Pointed

`PointedCone.smul_mem_iff`

```
∀ {□ : Type u_1} {M : Type*} [Field □] [LinearOrder □] [IsStrictOrderedRing □] [AddCommMonoid M] [Module □ M] (C : PointedCone □ M) {c : □}, 0 < c → ∀ {x : M}, c • x ∈ C ↔ x ∈ C
```

PartialOrder → Preorder (2, magnitude 1)

Geometry · Convex · ConvexSpace · Defs

`Convexity.StdSimplex.weights_ne_zero`

```
∀ {R : Type*} [PartialOrder R] [Semiring R] {M : Type*} [Nontrivial R] (w : Convexity.StdSimplex R M), w.weights ≠ 0
```

`Convexity.StdSimplex.weights_nonneg`

```
∀ {R : Type*} [PartialOrder R] [Semiring R] {M : Type*} {w : Convexity.StdSimplex R M} (i : M), 0 ≤ w.weights i
```

Semifield → CommSemiring (2, unranked)

Geometry · Convex · ConvexSpace · Defs

`Convexity.StdSimplex.restrict_ne_zero_aux` ?

signature not resolved

`Convexity.StdSimplex.restrict_nonneg_aux` ?

signature not resolved

GroupTheory

14 survivors in 6 families; 8 give up subtraction or division (†).

stated over	holds over	n	magnitude
`Group` †	`Monoid`	8	2
`Monoid`	`MulOneClass`	2	2
`Group`	`DivInvMonoid`	1	2
`CommGroup`	`Group`	1	1
`MulOneClass`	`MulOne`	1	1
`NonAssocRing`	`NonAssocSemiring`	1	-1

Group → Monoid (8, magnitude 2) †

GroupTheory · GroupAction · Blocks

`MulAction.IsBlock.image`

```
∀ {G : Type*} [Group G] {X : Type*} [MulAction G X] {B : Set X} {H Y : Type*} [SMul H Y] {φ : G → H} (j : X →ₑ[φ] Y), Function.Surjective φ → Function.Injective ⇑j → MulAction.IsBlock G B → MulAction.IsBlock H (⇑j '' B)
```

`MulAction.isBlock_subgroup`

```
∀ {G : Type*} [Group G] {S H : Type*} [Group H] [SetLike S H] [SubgroupClass S H] {s : S} [MulAction G H] [IsScalarTower G H H], MulAction.IsBlock G ↑s
```

`MulAction.isBlock_subgroup'`

```
∀ {G : Type*} [Group G] {S H : Type*} [Group H] [SetLike S H] [SubgroupClass S H] {s : S} [MulAction G H] [IsScalarTower G Hᵐᵒᵖ H], MulAction.IsBlock G ↑s
```

`MulAction.isBlock_subtypeVal`

```
∀ {G : Type*} [Group G] {X : Type*} [MulAction G X] {C : SubMulAction G X} {B : Set ↥C}, MulAction.IsBlock G (Subtype.val '' B) ↔ MulAction.IsBlock G B
```

GroupTheory · PGroup

`IsPGroup.ker_isPGroup_of_injective`

```
∀ {p : ℕ} {G : Type*} [Group G] {K : Type*} [Group K] {ϕ : K →* G}, Function.Injective ⇑ϕ → IsPGroup p ↥ϕ.ker
```

GroupTheory · Torsion

`ExponentExists.isMulTorsion`

```
∀ {G : Type*} [Group G], Monoid.ExponentExists G → IsMulTorsion G
```

`IsMulTorsion.exponentExists`

```
∀ {G : Type*} [Group G], IsMulTorsion G → (Set.range fun g => orderOf g).Finite → Monoid.ExponentExists G
```

`IsMulTorsion.of_surjective`

```
∀ {G H : Type*} [Group G] [Group H] {f : G →* H}, Function.Surjective ⇑f → IsMulTorsion G → IsMulTorsion H
```

Monoid → MulOneClass (2, magnitude 2)

GroupTheory · Coset · Basic

`mem_own_leftCoset`

```
∀ {α : Type u_1} [Monoid α] (s : Submonoid α) (a : α), a ∈ a • ↑s
```

`mem_own_rightCoset`

```
∀ {α : Type u_1} [Monoid α] (s : Submonoid α) (a : α), a ∈ MulOpposite.op a • ↑s
```

Group → DivInvMonoid (1, magnitude 2)

GroupTheory · Torsion

`IsMulTorsion.of_surjective`

```
∀ {G H : Type*} [Group G] [Group H] {f : G →* H}, Function.Surjective ⇑f → IsMulTorsion G → IsMulTorsion H
```

CommGroup → Group (1, magnitude 1)

GroupTheory · OrderOfElement

`IsMulTorsionFree.orderOf_le_one`

```
∀ {G : Type*} [CommGroup G] [IsMulTorsionFree G] (g : G), orderOf g ≤ 1
```

MulOneClass → MulOne (1, magnitude 1)

GroupTheory · Congruence · BigOperators

`Con.list_prod`

```
∀ {ι : Type u_1} {M : Type*} [MulOneClass M] (c : Con M) {l : List ι} {f g : ι → M}, (∀ x ∈ l, c (f x) (g x)) → c (List.map f l).prod (List.map g l).prod
```

NonAssocRing → NonAssocSemiring (1, unranked)

GroupTheory · OrderOfElement

```
CharP.addOrderOf_one
  ∀ (R : Type u_1) [NonAssocRing R], CharP R (addOrderOf 1)
```

LinearAlgebra

38 survivors in 10 families; 19 give up subtraction or division (†).

stated over	holds over	n	magnitude
`IsDomain`	`NoZeroDivisors`	7	3
`Field`	`CommRing`	3	3
`AddCommGroup` †	`AddCommMonoid`	13	2
`Ring` †	`Semiring`	6	2
`AddCommSemigroup`	`AddCommMagma`	2	2
`CommMonoid`	`CommSemigroup`	1	2
`LinearOrder`	`PartialOrder`	1	2
`NonUnitalSemiring`	`NonUnitalNonAssocSemiring`	3	1
`Field`	`DivisionRing`	1	1
`IsCancelAdd`	`IsLeftCancelAdd`	1	-1

IsDomain → NoZeroDivisors (7, magnitude 3)

LinearAlgebra · BilinearMap

`LinearMap.lsmul_injective`

`∀ {R M : Type*} [CommRing R] [IsDomain R] [AddCommGroup M] [Module R M] [Module.IsTorsionFree R M] {x : R}, x ≠ 0 → Function.Injective ⇑((LinearMap.lsmul R M) x)`

LinearAlgebra · DFinsupp

`iSupIndep.linearIndependent` ‡

`∀ {R N : Type*} [Ring R] [AddCommGroup N] [Module R N] [IsDomain R] [Module.IsTorsionFree R N] {ι : Type u_3} (p : ι → Submodule R N), iSupIndep p → ∀ {v : ι → N}, (∀ (i : ι), v i ∈ p i) → (∀ (i : ι), v i ≠ 0) → LinearIndependent R v`

LinearAlgebra · Eigenspace · Basic

`Module.End.HasUnifEigenvalue.isNilpotent_of_isNilpotent` ‡

`∀ {R M : Type*} [CommRing R] [AddCommGroup M] [Module R M] [IsDomain R] [Module.IsTorsionFree R M] {f : Module.End R M}, IsNilpotent f → ∀ {μ : R}, f.HasUnifEigenvalue μ 1 → IsNilpotent μ`

LinearAlgebra · Eigenspace · Minpoly

`Module.End.isRoot_of_hasEigenvalue`

`∀ {R M : Type*} [CommRing R] [AddCommGroup M] [Module R M] [IsDomain R] [Module.IsTorsionFree R M] {f : Module.End R M} {μ : R}, f.HasEigenvalue μ → (minpoly R f).IsRoot μ`

LinearAlgebra · LinearIndependent · Defs

`LinearIndepOn.singleton`

`∀ {ι : Type u_1} {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] {v : ι → M} {i : ι} [IsDomain R] [Module.IsTorsionFree R M], v i ≠ 0 → LinearIndepOn R v {i}`

`LinearIndependent.of_subsingleton`

`∀ {ι : Type u_1} {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] {v : ι → M} [IsDomain R] [Module.IsTorsionFree R M] [Subsingleton ι] (i : ι), v i ≠ 0 → LinearIndependent R v`

LinearAlgebra · LinearIndependent · Lemmas

`linearIndependent_algHom_toLinearMap'`

`∀ (K : Type u_1) (M : Type u_2) (L : Type u_3) [CommRing K] [IsDomain K] [Semiring M] [Algebra K M] [CommRing L] [IsDomain L] [Algebra K L] [Module.IsTorsionFree K L], LinearIndependent K AlgHom.toLinearMap`

Field → CommRing (3, magnitude 3)

LinearAlgebra · LinearIndependent · BaseChange

`LinearIndependent.linearIndependent_algebraMap_comp_aux` ?

signature not resolved

LinearAlgebra · QuadraticForm · Signature

`QuadraticForm.negSemidef_spanSubset` ?

signature not resolved

`QuadraticForm.posDef_spanSubset` ?

signature not resolved

AddCommGroup → AddCommMonoid (13, magnitude 2) †

LinearAlgebra · Basis · VectorSpace

`nonzero_span_atom`

`∀ {K V : Type*} [DivisionRing K] [AddCommGroup V] [Module K V] (v : V), v ≠ 0 → IsAtom (K ∙ v)`

LinearAlgebra · BilinearMap

`LinearMap.lsmul_injective`

```
∀ {R M : Type*} [CommRing R] [IsDomain R] [AddCommGroup M] [Module R M] [Module.IsTorsionFree R M] {x : R}, x ≠ 0 → Function.Injective ⇑((LinearMap.lsmul R M) x)
```

LinearAlgebra · Determinant

`LinearEquiv.isUnit_det`

```
∀ {R : Type*} [CommRing R] {M : Type*} [AddCommGroup M] [Module R M] {M' : Type*} [AddCommGroup M'] [Module R M'] {ι : Type u_4} [DecidableEq ι] [Fintype ι] (f : M ≃ₗ[R] M') (v : Module.Basis ι R M) (v' : Module.Basis ι R M'), IsUnit ((LinearMap.toMatrix v v') ↑f).det
```

LinearAlgebra · Dimension · Finite

`exists_mem_ne_zero_of_rank_pos`

```
∀ {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] [Nontrivial R] {s : Submodule R M}, 0 < Module.rank R ↥s → ∃ b ∈ s, b ≠ 0
```

LinearAlgebra · LinearIndependent · Lemmas

`mem_span_insert_exchange`

```
∀ {K V : Type*} [DivisionRing K] [AddCommGroup V] [Module K V] {s : Set V} {x y : V}, x ∈ Submodule.span K (insert y s) → x ∉ Submodule.span K s → y ∈ Submodule.span K (insert x s)
```

LinearAlgebra · Matrix · PosDef

`Matrix.PosDef._root_.LinearMap.BilinForm.posDef_toQuadraticMap_iff_matrix`

```
∀ {n : Type*} [Fintype n] {R M : Type*} [CommRing R] [PartialOrder R] [StarRing R] [TrivialStar R] [AddCommGroup M] [Module R M] [DecidableEq n] (b : Module.Basis n R M) (B : LinearMap.BilinForm R M), B.IsSymm → ((LinearMap.BilinMap.toQuadraticMap B).PosDef ↔ ((LinearMap.BilinForm.toMatrix b) B).PosDef)
```

LinearAlgebra · Ray

`sameRay_of_mem_orbit`

```
∀ {R : Type*} [CommRing R] [LinearOrder R] [IsStrictOrderedRing R] {M : Type*} [AddCommGroup M] [Module R M] {v₁ v₂ : M}, v₁ ∈ MulAction.orbit (↥(Units.posSubgroup R)) v₂ → SameRay R v₁ v₂
```

`sameRay_smul_smul_of_mul_nonneg`

```
∀ {R : Type*} [CommRing R] [LinearOrder R] [IsStrictOrderedRing R] {M : Type*} [AddCommGroup M] [Module R M] {v : M} {c₁ c₂ : R}, 0 ≤ c₁ * c₂ → SameRay R (c₁ • v) (c₂ • v)
```

LinearAlgebra · SesquilinearForm · Basic

`LinearMap.IsAdjointPair.smul`

```
∀ {R M M₁ M₂ : Type*} [CommRing R] [AddCommGroup M] [Module R M] [AddCommGroup M₁] [Module R M₁] [AddCommGroup M₂] [Module R M₂] {B : M →ₗ[R] M →ₗ[R] M₂} {B' : M₁ →ₗ[R] M₁ →ₗ[R] M₂} {f : M → M₁} {g : M₁ → M} (c : R), B.IsAdjointPair B' f g → B.IsAdjointPair B' (c • f) (c • g)
```

`LinearMap.IsAdjointPair.smul`

```
∀ {R M M₁ M₂ : Type*} [CommRing R] [AddCommGroup M] [Module R M] [AddCommGroup M₁] [Module R M₁] [AddCommGroup M₂] [Module R M₂] {B : M →ₗ[R] M →ₗ[R] M₂} {B' : M₁ →ₗ[R] M₁ →ₗ[R] M₂} {f : M → M₁} {g : M₁ → M} (c : R), B.IsAdjointPair B' f g → B.IsAdjointPair B' (c • f) (c • g)
```

`LinearMap.disjoint_ker_of_nondegenerate_restrict`

```
∀ {R M M₁ : Type*} [CommRing R] [AddCommGroup M] [Module R M] [AddCommGroup M₁] [Module R M₁] {B : M →ₗ[R] M →ₗ[R] M₁} {W : Submodule R M}, (B.domRestrict₁₂ W W).Nondegenerate → Disjoint W B.ker
```

`LinearMap.isPairSelfAdjoint_equiv`

```
∀ {R M M₁ M₂ : Type*} [CommRing R] [AddCommGroup M] [Module R M] [AddCommGroup M₁] [Module R M₁] [AddCommGroup M₂] [Module R M₂] {B F : M →ₗ[R] M →ₗ[R] M₂} (e : M₁ ≃ₗ[R] M) (f : Module.End R M), B.IsPairSelfAdjoint F ⇑f ↔ (B.compl₁₂ ↑e ↑e).IsPairSelfAdjoint (F.compl₁₂ ↑e ↑e) ⇑(e.symm.conj f)
```

LinearAlgebra · Trace

`LinearMap.isNilpotent_trace_of_isNilpotent`

```
∀ {R : Type*} [CommRing R] {M : Type*} [AddCommGroup M] [Module R M] {f : M →ₗ[R] M}, IsNilpotent f → IsNilpotent ((LinearMap.trace R M) f)
```

Ring → Semiring (6, magnitude 2) †

LinearAlgebra · Dimension · Finite

`exists_mem_ne_zero_of_rank_pos`

```
∀ {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] [Nontrivial R] {s : Submodule R M}, 0 < Module.rank R ↥s → ∃ b ∈ s, b ≠ 0
```

LinearAlgebra · LinearDisjoint

`Submodule.LinearDisjoint.linearIndependent_mul_of_flat_left`

```
∀ {R S : Type*} [CommRing R] [Ring S] [Algebra R S] {M N : Submodule R S}, M.LinearDisjoint N → ∀ [Module.Flat R ↥M] {κ : Type u_3} {ι : Type u_4} {m : κ → ↥M} {n : ι → ↥N}, LinearIndependent R m → LinearIndependent R n → LinearIndependent R fun i => ↑(m i.1) * ↑(n i.2)
```

`Submodule.LinearDisjoint.linearIndependent_mul_of_flat_right`

∀ {R S : Type*} [CommRing R] [Ring S] [Algebra R S] {M N : Submodule R S}, M.LinearDisjoint N → ∀ [Module.Flat R ↥N] {κ : Type u_3} {ι : Type u_4} {m : κ → ↥M} {n : ι → ↥N}, LinearIndependent R m → LinearIndependent R n → LinearIndependent R fun i => ↑(m i.1) * ↑(n i.2)

`Submodule.LinearDisjoint.of_le_left_of_flat`

∀ {R S : Type*} [CommRing R] [Ring S] [Algebra R S] {M N : Submodule R S}, M.LinearDisjoint N → ∀ {M' : Submodule R S}, M' ≤ M → ∀ [Module.Flat R ↥N], M'.LinearDisjoint N

`Submodule.LinearDisjoint.of_le_right_of_flat`

∀ {R S : Type*} [CommRing R] [Ring S] [Algebra R S] {M N : Submodule R S}, M.LinearDisjoint N → ∀ {N' : Submodule R S}, N' ≤ N → ∀ [Module.Flat R ↥M], M.LinearDisjoint N'

LinearAlgebra · LinearIndependent · Defs

`LinearIndependent.neg`

∀ {ι : Type u_1} {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] {v : ι → M}, LinearIndependent R v → LinearIndependent R (-v)

AddCommSemigroup → AddCommMagma (2, magnitude 2)

LinearAlgebra · Matrix · Symmetric

`Matrix.isSymm_add_transpose_self`

∀ {α : Type u_1} {n : Type*} [AddCommSemigroup α] (A : Matrix n n α), (A + A.transpose).IsSymm

`Matrix.isSymm_transpose_add_self`

∀ {α : Type u_1} {n : Type*} [AddCommSemigroup α] (A : Matrix n n α), (A.transpose + A).IsSymm

CommMonoid → CommSemigroup (1, magnitude 2)

LinearAlgebra · Matrix · Hermitian

`Matrix.IsHermitian.hadamard`

∀ {α : Type u_1} {n : Type*} [CommMonoid α] [StarMul α] {A B : Matrix n n α}, A.IsHermitian → B.IsHermitian → (A.hadamard B).IsHermitian

LinearOrder → PartialOrder (1, magnitude 2)

LinearAlgebra · Matrix · Block

`Matrix.blockTriangular_algebraMap`

∀ {α : Type u_2} {m R A : Type*} {b : m → α} [LinearOrder α] [CommSemiring R] [Semiring A] [Algebra R A] [DecidableEq m] [Fintype m] (r : R), ((algebraMap R (Matrix m m A)) r).BlockTriangular b

NonUnitalSemiring → NonUnitalNonAssocSemiring (3, magnitude 1)

LinearAlgebra · Matrix · Hermitian

`Matrix.IsHermitian.commute_iff`

∀ {α : Type u_1} {n : Type*} [NonUnitalSemiring α] [StarRing α] [Fintype n] {A B : Matrix n n α}, A.IsHermitian → B.IsHermitian → (Commute A B ↔ (A * B).IsHermitian)

`Matrix.isHermitian_conjTranspose_mul_self`

∀ {α : Type u_1} {m n : Type*} [NonUnitalSemiring α] [StarRing α] [Fintype m] (A : Matrix m n α), (A.conjTranspose * A).IsHermitian

`Matrix.isHermitian_mul_conjTranspose_self`

∀ {α : Type u_1} {m n : Type*} [NonUnitalSemiring α] [StarRing α] [Fintype n] (A : Matrix m n α), (A * A.conjTranspose).IsHermitian

Field → DivisionRing (1, magnitude 1)

LinearAlgebra · Projectivization · PSL · PSL2

`field_cond_of_four_le_card` ?

signature not resolved

IsCancelAdd → IsLeftCancelAdd (1, unranked)

LinearAlgebra · Matrix · SemiringInverse

`Matrix.IsDetpBalanced.trans`

∀ {n R : Type*} [Fintype n] [DecidableEq n] [CommSemiring R] {A : Matrix n n R} {a b c : R} [IsCancelAdd R], A.IsDetpBalanced a b → A.IsDetpBalanced b c → A.IsDetpBalanced a c

MeasureTheory

21 survivors in 6 families; 1 gives up subtraction or division (†).

stated over	holds over	n	magnitude
`PseudoEMetricSpace`	`EDist`	1	5
`Group`	`DivInvMonoid`	5	2
`Group` †	`Monoid`	1	2
`NormedAddCommGroup`	`SeminormedAddCommGroup`	11	1
`NontriviallyNormedField`	`NormedField`	1	1
`DivisionMonoid`	`DivInvMonoid`	2	-1

PseudoEMetricSpace → EDist (1, magnitude 5)

MeasureTheory · Function · ConvergenceInMeasure

`MeasureTheory.ExistsSeqTendstoAe.exists_nat_measure_lt_two_inv`

```
∀ {α : Type u_1} {E : Type*} {m : MeasurableSpace α} {μ : MeasureTheory.Measure α} [PseudoEMetricSpace E] {f : ℕ → α → E} {g : α → E}, MeasureTheory.TendstoInMeasure μ f Filter.atTop g → ∀ (n : ℕ), ∃ N, ∀ m_1 ≥ N, μ {x | 2⁻¹ ^ n ≤ edist (f m_1 x) (g x)} ≤ 2⁻¹ ^ n
```

Group → DivInvMonoid (5, magnitude 2)

MeasureTheory · Function · StronglyMeasurable · Basic

`MeasureTheory.StronglyMeasurable.leOnePart`

```
∀ {α : Type u_1} {β : Type u_2} {f : α → β} [MeasurableSpace α] [TopologicalSpace β] [Group β] [Lattice β] [ContinuousSup β] [ContinuousInv β], MeasureTheory.StronglyMeasurable f → MeasureTheory.StronglyMeasurable fun x => (f x)⁻ᵐ
```

`MeasureTheory.StronglyMeasurable.oneLePart`

```
∀ {α : Type u_1} {β : Type u_2} {f : α → β} [MeasurableSpace α] [TopologicalSpace β] [Group β] [Lattice β] [ContinuousSup β], MeasureTheory.StronglyMeasurable f → MeasureTheory.StronglyMeasurable fun x => (f x)⁺ᵐ
```

MeasureTheory · Group · Measure

`MeasureTheory.measurePreserving_div_right`

```
∀ {G : Type*} [MeasurableSpace G] [Group G] [MeasurableMul G] (μ : MeasureTheory.Measure G) [μ.IsMulRightInvariant] (g : G), MeasureTheory.MeasurePreserving (fun x => x / g) μ μ
```

MeasureTheory · Order · Group · Lattice

`measurable_leOnePart`

```
∀ {α : Type u_1} [Lattice α] [Group α] [MeasurableSpace α] [MeasurableInv α] [MeasurableSup α], Measurable leOnePart
```

`measurable_oneLePart`

```
∀ {α : Type u_1} [Lattice α] [Group α] [MeasurableSpace α] [MeasurableSup α], Measurable oneLePart
```

Group → Monoid (1, magnitude 2) †

MeasureTheory · Group · FundamentalDomain

`MeasureTheory.IsFundamentalDomain.hasFundamentalDomain`

```
∀ {G : Type*} {α : Type u_2} [Group G] [MulAction G α] [MeasurableSpace α] (ν : MeasureTheory.Measure α) {s : Set α}, MeasureTheory.IsFundamentalDomain G s ν → MeasureTheory.HasFundamentalDomain G α ν
```

NormedAddCommGroup → SeminormedAddCommGroup (11, magnitude 1)

MeasureTheory · Constructions · BorelSpace · Metric

`measurable_nnnorm`

```
∀ {α : Type u_1} [MeasurableSpace α] [NormedAddCommGroup α] [OpensMeasurableSpace α], Measurable nnnorm
```

`measurable_norm`

```
∀ {α : Type u_1} [MeasurableSpace α] [NormedAddCommGroup α] [OpensMeasurableSpace α], Measurable norm
```

MeasureTheory · Function · LpSpace · Complete

`MeasureTheory.Lp.cauchy_tendsto_of_tendsto`

```
∀ {α : Type u_1} {m : MeasurableSpace α} {p : ENNReal} {μ : MeasureTheory.Measure α} {E : Type*} [NormedAddCommGroup E] {f : ℕ → α → E}, (∀ (n : ℕ), MeasureTheory.AEStronglyMeasurable (f n) μ) → ∀ (f_lim : α → E) {B : ℕ → ENNReal}, ∑' (i : ℕ), B i ≠ □ → (∀ (N n m_1 : ℕ), N ≤ n → N ≤ m_1 → MeasureTheory.eLpNorm (f n - f m_1) p μ < B N) → (∀ᵐ (x : α) ∂μ, Filter.Tendsto (fun n => f n x) Filter.atTop (nhds (f_lim x))) → Filter.Tendsto (fun n => MeasureTheory.eLpNorm (f n - f_lim) p μ) Filter.atTop (nhds 0)
```

MeasureTheory · Function · SimpleFuncDenseLp

`MeasureTheory.SimpleFunc.nnnorm_approxOn_le`

```
∀ {β : Type u_1} {E : Type*} [MeasurableSpace β] [MeasurableSpace E] [NormedAddCommGroup E] [OpensMeasurableSpace E] {f : β → E} (hf : Measurable f) {s : Set E} {y₀ : E} (h₀ : y₀ ∈ s) [TopologicalSpace.SeparableSpace ↑s] (x : β) (n : ℕ), □(MeasureTheory.SimpleFunc.approxOn f hf s y₀ h₀ n) x - f x□₊ ≤ □f x - y₀□₊
```

MeasureTheory.SimpleFunc.norm_approxOn_y₀_le

```
∀ {β : Type u_1} {E : Type*} [MeasurableSpace β] [MeasurableSpace E] [NormedAddCommGroup E] [OpensMeasurableSpace E] {f : β → E} (hf : Measurable f) {s : Set E} {y₀ : E} (h₀ : y₀ ∈ s) [TopologicalSpace.SeparableSpace ↑s] (x : β) (n : ℕ), ‖(MeasureTheory.SimpleFunc.approxOn f hf s y₀ h₀ n) x − y₀‖ ≤ ‖f x − y₀‖ + ‖f x − y₀‖
```

MeasureTheory.SimpleFunc.norm_approxOn_zero_le

```
∀ {β : Type u_1} {E : Type*} [MeasurableSpace β] [MeasurableSpace E] [NormedAddCommGroup E] [OpensMeasurableSpace E] {f : β → E} (hf : Measurable f) {s : Set E} (h₀ : 0 ∈ s) [TopologicalSpace.SeparableSpace ↑s] (x : β) (n : ℕ), ‖(MeasureTheory.SimpleFunc.approxOn f hf s 0 h₀ n) x‖ ≤ ‖f x‖ + ‖f x‖
```

MeasureTheory · Function · UniformIntegrable

MeasureTheory.tendsto_indicator_ge

```
∀ {α : Type u_1} {β : Type u_2} [NormedAddCommGroup β] (f : α → β) (x : α), Filter.Tendsto (fun M => {x | ↑M ≤ ↑‖f x‖₊}.indicator f x) Filter.atTop (nhds 0)
```

MeasureTheory · Integral · BoundedContinuousFunction

BoundedContinuousFunction.lintegral_nnnorm_le

```
∀ {X : Type*} [MeasurableSpace X] [TopologicalSpace X] (μ : MeasureTheory.Measure X) {E : Type*} [NormedAddCommGroup E] (f : BoundedContinuousFunction X E), ∫⁻ (x : X), ↑‖f x‖₊ ∂μ ≤ ↑‖f‖₊ * μ Set.univ
```

MeasureTheory · Integral · IntegralEqImproper

MeasureTheory.AECover.integrable_of_lintegral_enorm_bounded

```
∀ {α : Type u_1} {ι : Type u_2} {E : Type*} [MeasurableSpace α] {μ : MeasureTheory.Measure α} {l : Filter ι} [NormedAddCommGroup E] [l.NeBot] [l.IsCountablyGenerated] {φ : ι → Set α}, MeasureTheory.AECover μ l φ → ∀ {f : α → E} (I : ℝ), MeasureTheory.AEStronglyMeasurable f μ → (∀ᶠ (i : ι) in l, ∫⁻ (x : α) in φ i, ‖f x‖ₑ ∂μ ≤ ENNReal.ofReal I) → MeasureTheory.Integrable f μ
```

MeasureTheory · SpecificCodomains · ContinuousMap

ContinuousMap.aeStronglyMeasurable_mkD_of_uncurry ‡

```
∀ {X Y : Type*} [MeasurableSpace X] {μ : MeasureTheory.Measure X} [TopologicalSpace Y] {E : Type*} [NormedAddCommGroup E] [CompactSpace Y] [TopologicalSpace X] [OpensMeasurableSpace X] [SecondCountableTopologyEither X C(Y, E)] (f : X → Y → E) (g : C(Y, E)), Continuous (Function.uncurry f) → MeasureTheory.AEStronglyMeasurable (fun x => ContinuousMap.mkD (f x) g) μ
```

ContinuousMap.aeStronglyMeasurable_restrict_mkD_restrict_of_uncurry ‡

```
∀ {X Y : Type*} [MeasurableSpace X] {μ : MeasureTheory.Measure X} [TopologicalSpace Y] {E : Type*} [NormedAddCommGroup E] {s : Set X} {t : Set Y} [CompactSpace ↑t] [TopologicalSpace X] [OpensMeasurableSpace X] [SecondCountableTopologyEither X C(↑t, E)], MeasurableSet s → ∀ (f : X → Y → E) (g : C(↑t, E)), ContinuousOn (Function.uncurry f) (s ×ˢ t) → MeasureTheory.AEStronglyMeasurable (fun x => ContinuousMap.mkD (t.domRestrict (f x)) g) (μ.restrict s)
```

NontriviallyNormedField → NormedField (1, magnitude 1)

MeasureTheory · Measure · ResolventTransform

MeasureTheory.norm_resolvent_le_inv_infDist_support

```
∀ {𝕜 : Type u_1} {A : Type*} [NontriviallyNormedField 𝕜] [MeasurableSpace 𝕜] [CompleteSpace 𝕜] [NormedDivisionRing A] [NormedAlgebra 𝕜 A] {μ : MeasureTheory.Measure 𝕜} {a : A}, a ∉ ⇑(algebraMap 𝕜 A) '' μ.support → ∀ {x : 𝕜}, x ∈ μ.support → ‖resolvent a x‖ ≤ (Metric.infDist a (⇑(algebraMap 𝕜 A) '' μ.support))⁻¹
```

DivisionMonoid → DivInvMonoid (2, unranked)

MeasureTheory · Group · Measure

MeasureTheory.Measure.measurePreserving_div_left

```
∀ {G : Type*} [MeasurableSpace G] [DivisionMonoid G] [MeasurableMul G] [MeasurableInv G] (μ : MeasureTheory.Measure G) [μ.IsInvInvariant] [μ.IsMulLeftInvariant] (g : G), MeasureTheory.MeasurePreserving (fun t => g / t) μ μ
```

MeasureTheory.Measure.measurePreserving_mul_right_inv

```
∀ {G : Type*} [MeasurableSpace G] [DivisionMonoid G] [MeasurableMul G] [MeasurableInv G] (μ : MeasureTheory.Measure G) [μ.IsInvInvariant] [μ.IsMulLeftInvariant] (g : G), MeasureTheory.MeasurePreserving (fun t => (g * t)⁻¹) μ μ
```

NumberTheory

14 survivors in 6 families; 4 give up subtraction or division (†).

stated over	holds over	n	magnitude
CommRing	CommMonoid	1	4
Field	CommRing	3	3
CommRing †	CommSemiring	4	2
NontriviallyNormedField	NormedField	3	1
Field	DivisionRing	2	1
CommRing	Ring	1	1

CommRing → CommMonoid (1, magnitude 4)

NumberTheory · MulChar · Lemmas

MulChar.apply_mem_rootsOfUnity

∀ {R R' : Type*} [CommRing R] [CommRing R'] [Fintype Rˣ] (a : Rˣ) {χ : MulChar R R'}, (MulChar.equivToUnitHom χ) a ∈ rootsOfUnity (Fintype.card Rˣ) R'

Field → CommRing (3, magnitude 3)

NumberTheory · Cyclotomic · Basic

IsCyclotomicExtension.splits_X_pow_sub_one

∀ {n : ℕ} [NeZero n] {S : Set ℕ} (K : Type u_1) (L : Type u_2) [Field K] [Field L] [Algebra K L] [H : IsCyclotomicExtension S K L], n ∈ S → (Polynomial.map (algebraMap K L) (Polynomial.X ^ n - 1)).Splits

NumberTheory · Height · MvPolynomial

AbsoluteValue.iSup_abv_linearMap_apply_le

∀ {K : Type*} [Field K] {ι : Type u_2} {ι' : Type u_3} [Fintype ι] [Finite ι'] (v : AbsoluteValue K ℝ) (A : ι' × ι → K) (x : ι → K), □ j, v (∑ i, A (j, i) * x i) ≤ (↑(Nat.card ι) * □ ji, v (A ji)) * □ i, v (x i)

IsNonarchimedean.iSup_abv_linearMap_apply_le

∀ {K : Type*} [Field K] {ι : Type u_2} {ι' : Type u_3} [Fintype ι] [Finite ι'] {v : AbsoluteValue K ℝ}, IsNonarchimedean ⇑v → ∀ (A : ι' × ι → K) (x : ι → K), □ j, v (∑ i, A (j, i) * x i) ≤ (□ ji, v (A ji)) * □ i, v (x i)

CommRing → CommSemiring (4, magnitude 2) †

NumberTheory · MulChar · Duality

MulChar.mulEquiv_units

∀ (M : Type u_1) (R : Type u_2) [CommMonoid M] [CommRing R] [Finite M] [HasEnoughRootsOfUnity R (Monoid.exponent Mˣ)], Nonempty (MulChar M R ≃* Mˣ)

MulChar.restrictHom_surjective ‡

∀ (M : Type u_1) (R : Type u_2) [CommMonoid M] [CommRing R] [Finite M] [HasEnoughRootsOfUnity R (Monoid.exponent Mˣ)] (N : Submonoid M), Function.Surjective ⇑(MulChar.domRestrictHom N R)

NumberTheory · MulChar · Lemmas

MulChar.orderOf_dvd_card_sub_one

∀ (F : Type u_1) [Field F] [Fintype F] {R : Type*} [CommRing R] (χ : MulChar F R), orderOf χ □ Fintype.card F - 1

NumberTheory · RamificationInertia · Basic

Ideal.Factors.ne_bot

∀ {R : Type*} [CommRing R] {S : Type*} [CommRing S] [Algebra R S] (p : Ideal R) [IsDedekindDomain S] (P : ↥(UniqueFactorizationMonoid.factors (Ideal.map (algebraMap R S) p)).toFinset), ↑P ≠ □

NontriviallyNormedField → NormedField (3, magnitude 1)

NumberTheory · TsumDivisorsAntidiagonal

summable_divisorsAntidiagonal_aux ?

signature not resolved

summable_norm_pow_mul_geometric_div_one_sub ‡

∀ {□ : Type u_1} [NontriviallyNormedField □] [CompleteSpace □] (k : ℕ) {r : □}, □r□ < 1 → Summable fun n => ↑n ^ k * r ^ n / (1 - r ^ n)

tendsto_zero_geometric_tsum_pnat ‡

∀ {□ : Type u_1} [NontriviallyNormedField □] {r : □}, □r□ < 1 → Filter.Tendsto (fun m => ∑' (n : ℕ+), r ^ (↑n * ↑m)) Filter.atTop (nhds 0)

Field → DivisionRing (2, magnitude 1)

NumberTheory · FLT · Polynomial

`Ne.isUnit_C` ?

signature not resolved

NumberTheory · FunctionField

`FunctionField.algebraMap_injective` ‡

```
∀ (F : Type u_1) (K : Type u_2) [Field F] [Field K] [Algebra (Polynomial F) K] [Algebra (RatFunc F) K] [IsScalarTower (Polynomial F) (RatFunc F) K], Function.Injective ⇑(algebraMap (Polynomial F) K)
```

CommRing → Ring (1, magnitude 1)

NumberTheory · LegendreSymbol · AddCharacter

`AddChar.val_mem_rootsOfUnity`

```
∀ {R : Type*} [CommRing R] {R' : Type*} [CommMonoid R'] (φ : AddChar R R') (a : R), 0 < ringChar R → ⋯.unit ∈ rootsOfUnity (↑(ringChar R).toPNat') R'
```

Order

45 survivors in 8 families; 0 give up subtraction or division (†).

stated over	holds over	n	magnitude
`LinearOrder`	`PartialOrder`	4	2
`Monoid`	`MulOneClass`	1	2
`PartialOrder`	`Preorder`	25	1
`MulOneClass`	`MulOne`	3	1
`CommGroup`	`Group`	1	1
`Lattice`	`SemilatticeInf`	3	0
`CompleteLattice`	`CompleteSemilatticeSup`	6	-1
`OrderTop`	`Top`	2	-1

LinearOrder → PartialOrder (4, magnitude 2)

Order · Filter · AtTopBot · Archimedean

`Filter.Tendsto.atTop_nsmul_const`

`∀ {α : Type u_1} {R : Type*} {l : Filter α} {r : R} [AddCommMonoid R] [LinearOrder R] [IsOrderedCancelAddMonoid R] [Archimedean R] {f : α → ℕ}, 0 < r → Filter.Tendsto f l Filter.atTop → Filter.Tendsto (fun x => f x • r) l Filter.atTop`

`Filter.Tendsto.atTop_zsmul_const`

`∀ {α : Type u_1} {R : Type*} {l : Filter α} {r : R} [AddCommGroup R] [LinearOrder R] [IsOrderedAddMonoid R] [Archimedean R] {f : α → ℤ}, 0 < r → Filter.Tendsto f l Filter.atTop → Filter.Tendsto (fun x => f x • r) l Filter.atTop`

Order · Interval · Set · ProjIcc

`Set.OrdConnected.domRestrict`

`∀ {α : Type u_1} [LinearOrder α] {s t : Set α}, s.OrdConnected → {x | t.domRestrict (fun x => x ∈ s) x}.OrdConnected`

Order · Monotone · Extension

`MonotoneOn.exists_monotone_extension`

`∀ {α : Type u_1} {β : Type u_2} [LinearOrder α] [ConditionallyCompleteLinearOrder β] {f : α → β} {s : Set α}, MonotoneOn f s → BddBelow (f '' s) → BddAbove (f '' s) → ∃ g, Monotone g ∧ Set.EqOn f g s`

Monoid → MulOneClass (1, magnitude 2)

Order · Filter · IndicatorFunction

`mulIndicator_union_eventuallyEq`

`∀ {α : Type u_1} {M : Type*} [Monoid M] {s t : Set α} {f : α → M} {l : Filter α}, (∀ᶠ (a : α) in l, a ∉ s ∩ t) → (s ∪ t).mulIndicator f =ᶠ[l] s.mulIndicator f * t.mulIndicator f`

PartialOrder → Preorder (25, magnitude 1)

Order · Antichain

`IsAntichain.of_monotoneOn_strictAntiOn`

`∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {f : α → β} {s : Set α}, MonotoneOn f s → StrictAntiOn f s → IsAntichain (fun x1 x2 => x1 ≤ x2) s`

`IsAntichain.of_strictMonoOn_antitoneOn`

`∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {f : α → β} {s : Set α}, StrictMonoOn f s → AntitoneOn f s → IsAntichain (fun x1 x2 => x1 ≤ x2) s`

Order · Atoms

`IsStronglyAtomic.of_wellFounded_lt`

`∀ {α : Type u_1} [PartialOrder α], (WellFounded fun x1 x2 => x1 < x2) → IsStronglyAtomic α`

Order · Birkhoff

`UpperSet.infIrred_Ici`

`∀ {α : Type u_1} [PartialOrder α] (a : α), InfIrred (UpperSet.Ici a)`

`LowerSet.supIrred_Iic` ‡

`∀ {α : Type u_1} [PartialOrder α] (a : α), SupIrred (LowerSet.Iic a)`

Order · Cover

`Prod._root_.WCovBy.fst`

`∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x ⩿ y → x.1 ⩿ y.1`

`Prod._root_.WCovBy.fst`

`∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x ⩿ y → x.1 ⩿ y.1`

`Prod._root_.WCovBy.snd`

`∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x ⩿ y → x.2 ⩿ y.2`

`Prod._root_.WCovBy.snd`

```
∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x □ y → x.2 □ y.2
```

`Prod.swap_covBy_swap`

```
∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x.swap □ y.swap ↔ x □ y
```

`Prod.swap_covBy_swap`

```
∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x.swap □ y.swap ↔ x □ y
```

`Prod.swap_wcovBy_swap`

```
∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x.swap □ y.swap ↔ x □ y
```

`Prod.swap_wcovBy_swap`

```
∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] {x y : α × β}, x.swap □ y.swap ↔ x □ y
```

Order · Filter · AtTopBot · Group

`Filter.tendsto_atTop_mul_left_of_le'`

```
∀ {α : Type u_1} {G : Type*} [CommGroup G] [PartialOrder G] [IsOrderedMonoid G] (l : Filter α) {f g : α → G} (C : G), (∀ᶠ (x : α) in l, C ≤ f x) → Filter.Tendsto g l Filter.atTop → Filter.Tendsto (fun x => f x * g x) l Filter.atTop
```

`Filter.tendsto_atTop_mul_right_of_le'`

```
∀ {α : Type u_1} {G : Type*} [CommGroup G] [PartialOrder G] [IsOrderedMonoid G] (l : Filter α) {f g : α → G} (C : G), Filter.Tendsto f l Filter.atTop → (∀ᶠ (x : α) in l, C ≤ g x) → Filter.Tendsto (fun x => f x * g x) l Filter.atTop
```

`Filter.tendsto_comp_inv_atTop_iff` ‡

```
∀ {α : Type u_1} {G : Type*} [CommGroup G] [PartialOrder G] [IsOrderedMonoid G] {l : Filter α} {f : G → α}, Filter.Tendsto (fun x => f x⁻¹) Filter.atTop l ↔ Filter.Tendsto f Filter.atBot l
```

`Filter.tendsto_inv_atBot_iff`

```
∀ {α : Type u_1} {G : Type*} [CommGroup G] [PartialOrder G] [IsOrderedMonoid G] {l : Filter α} {f : α → G}, Filter.Tendsto (fun x => (f x)⁻¹) l Filter.atBot ↔ Filter.Tendsto f l Filter.atTop
```

`Filter.tendsto_inv_atTop_atBot`

```
∀ {G : Type*} [CommGroup G] [PartialOrder G] [IsOrderedMonoid G], Filter.Tendsto Inv.inv Filter.atTop Filter.atBot
```

`Filter.tendsto_inv_atTop_iff`

```
∀ {α : Type u_1} {G : Type*} [CommGroup G] [PartialOrder G] [IsOrderedMonoid G] {l : Filter α} {f : α → G}, Filter.Tendsto (fun x => (f x)⁻¹) l Filter.atTop ↔ Filter.Tendsto f l Filter.atBot
```

Order · Interval · Basic

`NonemptyInterval.subset_coe_map` ‡

```
∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] (f : α →o β) (s : NonemptyInterval α), ⇑f '' ↑s ⊆ ↑(NonemptyInterval.map f s)
```

`NonemptyInterval.subset_coe_map` ‡

```
∀ {α : Type u_1} {β : Type u_2} [PartialOrder α] [PartialOrder β] (f : α →o β) (s : NonemptyInterval α), ⇑f '' ↑s ⊆ ↑(NonemptyInterval.map f s)
```

Order · Interval · Finset · Basic

`Finset.Ico_disjoint_Ico_consecutive`

```
∀ {α : Type u_1} [PartialOrder α] [LocallyFiniteOrder α] (a b c : α), Disjoint (Finset.Ico a b) (Finset.Ico b c)
```

`Finset.notMem_Iio_self`

```
∀ {α : Type u_1} [PartialOrder α] [LocallyFiniteOrderBot α] {b : α}, b ∉ Finset.Iio b
```

`Finset.notMem_Ioi_self`

```
∀ {α : Type u_1} [PartialOrder α] [LocallyFiniteOrderTop α] {b : α}, b ∉ Finset.Ioi b
```

Order · Interval · Multiset

`Multiset.Ico_disjoint_Ico`

```
∀ {α : Type u_1} [PartialOrder α] [LocallyFiniteOrder α] {a b c d : α}, b ≤ c → Disjoint (Multiset.Ico a b) (Multiset.Ico c d)
```

MulOneClass → MulOne (3, magnitude 1)

Order · Filter · Pointwise

`Filter.Tendsto.mul_mul`

```
∀ {F : Type*} {α : Type u_2} {β : Type u_3} [MulOneClass α] [MulOneClass β] [FunLike F α β] [MulHomClass F α β] (m : F) {f₁ g₁ : Filter α} {f₂ g₂ : Filter β}, Filter.Tendsto (⇑m) f₁ f₂ → Filter.Tendsto (⇑m) g₁ g₂ → Filter.Tendsto (⇑m) (f₁ * g₁) (f₂ * g₂)
```

`Filter.Tendsto.mul_mul`

```
∀ {F : Type*} {α : Type u_2} {β : Type u_3} [MulOneClass α] [MulOneClass β] [FunLike F α β] [MulHomClass F α β] (m : F) {f₁ g₁ : Filter α} {f₂ g₂ : Filter β}, Filter.Tendsto (⇑m) f₁ f₂ → Filter.Tendsto (⇑m) g₁ g₂ → Filter.Tendsto (⇑m) (f₁ * g₁) (f₂ * g₂)
```

`Filter.comap_mul_comap_le` ‡

```
∀ {F : Type*} {α : Type u_2} {β : Type u_3} [MulOneClass α] [MulOneClass β] [FunLike F α β] [MulHomClass F α β] (m : F) {f g : Filter β}, Filter.comap (⇑m) f * Filter.comap (⇑m) g ≤ Filter.comap (⇑m) (f * g)
```

CommGroup → Group (1, magnitude 1)

Order · Filter · AtTopBot · Group

`Filter.tendsto_mabs_atTop_atTop`

```
∀ {G : Type*} [CommGroup G] [LinearOrder G], Filter.Tendsto mabs Filter.atTop Filter.atTop
```

Lattice → SemilatticeInf (3, magnitude 0)

Order · Disjoint

`isCompl_ofDual_iff`

```
∀ {α : Type u_1} [Lattice α] [BoundedOrder α] {a b : αᵒᵈ}, IsCompl (OrderDual.ofDual a) (OrderDual.ofDual b) ↔ IsCompl a b
```

`isCompl_toDual_iff`

```
∀ {α : Type u_1} [Lattice α] [BoundedOrder α] {a b : α}, IsCompl (OrderDual.toDual a) (OrderDual.toDual b) ↔ IsCompl a b
```

Order · Hom · BoundedLattice

`Disjoint.map`

```
∀ {F : Type*} {α : Type u_2} {β : Type u_3} [Lattice α] [Lattice β] [FunLike F α β] [OrderBot α] [OrderBot β] [BotHomClass F α β] [InfHomClass F α β] {a b : α} (f : F), Disjoint a b → Disjoint (f a) (f b)
```

CompleteLattice → CompleteSemilatticeSup (6, unranked)

Order · CompactlyGenerated · Basic

`CompleteLattice.isCompactElement_iff_le_of_directed_sSup_le` ‡

```
∀ (α : Type u_1) [CompleteLattice α] (k : α), IsCompactElement k ↔ ∀ (s : Set α), s.Nonempty → DirectedOn (fun x1 x2 => x1 ≤ x2) s → k ≤ sSup s → ∃ x ∈ s, k ≤ x
```

Order · CompleteLattice · Basic

`iSup_le` ‡

```
∀ {α : Type u_1} {ι : Sort u_2} [CompleteLattice α] {f : ι → α} {a : α}, (∀ (i : ι), f i ≤ a) → iSup f ≤ a
```

Order · DirSupClosed

`dirSupClosedOn_iff_forall_sSup`

```
∀ {α : Type u_1} {s : Set α} {D : Set (Set α)} [CompleteLattice α], DirSupClosedOn D s ↔ ∀ □d : Set α□ , d ∈ D → d ⊆ s → d.Nonempty → DirectedOn (fun x1 x2 => x1 ≤ x2) d → sSup d ∈ s
```

`dirSupClosed_iff_forall_sSup`

```
∀ {α : Type u_1} {s : Set α} [CompleteLattice α], DirSupClosed s ↔ ∀ □d : Set α□ , d ⊆ s → d.Nonempty → DirectedOn (fun x1 x2 => x1 ≤ x2) d → sSup d ∈ s
```

`dirSupInaccOn_iff_forall_sSup`

```
∀ {α : Type u_1} {s : Set α} {D : Set (Set α)} [CompleteLattice α], DirSupInaccOn D s ↔ ∀ □d : Set α□ , d ∈ D → d.Nonempty → DirectedOn (fun x1 x2 => x1 ≤ x2) d → sSup d ∈ s → (d ∩ s).Nonempty
```

`dirSupInacc_iff_forall_sSup`

```
∀ {α : Type u_1} {s : Set α} [CompleteLattice α], DirSupInacc s ↔ ∀ □d : Set α□ , d.Nonempty → DirectedOn (fun x1 x2 => x1 ≤ x2) d → sSup d ∈ s → (d ∩ s).Nonempty
```

OrderTop → Top (2, unranked)

Order · BoundedOrder · Lattice

`WellFoundedGT.induction_top`

```
∀ {α : Type u_1} [Preorder α] [WellFoundedGT α] [OrderTop α] {P : α → Prop}, (∃ M, P M) → (∀ (N : α), N ≠ □ → P N → ∃ M > N, P M) → P □
```

Order · KrullDimension

`Order.coheight_pos_of_lt_top`

```
∀ {α : Type u_1} [Preorder α] {x : α} [OrderTop α], x < □ → 0 < Order.coheight x
```

Probability

7 survivors in 3 families; 0 give up subtraction or division (†).

stated over	holds over	n	magnitude
`Group`	`DivInvMonoid`	2	2
`LinearOrder`	`PartialOrder`	2	2
`NormedAddCommGroup`	`SeminormedAddCommGroup`	3	1

Group → DivInvMonoid (2, magnitude 2)

Probability · Process · Adapted

`MeasureTheory.IsStronglyProgressive.div'`

```
∀ {Ω : Type u_1} {ι : Type u_2} {m : MeasurableSpace Ω} [Preorder ι] {f : MeasureTheory.Filtration ι m} {β : Type u_3} [TopologicalSpace β] {u v : ι → Ω → β} [MeasurableSpace ι] [Group β] [ContinuousDiv β], MeasureTheory.IsStronglyProgressive f u → MeasureTheory.IsStronglyProgressive f v → MeasureTheory.IsStronglyProgressive f fun i ω => u i ω / v i ω
```

`MeasureTheory.IsStronglyProgressive.inv`

```
∀ {Ω : Type u_1} {ι : Type u_2} {m : MeasurableSpace Ω} [Preorder ι] {f : MeasureTheory.Filtration ι m} {β : Type u_3} [TopologicalSpace β] {u : ι → Ω → β} [MeasurableSpace ι] [Group β] [ContinuousInv β], MeasureTheory.IsStronglyProgressive f u → MeasureTheory.IsStronglyProgressive f fun i ω => (u i ω)⁻¹
```

LinearOrder → PartialOrder (2, magnitude 2)

Probability · Process · Predictable

`MeasureTheory.measurableSet_predictable_Ioi_prod`

```
∀ {Ω : Type u_1} {ι : Type u_2} {m : MeasurableSpace Ω} [LinearOrder ι] [OrderBot ι] {□ : MeasureTheory.Filtration ι m} {i : ι} {s : Set Ω}, MeasurableSet s → MeasurableSet (Set.Ioi i ×ˢ s)
```

`MeasureTheory.measurableSet_predictable_singleton_bot_prod`

```
∀ {Ω : Type u_1} {ι : Type u_2} {m : MeasurableSpace Ω} [LinearOrder ι] [OrderBot ι] {□ : MeasureTheory.Filtration ι m} {s : Set Ω}, MeasurableSet s → MeasurableSet ({□} ×ˢ s)
```

NormedAddCommGroup → SeminormedAddCommGroup (3, magnitude 1)

Probability · Distributions · Gaussian · Basic

`ProbabilityTheory.IsGaussian.memLp_dual`

```
∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [MeasurableSpace E] [BorelSpace E] (μ : MeasureTheory.Measure E) [ProbabilityTheory.IsGaussian μ] (L : StrongDual ℝ E) (p : ENNReal), p ≠ □ → MeasureTheory.MemLp (⇑L) p μ
```

Probability · Distributions · Gaussian · Fernique

`ProbabilityTheory.IsGaussian.integrable_exp_sq_of_conv_neg`

```
∀ {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [MeasurableSpace E] [BorelSpace E] [SecondCountableTopology E] (μ : MeasureTheory.Measure E) [ProbabilityTheory.IsGaussian μ] {C C' : ℝ}, MeasureTheory.Integrable (fun x => Real.exp (C * □x□ ^ 2)) (μ.conv (MeasureTheory.Measure.map (⇑(ContinuousLinearEquiv.neg ℝ)) μ)) → 0 < C' → C' < C → MeasureTheory.Integrable (fun x => Real.exp (C' * □x□ ^ 2)) μ
```

Probability · Distributions · Poisson · Basic

`ProbabilityTheory.integrable_map_cast_poissonMeasure_iff`

```
∀ {E : Type*} [NormedAddCommGroup E] {R : Type*} [NatCast R] [MeasurableSpace R] {r : NNReal} [Countable R] [MeasurableSingletonClass R] {f : R → E}, MeasureTheory.Integrable f (MeasureTheory.Measure.map Nat.cast (ProbabilityTheory.poissonMeasure r)) ↔ MeasureTheory.Integrable (f ∘ Nat.cast) (ProbabilityTheory.poissonMeasure r)
```

RepresentationTheory

1 survivor in 1 family; 1 gives up subtraction or division (†).

stated over	holds over	n	magnitude
Ring †	Semiring	1	2

Ring → Semiring (1, magnitude 2) †

RepresentationTheory · Basic

`Representation.apply_sub_id_partialSum_eq`

```
∀ {k G V : Type*} [Ring k] [Monoid G] [AddCommGroup V] [Module k V] (ρ : Representation k G V) (n : ℕ) (g : G) (x : V), (ρ g - LinearMap.id) (Fin.partialSum (fun j => (ρ (g ^ ↑j)) x) (Fin.last (n + 1))) = (ρ (g ^ (n + 1))) x - x
```

RingTheory

66 survivors in 14 families; 25 give up subtraction or division (†).

stated over	holds over	n	magnitude
`IsDomain`	`NoZeroDivisors`	10	3
`Field`	`CommRing`	5	3
`CommRing` †	`CommSemiring`	13	2
`Ring` †	`Semiring`	8	2
`AddCommGroup` †	`AddCommMonoid`	4	2
`LinearOrder`	`PartialOrder`	2	2
`Monoid`	`MulOneClass`	2	2
`CommRing`	`Ring`	9	1
`CommSemiring`	`Semiring`	6	1
`NonUnitalRing`	`NonUnitalNonAssocRing`	2	1
`AddCommMonoid`	`AddMonoid`	1	1
`CommMonoid`	`Monoid`	1	1
`IsDomain`	`Nontrivial`	2	-1
`ValuationRing`	`PreValuationRing`	1	-1

IsDomain → NoZeroDivisors (10, magnitude 3)

RingTheory · AlgebraTower

`Module.Basis.algebraMap_injective`

```
∀ {R S : Type*} [CommRing R] [IsDomain R] [Ring S] [Nontrivial S] [Algebra R S] {ι : Type u_3} (b : Module.Basis ι R S), Function.Injective ⇑(algebraMap R S)
```

RingTheory · Algebraic · Cardinality

`Algebra.IsAlgebraic.lift_cardinalMk_le_sigma_polynomial` ‡

```
∀ (R : Type u_1) [CommRing R] [IsDomain R] (L : Type u_2) [CommRing L] [IsDomain L] [Algebra R L] [Module.IsTorsionFree R L] [Algebra.IsAlgebraic R L], Cardinal.lift.{u_1, u_2} (Cardinal.mk L) ≤ Cardinal.mk ((p : Polynomial R) × { x // x ∈ p.aroots L })
```

RingTheory · DedekindDomain · Dvr

`IsLocalization.AtPrime.not_isField`

```
∀ (A : Type u_1) [CommRing A] [IsDomain A] {P : Ideal A}, P ≠ □ → ∀ [pP : P.IsPrime] (A□ : Type u_2) [CommRing A□] [Algebra A A□] [IsLocalization.AtPrime A□ P], ¬IsField A□
```

RingTheory · Ideal · Maps

`Ideal.map_ne_bot_of_ne_bot`

```
∀ {R S : Type*} [CommSemiring R] [Semiring S] [Algebra R S] [FaithfulSMul R S] {I : Ideal R}, I ≠ □ → Ideal.map (algebraMap R S) I ≠ □
```

RingTheory · IntegralClosure · Algebra · Basic

`isIntegral_of_smul_mem_submodule`

```
∀ {R A : Type*} [CommRing R] [CommRing A] [Algebra R A] [IsDomain A] {M : Type*} [AddCommGroup M] [Module R M] [Module A M] [IsScalarTower R A M] [Module.IsTorsionFree A M] (N : Submodule R M), N ≠ □ → N.FG → ∀ (x : A), (∀ n ∈ N, x • n ∈ N) → IsIntegral R x
```

RingTheory · IntegralClosure · IsIntegralClosure · Basic

`Algebra.IsIntegral.tower_bot`

```
∀ {R S : Type*} (T : Type u_3) [CommRing R] [CommRing S] [CommRing T] [IsDomain S] [Algebra R S] [Algebra R T] [Algebra S T] [Module.IsTorsionFree S T] [Nontrivial T] [IsScalarTower R S T] [h : Algebra.IsIntegral R T], Algebra.IsIntegral R S
```

RingTheory · Localization · Basic

`IsLocalization.map_injective_of_injective'`

```
∀ {R : Type*} [CommRing R] (M : Submonoid R) (S : Type u_2) [CommRing S] {f : R →+* S} {R□ : Type*} [CommRing R□] [Algebra R R□] [IsLocalization M R□] (S□ : Type u_4) {N : Submonoid S} [CommRing S□] [Algebra S S□] [IsLocalization N S□] (hf : M ≤ Submonoid.comap f N), 0 ∉ N → ∀ [IsDomain S], Function.Injective ⇑f → Function.Injective ⇑(IsLocalization.map S□ f hf)
```

RingTheory · PowerSeries · NoZeroDivisors

`PowerSeries.rescale_injective`

```
∀ {R : Type*} [CommRing R] [IsDomain R] {a : R}, a ≠ 0 → Function.Injective ⇑(PowerSeries.rescale a)
```

RingTheory · PrincipalIdealDomain

`Prime.coprime_iff_not_dvd`

```
∀ {R : Type*} [CommRing R] [IsBezout R] [IsDomain R] {p n : R}, Prime p → (IsCoprime p n ↔ ¬p □ n)
```

RingTheory · RootsOfUnity · CyclotomicUnits

`IsPrimitiveRoot.associated_sub_one_pow_sub_one_of_coprime` ‡

```
∀ {n j : ℕ} {A : Type*} {ζ : A} [CommRing A] [IsDomain A], IsPrimitiveRoot ζ n → j.Coprime n → Associated (ζ - 1) (ζ ^ j - 1)
```

Field → CommRing (5, magnitude 3)

RingTheory · FractionalIdeal · Operations

`FractionalIdeal.isNoetherian_iff`

```
∀ {R₁ : Type*} [CommRing R₁] {K : Type*} [Field K] [Algebra R₁ K] {I : FractionalIdeal (nonZeroDivisors R₁) K}, IsNoetherian R₁ ↥↑I ↔ ∀ J ≤ I, (↑J).FG
```

RingTheory · Polynomial · Chebyshev

`Polynomial.Chebyshev.derivative_U_eval_one_dvd`

```
∀ {□ : Type u_1} [Field □] (n : ℤ), 3 □ (↑n + 2) * (↑n + 1) * ↑n
```

`Polynomial.Chebyshev.iterate_derivative_T_eval_one_dvd`

```
∀ {□ : Type u_1} [Field □] (n : ℤ) (k : ℕ), ↑(∏ l ∈ Finset.range k, (2 * l + 1)) □ ↑(∏ l ∈ Finset.range k, (n ^ 2 - ↑l ^ 2))
```

`Polynomial.Chebyshev.iterate_derivative_U_eval_one_dvd`

```
∀ {□ : Type u_1} [Field □] (n : ℤ) (k : ℕ), ↑(∏ l ∈ Finset.range k, (2 * l + 3)) □ ↑(∏ l ∈ Finset.range k, ((n + 1) ^ 2 - (↑l + 1) ^ 2)) * (↑n + 1)
```

RingTheory · Unramified · Field

`Algebra.FormallyUnramified.of_isSeparable` ‡

```
∀ (K : Type u_1) (L : Type u_2) [Field K] [Field L] [Algebra K L] [Algebra.IsSeparable K L], Algebra.FormallyUnramified K L
```

CommRing → CommSemiring (13, magnitude 2) †

RingTheory · DedekindDomain · Ideal · Basic

`Ideal.liesOver_iff_dvd_map` ‡

```
∀ {R A : Type*} [CommRing R] [CommRing A] [IsDedekindDomain A] [Algebra R A] {p : Ideal R} {P : Ideal A}, P ≠ □ → ∀ [p.IsMaximal], P.LiesOver p ↔ P □ Ideal.map (algebraMap R A) p
```

RingTheory · EssentialFiniteness

`Algebra.EssFiniteType.aux`

```
∀ (R : Type u_1) (S : Type u_2) (T : Type u_3) [CommRing R] [CommRing S] [CommRing T] [Algebra R S] [Algebra R T] [Algebra S T] [IsScalarTower R S T] (σ : Subalgebra R S), (∀ (s : S), ∃ t ∈ σ, IsUnit t ∧ s * t ∈ σ) → ∀ (τ : Set T), ∀ t ∈ Algebra.adjoin S τ, ∃ s ∈ σ, IsUnit s ∧ s • t ∈ Subalgebra.map (IsScalarTower.toAlgHom R S T) σ □ Algebra.adjoin R τ
```

`Algebra.essFiniteType_cond_iff`

```
∀ (R : Type u_1) (S : Type u_2) [CommRing R] [CommRing S] [Algebra R S] (σ : Finset S), IsLocalization (Submonoid.comap (algebraMap (↥(Algebra.adjoin R ↑σ)) S) (IsUnit.submonoid S)) S ↔ ∀ (s : S), ∃ t ∈ Algebra.adjoin R ↑σ, IsUnit t ∧ s * t ∈ Algebra.adjoin R ↑σ
```

RingTheory · Ideal · Quotient · Over

`Ideal.ncard_primesOver_lt_of_not_le`

```
∀ {R S T : Type*} [CommRing R] [CommRing S] [CommRing T] [Algebra R S] [Algebra R T] (f : S →ₐ[R] T), Function.Surjective ⇑f → ∀ (P : Ideal R) (P' : Ideal S) [P'.IsPrime] [P'.LiesOver P], ¬RingHom.ker f.toRingHom ≤ P' → (P.primesOver S).Finite → (P.primesOver T).ncard < (P.primesOver S).ncard
```

RingTheory · Invariant · Basic

`Algebra.IsInvariant.charpoly_mem_lifts`

```
∀ (A : Type u_1) (B : Type u_2) (G : Type u_3) [CommRing A] [CommRing B] [Algebra A B] [Group G] [MulSemiringAction G B] [Algebra.IsInvariant A B G] [Fintype G] (b : B), MulSemiringAction.charpoly G b ∈ Polynomial.lifts (algebraMap A B)
```

RingTheory · LinearDisjoint

`Subalgebra.LinearDisjoint.exists_field_of_isDomain_of_injective`

```
∀ (R : Type u_1) [CommRing R] (A : Type u_2) [CommRing A] (B : Type u_3) [CommRing B] [Algebra R A] [Algebra R B] [Module.Flat R A] [Module.Flat R B] [IsDomain (TensorProduct R A B)], Function.Injective ⇑(algebraMap R A) → Function.Injective ⇑(algebraMap R B) → ∃ K x x_1 fa fb, Function.Injective ⇑fa ∧ Function.Injective ⇑fb ∧ fa.range.LinearDisjoint fb.range
```

RingTheory · Localization · AsSubring

`Localization.map_isUnit_of_le`

```
∀ {A : Type*} (K : Type u_2) [CommRing A] (S : Submonoid A) [CommRing K] [Algebra A K] [IsFractionRing A K], S ≤ nonZeroDivisors A → ∀ (s : ↥S), IsUnit ((algebraMap A K) ↑s)
```

RingTheory · Localization · Basic

`IsLocalization.map_injective_of_injective'` ‡

```
∀ {R : Type*} [CommRing R] (M : Submonoid R) (S : Type u_2) [CommRing S] {f : R →+* S} {R□ : Type*} [CommRing R□] [Algebra R R□] [IsLocalization M R□] (S□ : Type u_4) {N : Submonoid S} [CommRing S□] [Algebra S S□] [IsLocalization N S□] (hf : M ≤ Submonoid.comap f N), 0 ∉ N → ∀ [IsDomain S], Function.Injective ⇑f → Function.Injective ⇑(IsLocalization.map S□ f hf)
```

`IsLocalization.map_injective_of_injective'`

```
∀ {R : Type*} [CommRing R] (M : Submonoid R) (S : Type u_2) [CommRing S] {f : R →+* S} {R□ : Type*} [CommRing R□] [Algebra R R□] [IsLocalization M R□] (S□ : Type u_4) {N : Submonoid S} [CommRing S□] [Algebra S S□] [IsLocalization N S□] (hf : M ≤ Submonoid.comap f N), 0 ∉ N → ∀ [IsDomain S], Function.Injective ⇑f → Function.Injective ⇑(IsLocalization.map S□ f hf)
```

RingTheory · Localization · FractionRing

IsFractionRing.isFractionRing_iff_of_base_ringEquiv

∀ {R : Type*} [CommRing R] (S : Type u_2) [CommRing S] [Algebra R S] {P : Type*} [CommRing P] (h : R ≃+* P), IsFractionRing R S ↔ IsFractionRing P S

RingTheory · Localization · InvSubmonoid

IsLocalization.submonoid_map_le_is_unit

∀ {R : Type*} [CommRing R] (M : Submonoid R) (S : Type u_2) [CommRing S] [Algebra R S] [IsLocalization M S], Submonoid.map (algebraMap R S) M ≤ IsUnit.submonoid S

RingTheory · SurjectiveOnStalks

RingHom.SurjectiveOnStalks.tensorProductMap_id ?

signature not resolved

RingHom.SurjectiveOnStalks.tensorProductMap_id ?

signature not resolved

Ring → Semiring (8, magnitude 2) †

RingTheory · Ideal · Quotient · Basic

Ideal.Quotient.map_pi ?

∀ {ι : Type u_1} {ι' : Type u_2} {R : Type*} [Ring R] (I : Ideal R) [I.IsTwoSided] [Finite ι] (x : ι → R), (∀ (i : ι), x i ∈ I) → ∀ (f : (ι → R) →□ [R] ι' → R) (i : ι'), f x i ∈ I

RingTheory · IsAdjoinRoot

IsAdjoinRoot.mem_ker_map

∀ {R S : Type*} [CommRing R] [Ring S] {f : Polynomial R} [Algebra R S] (h : IsAdjoinRoot S f) {p : Polynomial R}, p ∈ RingHom.ker h.map ↔ f □ p

RingTheory · Noetherian · Basic

Submodule.FG.of_le_of_isNoetherian

∀ {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] {S T : Submodule R M} [IsNoetherian R ↥T], S ≤ T → S.FG

RingTheory · Polynomial · Basic

Ideal.polynomial_not_isField ‡

∀ (R : Type u_1) [Ring R], ¬IsField (Polynomial R)

RingTheory · Valuation · ValuativeRel · Basic

ValuativeRel.mul_vlt_mul

∀ {R : Type*} [Ring R] [ValuativeRel R] {a b c d : R}, a <ᵥ b → c <ᵥ d → a * c <ᵥ b * d

ValuativeRel.mul_vlt_mul_of_vle_of_vlt

∀ {R : Type*} [Ring R] [ValuativeRel R] {a b c d : R}, a ≤ᵥ b → c <ᵥ d → 0 <ᵥ a → a * c <ᵥ b * d

ValuativeRel.mul_vlt_mul_of_vlt_of_vle

∀ {R : Type*} [Ring R] [ValuativeRel R] {a b c d : R}, a <ᵥ b → c ≤ᵥ d → 0 <ᵥ d → a * c <ᵥ b * d

ValuativeRel.pow_vle_pow

∀ {R : Type*} [Ring R] [ValuativeRel R] {a b : R}, a ≤ᵥ b → ∀ (n : ℕ), a ^ n ≤ᵥ b ^ n

AddCommGroup → AddCommMonoid (4, magnitude 2) †

RingTheory · AdicCompletion · Functoriality

AdicCompletion.exists_smodEq_pow_add_one_smul ‡

∀ {R : Type*} [CommRing R] {I : Ideal R} {M : Type*} [AddCommGroup M] [Module R M] {N : Type*} [AddCommGroup N] [Module R N] {f : M →□ [R] N}, Function.Surjective ⇑((I • □).mkQ ∘□ f) → ∀ {y : N} {n : ℕ}, y ∈ I ^ n • □ → ∃ x ∈ I ^ n • □ , f x ≡ y [SMOD I ^ (n + 1) • □]

RingTheory · Localization · Module

LinearIndependent.iff_fractionRing

∀ (R : Type u_1) (K : Type u_2) [CommRing R] [CommRing K] [Algebra R K] [IsFractionRing R K] {V : Type*} [AddCommGroup V] [Module R V] [Module K V] [IsScalarTower R K V] {ι : Type u_4} {b : ι → V}, LinearIndependent R b ↔ LinearIndependent K b

RingTheory · Noetherian · Basic

Submodule.FG.of_le_of_isNoetherian

∀ {R M : Type*} [Ring R] [AddCommGroup M] [Module R M] {S T : Submodule R M} [IsNoetherian R ↥T], S ≤ T → S.FG

RingTheory · Regular · LinearMap

IsSMulRegular.linearMap_subsingleton_of_mem_annihilator

∀ {R M N : Type*} [CommRing R] [AddCommGroup M] [AddCommGroup N] [Module R M] [Module R N] {r : R}, IsSMulRegular M r → r ∈ Module.annihilator R N → Subsingleton (N →□ [R] M)

LinearOrder → PartialOrder (2, magnitude 2)

RingTheory · HahnSeries · Basic

HahnSeries.BddBelow_zero

```
∀ {Γ : Type u_1} {R : Type*} [Zero R] [LinearOrder Γ] [Nonempty Γ], BddBelow (Function.support 0)
```

RingTheory · HahnSeries · Lex

HahnSeries.leadingCoeff_pos_iff ‡

```
∀ {Γ : Type u_1} {R : Type*} [LinearOrder Γ] [Zero R] [LinearOrder R] {x : Lex (HahnSeries Γ R)}, 0 < (ofLex x).leadingCoeff ↔ 0 < x
```

Monoid → MulOneClass (2, magnitude 2)

RingTheory · Bialgebra · MonoidAlgebra

MonoidAlgebra.isGroupLikeElem_of

```
∀ {R A M : Type*} [CommSemiring R] [Semiring A] [Bialgebra R A] [Monoid M] (m : M), IsGroupLikeElem R ((MonoidAlgebra.of A M) m)
```

RingTheory · Congruence · BigOperators

RingCon.listProd

```
∀ {ι : Type u_1} {S : Type*} [Add S] [Monoid S] (t : RingCon S) (l : List ι) {f g : ι → S}, (∀ i ∈ l, t (f i) (g i)) → t (List.map f l).prod (List.map g l).prod
```

CommRing → Ring (9, magnitude 1)

RingTheory · Bezout

IsBezout.iff_span_pair_isPrincipal

```
∀ {R : Type*} [CommRing R], IsBezout R ↔ ∀ (x y : R), Submodule.IsPrincipal (Ideal.span {x, y})
```

RingTheory · Ideal · GoingUp

Ideal.coeff_zero_mem_comap_of_root_mem_of_eval_mem

```
∀ {R : Type*} [CommRing R] {S : Type*} [CommRing S] {f : R →+* S} {I : Ideal S} {r : S}, r ∈ I → ∀ {p : Polynomial R}, Polynomial.eval₂ f r p ∈ I → p.coeff 0 ∈ Ideal.comap f I
```

Ideal.mem_of_one_mem

```
∀ {S : Type*} [CommRing S] {I : Ideal S}, 1 ∈ I → ∀ (x : S), x ∈ I
```

RingTheory · Ideal · Quotient · Over

Ideal.ncard_primesOver_lt_of_not_le

```
∀ {R S T : Type*} [CommRing R] [CommRing S] [CommRing T] [Algebra R S] [Algebra R T] (f : S →ₐ[R] T), Function.Surjective ⇑f → ∀ (P : Ideal R) (P' : Ideal S) [P'.IsPrime] [P'.LiesOver P], ¬RingHom.ker f.toRingHom ≤ P' → (P.primesOver S).Finite → (P.primesOver T).ncard < (P.primesOver S).ncard
```

RingTheory · LocalRing · Basic

IsLocalRing.isUnit_or_isUnit_one_sub_self

```
∀ {R : Type*} [CommRing R] [IsLocalRing R] (a : R), IsUnit a ∨ IsUnit (1 - a)
```

RingTheory · Polynomial · Cyclotomic · Eval

Polynomial.cyclotomic_neg_one_pos ?

signature not resolved

RingTheory · Polynomial · Cyclotomic · Roots

Polynomial.cyclotomic_injective ‡

```
∀ {R : Type*} [CommRing R] [CharZero R], Function.Injective fun n => Polynomial.cyclotomic n R
```

RingTheory · Polynomial · DegreeLT

Polynomial.taylor_mem_degreeLT

```
∀ {R : Type*} [CommRing R] {r : R} {n : ℕ} {f : Polynomial R}, (Polynomial.taylor r) f ∈ Polynomial.degreeLT R n ↔ f ∈ Polynomial.degreeLT R n
```

RingTheory · Regular · LinearMap

IsSMulRegular.linearMap_subsingleton_of_mem_annihilator

```
∀ {R M N : Type*} [CommRing R] [AddCommGroup M] [AddCommGroup N] [Module R M] [Module R N] {r : R}, IsSMulRegular M r → r ∈ Module.annihilator R N → Subsingleton (N →ₗ[R] M)
```

CommSemiring → Semiring (6, magnitude 1)

RingTheory · Coprime · Basic

IsRelPrime.of_add_mul_left_left

```
∀ {R : Type*} [CommSemiring R] {x y z : R}, IsRelPrime (x + y * z) y → IsRelPrime x y
```

RingTheory · Ideal · Maps

Ideal.disjoint_map_primeCompl_iff_comap_le ‡

```
∀ {R : Type*} [CommSemiring R] {S : Type*} [Semiring S] {f : R →+* S} {p : Ideal R} {I : Ideal S} [p.IsPrime], Disjoint ↑I ↑(Submonoid.map f p.primeCompl) ↔ Ideal.comap f I ≤ p
```

RingTheory · Ideal · Operations

Ideal.primeCompl_le_nonZeroDivisors

∀ {R : Type*} [CommSemiring R] [NoZeroDivisors R] (P : Ideal R) [P.IsPrime], P.primeCompl ≤ nonZeroDivisors R

RingTheory · LocalRing · RingHom · Basic

IsLocalRing.of_surjective

∀ {R S : Type*} [CommSemiring R] [IsLocalRing R] [Semiring S] [Nontrivial S] (f : R →+* S) [IsLocalHom f], Function.Surjective ⇑f → IsLocalRing S

RingTheory · Polynomial · Eisenstein · Basic

Polynomial.IsEisensteinAt._root_.Polynomial.Monic.leadingCoeff_notMem

∀ {R : Type*} [CommSemiring R] {□ : Ideal R} {f : R[X]}, f.Monic → □ ≠ □ → f.leadingCoeff ∉ □

RingTheory · PowerSeries · PiTopology

PowerSeries.WithPiTopology.tendsto_trunc_atTop

∀ (R : Type u_1) [TopologicalSpace R] [CommSemiring R] (f : PowerSeries R), Filter.Tendsto (fun d => ↑((PowerSeries.trunc d) f)) Filter.atTop (nhds f)

NonUnitalRing → NonUnitalNonAssocRing (2, magnitude 1)

RingTheory · TwoSidedIdeal · Operations

TwoSidedIdeal.set_mul_subset

∀ {R : Type*} [NonUnitalRing R] {s : Set R} {I : TwoSidedIdeal R}, s ⊆ ↑I → ∀ (t : Set R), t * s ⊆ ↑I

TwoSidedIdeal.subset_mul_set

∀ {R : Type*} [NonUnitalRing R] {s : Set R} {I : TwoSidedIdeal R}, s ⊆ ↑I → ∀ (t : Set R), s * t ⊆ ↑I

AddCommMonoid → AddMonoid (1, magnitude 1)

RingTheory · GradedAlgebra · FiniteType

GradedAlgebra.exists_finset_adjoin_eq_top_and_homogeneous

∀ {S : Type*} {σ : Type u_2} {ι : Type u_3} [DecidableEq ι] [AddCommMonoid ι] [CommRing S] [SetLike σ S] [AddSubgroupClass σ S] (□ : ι → σ) [GradedRing □] [Algebra.FiniteType (↥(□ 0)) S], ∃ s, Algebra.adjoin ↥(□ 0) ↑s = □ ∧ ∀ i ∈ s, SetLike.IsHomogeneousElem □ i

CommMonoid → Monoid (1, magnitude 1)

RingTheory · UniqueFactorizationDomain · Multiplicity

UniqueFactorizationMonoid.hasFiniteMulSupport_fun_pow_multiplicity

∀ {R : Type*} [CommMonoidWithZero R] [UniqueFactorizationMonoid R] {α : Type u_2} {M : Type*} [CommMonoid M] [Subsingleton Rˣ] (f : α → M) {g : α → R}, Function.Injective g → (∀ (s : α), Irreducible (g s)) → ∀ {r : R}, r ≠ 0 → Function.HasFiniteMulSupport fun s => f s ^ multiplicity (g s) r

IsDomain → Nontrivial (2, unranked)

RingTheory · FractionalIdeal · Operations

FractionalIdeal.isPrincipal_of_isPrincipal_num ‡

∀ {R : Type*} [CommRing R] [IsDomain R] (I : FractionalIdeal (nonZeroDivisors R) (FractionRing R)), Submodule.IsPrincipal I.num → (↑I).IsPrincipal

RingTheory · Polynomial · GaussLemma

Polynomial.IsPrimitive.mul_map_mem_lifts_iff ‡

∀ {R : Type*} [CommRing R] {K : Type*} [Field K] [Algebra R K] [IsFractionRing R K] [IsDomain R] [IsGCDMonoid R] {f : Polynomial R}, f.IsPrimitive → ∀ {g : Polynomial K}, g * Polynomial.map (algebraMap R K) f ∈ Polynomial.lifts (algebraMap R K) ↔ g ∈ Polynomial.lifts (algebraMap R K)

ValuationRing → PreValuationRing (1, unranked)

RingTheory · Valuation · ValuationRing

ValuationRing.le_total

∀ (A : Type u_1) [CommRing A] (K : Type u_2) [Field K] [Algebra A K] [IsDomain A] [ValuationRing A] [IsFractionRing A K] (a b : ValuationRing.ValueGroup A K), a ≤ b ∨ b ≤ a

Topology

42 survivors in 18 families; 2 give up subtraction or division (†).

stated over	holds over	n	magnitude
`PseudoEMetricSpace`	`EDist`	3	5
`NormedField`	`NormedRing`	1	3
`LinearOrder`	`PartialOrder`	5	2
`Ring` †	`Semiring`	2	2
`Monoid`	`MulOneClass`	1	2
`PartialOrder`	`Preorder`	6	1
`EMetricSpace`	`PseudoEMetricSpace`	4	1
`CommMonoid`	`Monoid`	3	1
`MetricSpace`	`PseudoMetricSpace`	3	1
`MulOneClass`	`MulOne`	3	1
`T3Space`	`RegularSpace`	3	1
`CommGroup`	`Group`	2	1
`AddCommMonoid`	`AddMonoid`	1	1
`CommRing`	`Ring`	1	1
`CommSemiring`	`Semiring`	1	1
`Field`	`DivisionRing`	1	1
`NormedAddCommGroup`	`SeminormedAddCommGroup`	1	1
`IrreducibleSpace`	`PreirreducibleSpace`	1	-1

PseudoEMetricSpace → EDist (3, magnitude 5)

Topology · EMetricSpace · Defs

`Metric.eball_subset_eball`

```
∀ {α : Type u_1} [PseudoEMetricSpace α] {x : α} {ε₁ ε₂ : ENNReal}, ε₁ ≤ ε₂ → Metric.eball x ε₁ ⊆ Metric.eball x ε₂
```

Topology · EMetricSpace · PairReduction

`PairReduction.exists_radius_le`

```
∀ {T : Type*} [PseudoEMetricSpace T] {a : ENNReal} (t : T) (V : Finset T), 1 < a → ∀ (c : ENNReal), ∃ r, 1 ≤ r ∧ ↑{x ∈ V | edist t x ≤ ↑r * c}.card ≤ a ^ r
```

Topology · EMetricSpace · Pi

`edist_pi_const_le`

```
∀ {α : Type u_1} {β : Type u_2} [PseudoEMetricSpace α] [Fintype β] (a b : α), (edist (fun x => a) fun x => b) ≤ edist a b
```

NormedField → NormedRing (1, magnitude 3)

Topology · MetricSpace · CauSeqFilter

`CauchySeq.isCauSeq`

```
∀ {β : Type u_1} [NormedField β] {f : ℕ → β}, CauchySeq f → IsCauSeq norm f
```

LinearOrder → PartialOrder (5, magnitude 2)

Topology · EMetricSpace · BoundedVariation

`eVariationOn.nonempty_monotone_mem`

```
∀ {α : Type u_1} [LinearOrder α] {s : Set α}, s.Nonempty → Nonempty { u // Monotone u ∧ ∀ (i : ℕ), u i ∈ s }
```

Topology · Instances · AddCircle · Defs

`AddCircle.not_isOfFinAddOrder_iff_forall_rat_ne_div`

```
∀ {□ : Type u_1} [Field □ ] {p : □ } [LinearOrder □ ] [IsStrictOrderedRing □ ] [hp : Fact (0 < p)] {a : □ }, ¬IsOfFinAddOrder ↑a ↔ ∀ (q : ℚ), ↑q ≠ a / p
```

Topology · Instances · Matrix

`isClosed_setOfPred_blockTriangular`

```
∀ {m R : Type*} [TopologicalSpace R] {α : Type u_3} {b : m → α} [LinearOrder α] [Zero R] [T2Space R], IsClosed {M | M.BlockTriangular b}
```

Topology · Order · MonotoneContinuity

`StrictMonoOn.continuousWithinAt_right_of_exists_between`

```
∀ {α : Type u_1} {β : Type u_2} [LinearOrder α] [TopologicalSpace α] [OrderTopology α] [LinearOrder β] [TopologicalSpace β] [OrderTopology β] {f : α → β} {s : Set α} {a : α}, StrictMonoOn f s → s ∈ nhdsWithin a (Set.Ici a) → (∀ b > f a, ∃ c ∈ s, f c ∈ Set.Ioc (f a) b) → ContinuousWithinAt f (Set.Ici a) a
```

`continuousWithinAt_right_of_monotoneOn_of_exists_between`

```
∀ {α : Type u_1} {β : Type u_2} [LinearOrder α] [TopologicalSpace α] [OrderTopology α] [LinearOrder β] [TopologicalSpace β]
```

```
[OrderTopology β] {f : α → β} {s : Set α} {a : α}, MonotoneOn f s → s ∈ nhdsWithin a (Set.Ici a) → (∀ b > f a, ∃ c ∈ s, f c ∈ Set.Ioo (f a) b) → ContinuousWithinAt f (Set.Ici a) a
```

Ring → Semiring (2, magnitude 2) †

Topology · Algebra · UniformMulAction

`UniformContinuous.const_mul'`

```
∀ {R : Type*} {β : Type u_2} [Ring R] [UniformSpace R] [UniformSpace β] [UniformContinuousConstSMul R R] {f : β → R}, UniformContinuous f → ∀ (a : R), UniformContinuous fun x => a * f x
```

`UniformContinuous.mul_const'`

```
∀ {R : Type*} {β : Type u_2} [Ring R] [UniformSpace R] [UniformSpace β] [UniformContinuousConstSMul Rᵐᵒᵖ R] {f : β → R}, UniformContinuous f → ∀ (a : R), UniformContinuous fun x => f x * a
```

Monoid → MulOneClass (1, magnitude 2)

Topology · Algebra · Monoid

`Submonoid.top_closure_mul_self_subset`

```
∀ {M : Type*} [TopologicalSpace M] [Monoid M] [SeparatelyContinuousMul M] (s : Submonoid M), closure ↑s * closure ↑s ⊆ closure ↑s
```

PartialOrder → Preorder (6, magnitude 1)

Topology · Algebra · Group · Basic

`tendsto_inv_nhdsGE`

```
∀ {H : Type*} [TopologicalSpace H] [CommGroup H] [PartialOrder H] [IsOrderedMonoid H] [ContinuousInv H] {a : H}, Filter.Tendsto Inv.inv (nhdsWithin a (Set.Ici a)) (nhdsWithin a⁻¹ (Set.Iic a⁻¹))
```

`tendsto_inv_nhdsLE`

```
∀ {H : Type*} [TopologicalSpace H] [CommGroup H] [PartialOrder H] [IsOrderedMonoid H] [ContinuousInv H] {a : H}, Filter.Tendsto Inv.inv (nhdsWithin a (Set.Iic a)) (nhdsWithin a⁻¹ (Set.Ici a⁻¹))
```

Topology · Algebra · InfiniteSum · Order

`Multipliable.tprod_subtype_le`

```
∀ {κ : Type u_1} {γ : Type u_2} [CommGroup γ] [PartialOrder γ] [IsOrderedMonoid γ] [UniformSpace γ] [IsUniformGroup γ] [OrderClosedTopology γ] [CompleteSpace γ] (f : κ → γ) (β : Set κ), (∀ (a : κ), 1 ≤ f a) → Multipliable f → ∏' (b : ↑β), f ↑b ≤ ∏' (a : κ), f a
```

Topology · Instances · Sign

`continuousAt_sign_of_neg`

```
∀ {α : Type u_1} [Zero α] [TopologicalSpace α] [PartialOrder α] [DecidableLT α] [OrderTopology α] {a : α}, a < 0 → ContinuousAt (⇑SignType.sign) a
```

`continuousAt_sign_of_pos`

```
∀ {α : Type u_1} [Zero α] [TopologicalSpace α] [PartialOrder α] [DecidableLT α] [OrderTopology α] {a : α}, 0 < a → ContinuousAt (⇑SignType.sign) a
```

Topology · Order · Filter

`Filter.tendsto_nhds_atTop`

```
∀ {X : Type*} [TopologicalSpace X] [PartialOrder X] [OrderTopology X] [NoMaxOrder X], Filter.Tendsto nhds Filter.atTop (nhds Filter.atTop)
```

EMetricSpace → PseudoEMetricSpace (4, magnitude 1)

Topology · Instances · ENNReal · Lemmas

`edist_ne_top_of_mem_ball`

```
∀ {β : Type u_1} [EMetricSpace β] {a : β} {r : ENNReal} (x y : ↑(Metric.eball a r)), edist ↑x ↑y ≠ □
```

Topology · MetricSpace · HolderNorm

`MemHolder.holderWith`

```
∀ {X Y : Type*} [MetricSpace X] [EMetricSpace Y] {r : NNReal} {f : X → Y}, MemHolder r f → HolderWith (nnHolderNorm r f) r f
```

Topology · MetricSpace · PartitionOfUnity

`Metric.eventually_nhds_zero_forall_closedEBall_subset`

```
∀ {ι : Type u_1} {X : Type*} [EMetricSpace X] {K U : ι → Set X}, (∀ (i : ι), IsClosed (K i)) → (∀ (i : ι), IsOpen (U i)) → (∀ (i : ι), K i ⊆ U i) → LocallyFinite K → ∀ (x : X), ∀ᶠ (p : ENNReal × X) in nhds 0 ×ˢ nhds x, ∀ (i : ι), p.2 ∈ K i → Metric.closedEBall p.2 p.1 ⊆ U i
```

`Metric.exists_forall_closedEBall_subset_aux₂` ‡

```
∀ {ι : Type u_1} {X : Type*} [EMetricSpace X] {K U : ι → Set X} (y : X), Convex ℝ (Set.Ioi 0 ∩ ENNReal.ofReal ⁻¹' □ i, □ (_ : y ∈ K i), {r | Metric.closedEBall y r ⊆ U i})
```

CommMonoid → Monoid (3, magnitude 1)

Topology · Algebra · InfiniteSum · Constructions

`HasProd.sum`

```
∀ {α : Type u_1} {β : Type u_2} {M : Type*} [CommMonoid M] [TopologicalSpace M] [ContinuousMul M] {f : α ⊕ β → M} {a b : M}, HasProd (f ∘ Sum.inl) a → HasProd (f ∘ Sum.inr) b → HasProd f (a * b)
```

`Multipliable.sum`

```
∀ {α : Type u_1} {β : Type u_2} {M : Type*} [CommMonoid M] [TopologicalSpace M] [ContinuousMul M] (f : α ⊕ β → M), Multipliable (f ∘ Sum.inl) → Multipliable (f ∘ Sum.inr) → Multipliable f
```

Topology · Algebra · Monoid

`Submonoid.mem_nhds_one`

```
∀ {M : Type*} [TopologicalSpace M] [CommMonoid M] (S : Submonoid M), IsOpen ↑S → ↑S ∈ nhds 1
```

MetricSpace → PseudoMetricSpace (3, magnitude 1)

Topology · MetricSpace · Perfect

`Perfect.small_diam_aux` ?

signature not resolved

Topology · MetricSpace · ShrinkingLemma

`exists_subset_iUnion_ball_radius_lt` ‡

```
∀ {α : Type u_1} {ι : Type u_2} [MetricSpace α] [ProperSpace α] {c : ι → α} {s : Set α} {r : ι → ℝ}, IsClosed s → (∀ x ∈ s, {i | x ∈ Metric.ball (c i) (r i)}.Finite) → s ⊆ ⋃ i, Metric.ball (c i) (r i) → ∃ r', s ⊆ ⋃ i, Metric.ball (c i) (r' i) ∧ ∀ (i : ι), r' i < r i
```

`exists_subset_iUnion_ball_radius_pos_lt` ‡

```
∀ {α : Type u_1} {ι : Type u_2} [MetricSpace α] [ProperSpace α] {c : ι → α} {s : Set α} {r : ι → ℝ}, (∀ (i : ι), 0 < r i) → IsClosed s → (∀ x ∈ s, {i | x ∈ Metric.ball (c i) (r i)}.Finite) → s ⊆ ⋃ i, Metric.ball (c i) (r i) → ∃ r', s ⊆ ⋃ i, Metric.ball (c i) (r' i) ∧ ∀ (i : ι), r' i ∈ Set.Ioo 0 (r i)
```

MulOneClass → MulOne (3, magnitude 1)

Topology · ApproximateUnit

`Filter.IsApproximateUnit.mono`

```
∀ {α : Type u_1} [TopologicalSpace α] [MulOneClass α] {l l' : Filter α}, l.IsApproximateUnit → l' ≤ l → ∀ [hl' : l'.NeBot], l'.IsApproximateUnit
```

Topology · Connected · PathConnected

`Joined.listProd`

```
∀ {M : Type*} [MulOneClass M] [TopologicalSpace M] [ContinuousMul M] {l l' : List M}, List.Forall₂ Joined l l' → Joined l.prod l'.prod
```

Topology · List

`List.tendsto_prod`

```
∀ {α : Type u_1} [TopologicalSpace α] [MulOneClass α] [ContinuousMul α] {l : List α}, Filter.Tendsto List.prod (nhds l) (nhds l.prod)
```

T3Space → RegularSpace (3, magnitude 1)

Topology · Algebra · InfiniteSum · Ring

`summable_sum_mul_antidiagonal_of_summable_mul`

```
∀ {α : Type u_1} {A : Type*} [AddCommMonoid A] [Finset.HasAntidiagonal A] [TopologicalSpace α] [NonUnitalNonAssocSemiring α] {f g : A → α} [T3Space α] [IsTopologicalSemiring α], (Summable fun x => f x.1 * g x.2) → Summable fun n => ∑ kl ∈ Finset.HasAntidiagonal.antidiagonal n, f kl.1 * g kl.2
```

Topology · Order · LeftRightLim

`continuousWithinAt_leftLim_Iic`

```
∀ {α : Type u_1} {β : Type u_2} [LinearOrder α] [TopologicalSpace β] [TopologicalSpace α] [OrderTopology α] [T3Space β] {f : α → β} {a : α}, Filter.Tendsto f (nhdsWithin a (Set.Iio a)) (nhds (Function.leftLim f a)) → ContinuousWithinAt (Function.leftLim f) (Set.Iic a) a
```

`tendsto_atTop_of_mapClusterPt`

```
∀ {α : Type u_1} {β : Type u_2} [LinearOrder α] [TopologicalSpace β] [TopologicalSpace α] [OrderTopology α] [T3Space β] [NoTopOrder α] {f g : α → β} {b : β}, Filter.Tendsto f Filter.atTop (nhds b) → (∀ᶠ (x : α) in Filter.atTop, MapClusterPt (g x) (nhds x) f) → Filter.Tendsto g Filter.atTop (nhds b)
```

CommGroup → Group (2, magnitude 1)

Topology · Algebra · InfiniteSum · Group

`Multipliable.hasFiniteMulSupport_of_discreteTopology`

∀ {α : Type u_1} [CommGroup α] [TopologicalSpace α] [DiscreteTopology α] {β : Type u_2} (f : β → α), Multipliable f → Function.HasFiniteMulSupport f

Topology · Algebra · InfiniteSum · Nonarchimedean

`NonarchimedeanGroup.cauchySeq_of_tendsto_div_nhds_one`

∀ {G : Type*} [CommGroup G] [UniformSpace G] [IsUniformGroup G] [NonarchimedeanGroup G] {f : ℕ → G}, Filter.Tendsto (fun n => f (n + 1) / f n) Filter.atTop (nhds 1) → CauchySeq f

AddCommMonoid → AddMonoid (1, magnitude 1)

Topology · Algebra · InfiniteSum · Ring

`summable_mul_prod_iff_summable_mul_sigma_antidiagonal`

∀ {α : Type u_1} {A : Type*} [AddCommMonoid A] [Finset.HasAntidiagonal A] [TopologicalSpace α] [NonUnitalNonAssocSemiring α] {f g : A → α}, (Summable fun x => f x.1 * g x.2) ↔ Summable fun x => f (↑x.snd).1 * g (↑x.snd).2

CommRing → Ring (1, magnitude 1)

Topology · Algebra · OpenSubgroup

`Submodule.isOpen_mono`

∀ {R M : Type*} [CommRing R] [AddCommGroup M] [TopologicalSpace M] [IsTopologicalAddGroup M] [Module R M] {U P : Submodule R M}, U ≤ P → IsOpen ↑U → IsOpen ↑P

CommSemiring → Semiring (1, magnitude 1)

Topology · Algebra · Module · Compact

`Submodule.isCompact_of_fg`

∀ {R M : Type*} [CommSemiring R] [TopologicalSpace R] [AddCommMonoid M] [Module R M] [TopologicalSpace M] [ContinuousAdd M] [ContinuousSMul R M] [CompactSpace R] {N : Submodule R M}, N.FG → IsCompact ↑N

Field → DivisionRing (1, magnitude 1)

Topology · Algebra · Valued · ValuedField

`Valued.valuation_isClosedMap`

∀ {K : Type*} [Field K] {Γ₀ : Type u_2} [LinearOrderedCommGroupWithZero Γ₀] [hv : Valued K Γ₀], IsClosedMap ⇑Valued.v.restrict

NormedAddCommGroup → SeminormedAddCommGroup (1, magnitude 1)

Topology · Algebra · InfiniteSum · ConditionalInt

`SummationFilter._root_.Summable.tendsto_zero_of_even_summable_symmetricIcc` ‡

∀ {F : Type*} [NormedAddCommGroup F] [NormSMulClass ℤ F] {f : ℤ → F}, Summable f (symmetricIcc ℤ) → Function.Even f → Filter.Tendsto f Filter.atTop (nhds 0)

IrreducibleSpace → PreirreducibleSpace (1, unranked)

Topology · Irreducible

`Topology.IsOpenEmbedding.irreducibleSpace`

∀ {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {f : Y → X}, Topology.IsOpenEmbedding f → ∀ [IrreducibleSpace X] [Nonempty Y], IrreducibleSpace Y

Index of declarations

Ideal.ncard_primesOver_lt_of_not_le RingTheory
Ideal.polynomial_not_isField RingTheory
Ideal.primeCompl_le_nonZeroDivisors RingTheory
Int.cast_one_le_of_pos Algebra
Int.sub_floor_div_mul_lt Algebra
Int.sub_floor_div_mul_nonneg Algebra
IsAbsoluteValue.abv_sub_le Algebra
IsAdjoinRoot.mem_ker_map RingTheory
IsAntichain.of_monotoneOn_strictAntiOn Order
IsAntichain.of_strictMonoOn_antitoneOn Order
IsBezout.iff_span_pair_isPrincipal RingTheory
IsBoundedLinearMap.hasFDerivAtFilter Analysis
IsCyclotomicExtension.splits_X_pow_sub_one NumberTheory
IsForwardInvariant.isInvariant Dynamics
IsFractionRing.charP_of_isFractionRing Algebra
IsFractionRing.isFractionRing_iff_of_base_ringEquiv RingTheory
IsLocalRing.isUnit_or_isUnit_one_sub_self RingTheory
IsLocalRing.of_surjective RingTheory
IsLocalization.AtPrime.not_isField RingTheory
IsLocalization.map_injective_of_injective' RingTheory
IsLocalization.submonoid_map_le_is_unit RingTheory
IsMulTorsion.exponentExists GroupTheory
IsMulTorsion.of_surjective GroupTheory
IsMulTorsionFree.orderOf_le_one GroupTheory
IsMulTorsionFree.pow_right_injective Algebra
IsNoetherian.finite_basis_index FieldTheory
IsNonarchimedean.iSup_abv_linearMap_apply_le NumberTheory
IsNonarchimedean.multiset_image_add Algebra
IsNonarchimedean.nsmul_le Algebra
IsOpen.balancedHull Analysis
IsOrderedModule.of_algebraMap_mono Algebra
IsPGroup.ker_isPGroup_of_injective GroupTheory
IsPrimitiveRoot.associated_sub_one_pow_sub_one_of_coprime RingTheory
IsRelPrime.isUnit_of_dvd Algebra
IsRelPrime.of_add_mul_left_left RingTheory
IsRelPrime.of_mul_left_left Algebra
IsRelPrime.symm Algebra
IsSMulRegular.linearMap_subsingleton_of_mem_annihilator RingTheory
IsSMulRegular.mul_iff Algebra
IsScalarTower.of_commMonoid Algebra
IsSelfAdjoint.div Algebra
IsSelfAdjoint.map' Algebra
IsSelfAdjoint.mul Algebra
IsSepClosed.splits_codomain FieldTheory
IsSquare.map Algebra
IsStarProjection.mul Algebra
IsStrictlyPositive.of_subsingleton Algebra
IsStrictlyPositive.smul Algebra
IsStronglyAtomic.of_wellFounded_lt Order
IsSumSq.sum_sq Algebra
Joined.listProd Topology
LinearEquiv.isUnit_det LinearAlgebra
LinearIndepOn.singleton LinearAlgebra
LinearIndependent.iff_fractionRing RingTheory
LinearIndependent.linearIndependent_algebraMap_comp_aux LinearAlgebra
LinearIndependent.neg LinearAlgebra
LinearIndependent.of_subsingleton LinearAlgebra
LinearMap.IsAdjointPair.smul LinearAlgebra
LinearMap.commute_mulLeft_right Algebra
LinearMap.disjoint_ker_of_nondegenerate_restrict LinearAlgebra
LinearMap.injective_codRestrict_iff Algebra
LinearMap.isNilpotent_trace_of_isNilpotent LinearAlgebra
LinearMap.isPairSelfAdjoint_equiv LinearAlgebra
LinearMap.lsmul_injective LinearAlgebra
LinearMap.mapsTo_biSup_of_mapsTo Algebra
List.Forall$_2$.prod_le_prod' Algebra
List.Sublist.prod_le_prod' Algebra
List.le_prod_nonempty_of_submultiplicative_on_pred Algebra
List.le_prod_of_mem Algebra
List.le_prod_of_submultiplicative_on_pred Algebra
List.length_pos_of_prod_ne_one Algebra
List.one_le_prod Algebra
List.one_le_prod_of_one_le Algebra
List.prod_hom_rel Algebra
List.prod_nonneg Algebra
List.sum_le_foldr_max Algebra
List.tendsto_prod Topology
Localization.map_isUnit_of_le RingTheory
LowerSet.supIrred_Iic Order
Mathlib.Meta.NormNum.IsRat.natCeil Data
Mathlib.Meta.NormNum.IsRat.natFloor Algebra
Mathlib.Meta.Positivity.div_ne_zero_of_ne_zero_of_pos Algebra
Mathlib.Meta.Positivity.div_ne_zero_of_pos_of_ne_zero Algebra
Matrix.IsDetpBalanced.trans LinearAlgebra
Matrix.IsHermitian.commute_iff LinearAlgebra
Matrix.IsHermitian.hadamard LinearAlgebra
Matrix.PosDef._root_.LinearMap.BilinForm.
posDef_toQuadraticMap_iff_matrix LinearAlgebra
Matrix.blockTriangular_algebraMap LinearAlgebra
Matrix.isHermitian_conjTranspose_mul_self LinearAlgebra
Matrix.isHermitian_mul_conjTranspose_self LinearAlgebra
Matrix.isSymm_add_transpose_self LinearAlgebra
Matrix.isSymm_transpose_add_self LinearAlgebra
Matrix.mul_left_injective_iff_vecMul_injective Data
Matrix.mul_right_injective_iff_mulVec_injective Data
Matrix.posSemidef_opNorm_smul_gram_sub_gram Analysis
MeasureTheory.AECover.integrable_of_lintegral_enorm_bounded MeasureTheory
MeasureTheory.ExistsSeqTendstoAe.exists_nat_measure_lt_two_inv MeasureTheory
MeasureTheory.IsFundamentalDomain.hasFundamentalDomain MeasureTheory
MeasureTheory.IsStronglyProgressive.div' Probability
MeasureTheory.IsStronglyProgressive.inv Probability
MeasureTheory.Lp.cauchy_tendsto_of_tendsto MeasureTheory
MeasureTheory.Measure.measurePreserving_div_left MeasureTheory
MeasureTheory.Measure.measurePreserving_mul_right_inv MeasureTheory
MeasureTheory.SimpleFunc.nnnorm_approxOn_le MeasureTheory
MeasureTheory.SimpleFunc.norm_approxOn_y$_0$_le MeasureTheory
MeasureTheory.SimpleFunc.norm_approxOn_zero_le MeasureTheory
MeasureTheory.StronglyMeasurable.leOnePart MeasureTheory
MeasureTheory.StronglyMeasurable.oneLePart MeasureTheory
MeasureTheory.measurableSet_predictable_Ioi_prod Probability
MeasureTheory.measurableSet_predictable_singleton_bot_prod Probability
MeasureTheory.measurePreserving_div_right MeasureTheory
MeasureTheory.norm_resolvent_le_inv_infDist_support MeasureTheory
MeasureTheory.tendsto_indicator_ge MeasureTheory
MemHolder.holderWith Topology
Metric.contractibleSpace_ball Analysis
Metric.contractibleSpace_closedBall Analysis
Metric.contractibleSpace_eball Analysis
Metric.eball_subset_eball Topology
Metric.eventually_nhds_zero_forall_closedEBall_subset Topology
Metric.exists_forall_closedEBall_subset_aux$_2$ Topology
Metric.isPathConnected_ball Analysis
Metric.isPathConnected_closedBall Analysis
Metric.isPathConnected_eball Analysis
Metric.isPreconnected_ball Analysis
Metric.isPreconnected_closedBall Analysis
Metric.isPreconnected_closedEBall Analysis
Metric.isPreconnected_eball Analysis
Module.AEval.isTorsion_of_finiteDimensional Algebra
Module.Basis.algebraMap_injective RingTheory
Module.End.HasUnifEigenvalue.isNilpotent_of_isNilpotent LinearAlgebra
Module.End.commute_id_right Algebra
Module.End.isRoot_of_hasEigenvalue LinearAlgebra
ModuleCat.restrictScalars_map_exact Algebra
MonoidAlgebra.isGroupLikeElem_of RingTheory
MonoidHom.CompTriple.comp Algebra
MonoidHom.fst_mono Algebra
MonoidHom.inl_mono Algebra
MonoidHom.inl_strictMono Algebra
MonoidHom.inr_mono Algebra
MonoidHom.inr_strictMono Algebra
MonoidHom.snd_mono Algebra
Monotone.cauchySeq_series_mul_of_tendsto_zero_of_bounded Analysis
Monotone.mul_strictMono Algebra
MonotoneOn.exists_monotone_extension Order
MulAction.IsBlock.image GroupTheory
MulAction.isBlock_subgroup GroupTheory
MulAction.isBlock_subgroup' GroupTheory

MulAction.isBlock_subtypeVal GroupTheory
MulChar.apply_mem_rootsOfUnity NumberTheory
MulChar.mulEquiv_units NumberTheory
MulChar.orderOf_dvd_card_sub_one NumberTheory
MulChar.restrictHom_surjective NumberTheory
MulEquiv.comp_left_injective Algebra
MulEquiv.comp_right_injective Algebra
Multipliable.hasFiniteMulSupport_of_discreteTopology Topology
Multipliable.sum Topology
Multipliable.tprod_subtype_le Topology
Multiset.Ico_disjoint_Ico Order
Multiset.le_prod_of_submultiplicative_on_pred_of_nonneg Algebra
Multiset.map_noncommProd_aux Data
Multiset.mulSupport_fun_pow_count_subset Algebra
Nat.Partition.tendsto_order_genFun_term_atTop_nhds_top Combinatorics
Nat.cast_le_pow_sub_div_sub Algebra
Nat.ceil_mono Algebra
Nat.floor_le Algebra
Nat.le_ceil Algebra
Nat.mono_cast Data
Ne.isUnit_C NumberTheory
NonUnitalSubalgebra._root_.Set.smul_mem_center Algebra
NonarchAddGroupSeminorm.add_bddBelow_range_add Analysis
NonarchimedeanGroup.cauchySeq_of_tendsto_div_nhds_one Topology
NonemptyInterval.one_mem_one Algebra
NonemptyInterval.subset_coe_map Order
NormedAddCommGroup.completeSpace_of_summable_imp_tendsto Analysis
NormedAddCommGroup.summable_imp_tendsto_of_complete Analysis
Order.coheight_pos_of_lt_top Order
Order.not_isSuccLimit_add_one Algebra
Order.not_isSuccPrelimit_add_one Algebra
PairReduction.exists_radius_le Topology
Pentagonal.tendsto_order_neg_X_pow Combinatorics
Perfect.small_diam_aux Topology
PointedCone.IsFaceOf.lineal_le Geometry
PointedCone.IsFaceOf.prod Geometry
PointedCone.basis_coord_mem_dual Analysis
PointedCone.smul_mem_iff Geometry
Polynomial.Chebyshev.derivative_U_eval_one_dvd RingTheory
Polynomial.Chebyshev.iterate_derivative_T_eval_one_dvd RingTheory
Polynomial.Chebyshev.iterate_derivative_U_eval_one_dvd RingTheory
Polynomial.IsEisensteinAt._root_.Polynomial.Monic.leadingCoeff_notMem RingTheory
Polynomial.IsPrimitive.mul_map_mem_lifts_iff RingTheory
Polynomial.IsRoot.map Algebra
Polynomial.Monic.isUnit_leadingCoeff_of_dvd Algebra
Polynomial.Sequence.linearIndependent Algebra
Polynomial.cyclotomic_injective RingTheory
Polynomial.cyclotomic_neg_one_pos RingTheory
Polynomial.dvd_term_of_dvd_eval_of_dvd_terms Algebra
Polynomial.le_rootMultiplicity_iff Algebra
Polynomial.le_rootMultiplicity_map Algebra
Polynomial.not_irreducible_C Algebra
Polynomial.taylor_mem_degreeLT RingTheory
Polynomial.valuation_le_one_of_valuation_X_le_one FieldTheory
PosSMulMono.of_smul_nonneg Algebra
PosSMulMono.toPosSMulStrictMono Algebra
PowerSeries.WithPiTopology.tendsto_trunc_atTop RingTheory
PowerSeries.rescale_injective RingTheory
Prime.coprime_iff_not_dvd RingTheory
Primrec.of_equiv_iff Computability
Primrec.of_equiv_symm_iff Computability
ProbabilityTheory.IsGaussian.integrable_exp_sq_of_conv_neg Probability
ProbabilityTheory.IsGaussian.memLp_dual Probability
ProbabilityTheory.integrable_map_cast_poissonMeasure_iff Probability
Prod._root_.WCovBy.fst Order
Prod._root_.WCovBy.snd Order
Prod.swap_covBy_swap Order
Prod.swap_wcovBy_swap Order
QuadraticForm.negSemidef_spanSubset LinearAlgebra
QuadraticForm.posDef_spanSubset LinearAlgebra
QuasispectrumRestricts.of_subset_range_algebraMap Algebra
Rat.cast_pos_of_pos Data
Real.fourierIntegral_convergent_iff' Analysis
Representation.apply_sub_id_partialSum_eq RepresentationTheory
RingCon.listProd RingTheory
RingHom.SurjectiveOnStalks.tensorProductMap_id RingTheory
SMulPosMono.toSMulPosStrictMono Algebra
SMulPosReflectLT.toSMulPosReflectLE Algebra
Seminorm.smul_closedBall_subset Analysis
Set.IsPWO.mul Data
Set.IsPWO.submonoid_closure Algebra
Set.OrdConnected.domRestrict Order
Set.OrdConnected.starConvex Analysis
Set.PairwiseDisjoint.attach Data
Set.PairwiseDisjoint.image_finset_of_le Data
Set.add_mem_Icc_iff_left Algebra
Set.add_mem_Icc_iff_right Algebra
Set.inter_mul_union_subset Algebra
Set.inv_mem_Icc_iff Algebra
Set.list_prod_mem_list_prod Algebra
Set.mulIndicator_apply_le Algebra
Set.mulIndicator_le Algebra
Set.mulIndicator_le_self Algebra
Set.sub_mem_Icc_iff_left Algebra
Set.sub_mem_Icc_iff_right Algebra
Set.union_mul_inter_subset Algebra
SetLike.lt_iff_le_and_exists Data
SimpleGraph.extremalNumber_le_iff_of_nonneg Combinatorics
Squarefree.isRadical Algebra
StarConvex.neg Analysis
StrictConvex.neg Analysis
StrictMono.mul_monotone Algebra
StrictMonoOn.continuousWithinAt_right_of_exists_between Topology
StrongDual.polar_nonempty Analysis
StrongDual.zero_mem_polar Analysis
Subalgebra.LinearDisjoint.exists_field_of_isDomain_of_injective RingTheory
Submodule.FG.of_le_of_isNoetherian RingTheory
Submodule.IsLattice.of_le_of_isLattice_of_fg Algebra
Submodule.LinearDisjoint.linearIndependent_mul_of_flat_left LinearAlgebra
Submodule.LinearDisjoint.linearIndependent_mul_of_flat_right LinearAlgebra
Submodule.LinearDisjoint.of_le_left_of_flat LinearAlgebra
Submodule.LinearDisjoint.of_le_right_of_flat LinearAlgebra
Submodule._root_.Module.isTorsionBySet_annihilator_top Algebra
Submodule.isCompact_of_fg Topology
Submodule.isOpen_mono Topology
Submonoid.closure_mul_le Algebra
Submonoid.mem_nhds_one Topology
Submonoid.mul_subset Algebra
Submonoid.top_closure_mul_self_subset Topology
Summable.mul_norm Analysis
SummationFilter._root_.Summable.tendsto_zero_of_even_summable_symmetricIcc Topology
ThreeGPFree.image' Combinatorics
Topology.IsOpenEmbedding.irreducibleSpace Topology
TwoSidedIdeal.set_mul_subset RingTheory
TwoSidedIdeal.subset_mul_set RingTheory
UniformContinuous.const_mul' Topology
UniformContinuous.mul_const' Topology
UniqueFactorizationMonoid.hasFiniteMulSupport_fun_pow_multiplicity RingTheory
Unitization.isStarProjection_inr_iff Algebra
Unitization.norm_splitMul_snd_sq Analysis
UpperHalfPlane.zero_form_isBoundedAtImInfty Analysis
UpperSet.infIrred_Ici Order
ValuationRing.le_total RingTheory
ValuativeRel.mul_vlt_mul RingTheory
ValuativeRel.mul_vlt_mul_of_vle_of_vlt RingTheory
ValuativeRel.mul_vlt_mul_of_vlt_of_vle RingTheory
ValuativeRel.pow_vle_pow RingTheory
Valued.valuation_isClosedMap Topology
WeakDual.exists_countable_separating Analysis
WeierstrassCurve.CXY AlgebraicGeometry
WellFoundedGT.induction_top Order
WithSeminorms.continuous_of_continuous_comp Analysis
WithSeminorms.firstCountableTopology Analysis
WithTop.le_of_untop$_0$_le_untop$_0$ Algebra
WithTop.untop$_0$_le_untop$_0$ Algebra
WithTop.untop$_0$_le_untop$_0$_iff Algebra
WithTop.untop$_0$_nonneg Algebra

WithZero.isOrderedAddMonoid Algebra
WithZero.le_log_iff_exp_le Algebra
WithZero.log_le_iff_le_exp Algebra
WithZero.log_lt_iff_lt_exp Algebra
WithZero.lt_log_iff_exp_lt Algebra
absConvexHull_add_subset Analysis
absorbent_nhds_zero Analysis
algebraMap_le_algebraMap Algebra
algebraMap_lt_algebraMap Algebra
algebraMap_mono Algebra
algebraMap_strictMono Algebra
ampleSet_univ Analysis
archimedean_iff_nat_lt Algebra
balancedCoreAux_balanced Analysis
balancedCoreAux_maximal Analysis
balancedCoreAux_subset Analysis
balancedHull.balanced Analysis
balancedHull_convexHull_subset_absConvexHull Analysis
balanced_zero Analysis
bddBelow_gauge_set Analysis
continuousAt_sign_of_neg Topology
continuousAt_sign_of_pos Topology
continuousWithinAt_leftLim_Iic Topology
continuousWithinAt_right_of_monotoneOn_of_exists_between Topology
controlled_prod_of_mem_closure_range Analysis
convex_Iic Analysis
convex_openSegment Analysis
convex_vadd Analysis
dirSupClosedOn_iff_forall_sSup Order
dirSupClosed_iff_forall_sSup Order
dirSupInaccOn_iff_forall_sSup Order
dirSupInacc_iff_forall_sSup Order
dist_birkhoffSum_birkhoffSum_le Dynamics
div_lt_arithGeom Analysis
div_neg_iff Algebra
div_nonneg_iff Algebra
div_nonpos_iff Algebra
div_pos_iff Algebra
eVariationOn.nonempty_monotone_mem Topology
edist_ne_top_of_mem_ball Topology
edist_pi_const_le Topology
exists_mem_ne_zero_of_rank_pos LinearAlgebra
exists_pow_lt Algebra
exists_rat_lt Algebra
exists_subset_iUnion_ball_radius_lt Topology
exists_subset_iUnion_ball_radius_pos_lt Topology
field_cond_of_four_le_card LinearAlgebra
finprod_le_finprod Algebra
finprod_le_finprod' Algebra
hasFDerivAtFilter_fst Analysis
hasFDerivAtFilter_snd Analysis
hasFDerivAt_iff_isLittleO_nhds_zero Analysis
hasFDerivWithinAt_comp_add_left Analysis
hasStrictFDerivAt_add_const_iff Analysis
has_fderiv_at_filter_real_equiv Analysis
iSupIndep.linearIndependent LinearAlgebra
iSup_le Order
infinite_range_add_smul_iff Algebra
inv_le_inv' Algebra
inv_le_one_of_one_le Algebra
isClosed_le_of_isClosed_nonneg Analysis
isClosed_setOfPred_blockTriangular Topology
isCompl_ofDual_iff Order
isCompl_toDual_iff Order
isIntegral_of_smul_mem_submodule RingTheory
isOpenMap_barycentric_coord Analysis
isOrderedModule_iff_algebraMap_mono Algebra
isQuotientCoveringMap_npow Analysis
le_egauge_smul_left Analysis
le_egauge_smul_right Analysis
le_gauge_of_subset_closedBall Analysis
le_of_smul_le_smul_of_neg Algebra
le_rpow_one_add_norm_iff_norm_le Analysis
linearIndependent_algHom_toLinearMap' LinearAlgebra
lt_tsub_iff_right Algebra
map_one_ne_zero Analysis
measurable_leOnePart MeasureTheory
measurable_nnnorm MeasureTheory
measurable_norm MeasureTheory
measurable_oneLePart MeasureTheory
mem_own_leftCoset GroupTheory
mem_own_rightCoset GroupTheory
mem_span_insert_exchange LinearAlgebra
mulIndicator_union_eventuallyEq Order
mul_le_mul_of_nonpos_left Algebra
mul_le_mul_of_nonpos_right Algebra
mul_lt_mul_of_neg_left Algebra
mul_lt_mul_of_neg_right Algebra
mul_nonneg_iff_of_pos_left Algebra
mul_nonneg_iff_of_pos_right Algebra
ne_zero_of_irreducible_X_pow_sub_C' FieldTheory
neg_of_smul_neg_left' Algebra
neg_of_smul_neg_right' Algebra
neg_one_lt_zero Algebra
nonneg_and_nonneg_or_nonpos_and_nonpos_of_mul_nonneg Algebra
nonneg_of_mul_nonneg_left Algebra
nonneg_of_mul_nonneg_right Algebra
nonpos_of_mul_nonneg_left Algebra
nonpos_of_mul_nonneg_right Algebra
nonpos_of_mul_nonpos_left Algebra
nonpos_of_mul_nonpos_right Algebra
nonzero_span_atom LinearAlgebra
not_ringChar_dvd_of_invertible Algebra
num_le_nat_mul_den Data
oneLePart_lt Algebra
one_add_norm_le_sqrt_two_mul_sqrt Analysis
one_le_inv_of_le_one Algebra
one_lt_oneLePart_iff Algebra
pow_div_pow_eventuallyEq_atBot Analysis
pow_div_pow_eventuallyEq_atTop Analysis
pow_le_pow_iff_right' Algebra
pow_lt_pow_iff_right' Algebra
riesz_lemma Analysis
sameRay_of_mem_orbit LinearAlgebra
sameRay_smul_smul_of_mul_nonneg LinearAlgebra
sbtw_iff_mem_image_Ioo_and_ne Analysis
seminormFromBounded_aux Analysis
smoothingSeminormSeq_exists_pnat Analysis
smoothingSeminormSeq_tendsto_aux Analysis
smul_dvd_smul Data
smul_le_smul_iff_of_neg_left Algebra
smul_le_smul_of_nonpos_left Algebra
smul_nonneg_of_nonpos_of_nonpos Algebra
spectrum.eventually_isUnit_resolvent Analysis
spectrum.hasDerivAt_resolvent_const_right Analysis
spectrum.isUnit_one_sub_smul_of_lt_inv_radius Analysis
spectrum.resolvent_isBigO_inv Analysis
sq_pos_of_pos Algebra
sqrt_one_add_norm_sq_le Analysis
squarefree_iff_emultiplicity_le_one Algebra
starConvex_iff_div Analysis
strictConvex_iff_div Analysis
strictMono_mul_left_of_pos Algebra
sub_one_lt Algebra
subset_balancedHull Analysis
summable_divisorsAntidiagonal_aux NumberTheory
summable_mul_prod_iff_summable_mul_sigma_antidiagonal Topology
summable_norm_pow_mul_geometric_div_one_sub NumberTheory
summable_norm_sum_mul_antidiagonal_of_summable_norm Analysis
summable_of_isBigO' Analysis
summable_of_isBigO_nat' Analysis
summable_sum_mul_antidiagonal_of_summable_mul Topology
tendsto_atTop_of_mapClusterPt Topology
tendsto_inv_nhdsGE Topology
tendsto_inv_nhdsLE Topology
tendsto_smul_comp_nat_floor_of_tendsto_nsmul Analysis
tendsto_zero_geometric_tsum_pnat NumberTheory
tendsto_zero_of_isBoundedUnder_smul_of_tendsto_cobounded Analysis
tsub_le_iff_left Algebra
wbtw_iff_of_le Analysis
zero_mem_lowerBounds_smoothingSeminormSeq_range Analysis
zpow_left_mono Algebra
zpow_right_mono Algebra